

Predicción del nivel mensual de operaciones aeroportuarias en Colombia mediante analítica de datos, meteorología aeronáutica y modelos de Machine Learning

Autores:

Mauricio David Correa Hernández
Alejandro Suarez Montoya
Sara Maria Cardona Villada
Veronica Zapata Vargas

Cursos:

Almacenamiento y Procesamiento de Grandes Datos
Aprendizaje Automático
Visualización de Datos

Programas:

Ingeniería de sistemas
Maestría en Ciencias de los Datos y Analítica

Profesores:

Edwin Nelson Montoya Múnera
Marco Tulio de La Hoz
Alberto Mauricio Arias Correa

Universidad EAFIT

Medellín, Colombia

Junio de 2026

Contenido.

Contenido.....	1
1. Introducción.....	5
2. Marco teórico y Referencias.....	6
2.1. Analítica aeroportuaria y unidad aeropuerto-mes.....	6
2.2. Datos abiertos aeronáuticos.....	6
2.3. Observaciones METAR e información meteorológica.....	7
2.4. Machine Learning supervisado y evaluación de clasificación.....	8
2.5. Ingeniería de datos y procesamiento distribuido.....	9
2.6. Arquitectura analítica en Databricks.....	10
3. Desarrollo metodológico de modelos de ML.....	11
3.1. Entendimiento del problema, pregunta de negocio o hipótesis.....	11
3.2. Análisis Exploratorio de Datos.....	14
3.2.1. Entendimiento de los datos.....	14
3.2.2. Preparación de los datos.....	16
3.2.3. Análisis descriptivo e insights importantes (correlación, causa-efecto, etc).....	22
3.3. Selección de modelos (supervisados y no supervisados), Ingeniería de Características, Entrenamiento, Evaluación.....	27
3.3.1. Modelos.....	27
3.3.2. Características e Ingeniería de Características.....	28
3.3.2.1. Variables temporales.....	30
3.3.2.2. Ratios operacionales.....	31
3.3.2.3. Variables meteorológicas derivadas.....	32
3.3.2.4. Rezagos temporales por aeropuerto.....	33
3.3.2.5. Promedios móviles.....	34
3.3.2.6. Cambios operacionales.....	35
3.3.2.7. Flags de disponibilidad histórica.....	35
3.3.2.8. Resultado de la ingeniería de características.....	36
3.3.2.9. Control de fuga de información.....	37
3.3.2.10. Síntesis de la ingeniería de características.....	38
3.3.3. Entrenamiento.....	38
3.3.3.1. Entrenamiento inicial de modelos base.....	38
3.3.3.2. Entrenamiento Base: Regresión Logística Multinomial (B01).....	39
Configuración corr_060_070.....	39
Configuración corr_060_080.....	39
Configuración corr_060_090.....	40
Comparación de Resultados.....	40
Análisis de resultados.....	41
3.3.3.3. Entrenamiento Base: Random Forest (M01).....	41

Hiperparámetros utilizados.....	41
Configuración corr_060_070.....	42
Configuración corr_060_080.....	42
Configuración corr_060_090.....	42
Comparación de resultados.....	43
Análisis de resultados.....	43
3.3.3.4. Entrenamiento Base: XGBoost (M02).....	44
Hiperparámetros utilizados.....	44
Configuración corr_060_070.....	45
Configuración corr_060_080.....	45
Configuración corr_060_090.....	45
Comparación de resultados.....	46
Análisis de resultados.....	46
3.3.3.5. Entrenamiento Base: LightGBM (M03).....	47
Hiperparámetros utilizados.....	47
Configuración corr_060_070.....	48
Configuración corr_060_080.....	48
Configuración corr_060_090.....	49
Comparación de resultados.....	49
Análisis de resultados.....	49
3.3.3.6. Entrenamiento Base: Extra Trees (M04).....	50
Hiperparámetros utilizados.....	50
Configuración corr_060_070.....	51
Configuración corr_060_080.....	51
Configuración corr_060_090.....	51
Comparación de resultados.....	52
Análisis de resultados.....	52
3.3.3.7. Entrenamiento Base: K-Nearest Neighbors (M05).....	53
Hiperparámetros utilizados.....	53
Configuración corr_060_070.....	54
Configuración corr_060_080.....	54
Configuración corr_060_090.....	55
Comparación de resultados.....	55
Análisis de resultados.....	55
3.3.3.8. Consolidación de Resultados de Validación.....	56
3.3.3.9. Mejores resultados por modelo.....	57
3.3.3.10. Ranking de modelos según Macro F1.....	57
3.3.3.11. Comparación de Accuracy y Balanced Accuracy.....	58
3.3.3.12. Comparación de tiempos de ejecución.....	58

3.3.3.13. Selección de modelos candidatos para optimización.....	59
3.3.3.14. Conclusiones de la fase de validación.....	59
3.3.4. Evaluación.....	60
3.3.4.1. Consolidación de resultados de validación.....	60
3.3.4.2. Comparación general de modelos.....	60
3.3.4.3. Análisis de desempeño por clases.....	61
3.3.5. Selección de modelos para optimización.....	61
3.3.5.1. Análisis del mejor modelo por configuración de variables.....	62
3.3.5.2. Interpretación de resultados.....	62
3.3.5.3. Conclusión.....	63
3.3.6. Optimización de Hiperparámetros con Optuna.....	63
3.3.6.1. Optimización de Random Forest.....	63
Resultados obtenidos.....	64
Análisis.....	64
3.3.6.2. Optimización de XGBoost.....	64
Resultados obtenidos.....	65
Comparación con la versión original.....	65
Análisis.....	65
3.3.6.3. Optimización de LightGBM.....	65
Resultados obtenidos.....	66
Comparación con la versión original.....	66
Análisis.....	66
Comparación final de los modelos optimizados.....	67
Conclusión.....	67
3.3.7. Validación Cruzada y Selección del Modelo Final.....	67
3.3.8. Evaluación Final del Modelo Seleccionado.....	69
3.3.9. Evaluación del Modelo Final en Conjunto de Prueba (Test).....	70
3.3.9.1. Resultados globales del modelo.....	71
3.3.9.2. Análisis de la Matriz de Confusión.....	71
3.3.10. Importancia de Variables.....	72
3.3.11. Análisis de Confianza de las Predicciones.....	73
3.3.12. Análisis de Errores.....	75
3.4. Análisis.....	76
3.4.1. Análisis de los Resultados.....	76
3.4.2. Cumplimiento de los Objetivos.....	77
4. TECNOLOGÍA: Ingeniería de Datos y uso de tecnología para:.....	77
4.1. Desarrollo del proyecto.....	77
4.2. Despliegue del proyecto (en un escenario hipotético de implementación real).....	78
4.3. Tener en cuenta:.....	78

4.3.1. Fuentes de datos y naturaleza (batch, streaming, transaccional, estructurada vs no estructurada, etc).....	78
4.3.2. Ingesta de datos (batch / Streaming).....	79
4.3.3. Almacenamiento.....	80
4.3.3.1. Lagos de datos.....	80
4.3.3.2. Bases de datos.....	80
4.3.3.3. Archivos.....	80
4.3.4. Ambiente de procesamiento (desde básicos hasta avanzados como Apache Spark).....	81
4.3.5. Aplicaciones (visualización, API, archivos, etc).....	81
5. Visualización y Comunicación de Datos.....	82
5.1. Requerimientos de Comunicación y Visualización.....	82
5.1.1. Audiencia objetivo.....	82
5.1.2. Pregunta de negocio.....	82
5.1.3. Requerimientos funcionales.....	82
5.1.4. Requerimientos no funcionales.....	83
5.2. Análisis.....	83
5.2.1. Análisis exploratorio visual previo.....	83
5.2.2. Decisiones de diseño derivadas del análisis.....	84
5.3. Diseño.....	84
5.3.1. Principios de diseño aplicados.....	84
5.3.2. Paleta de colores.....	85
5.3.3. Arquitectura de la información.....	86
5.3.4. Tipo de dashboard.....	86
5.4. Implementación.....	87
5.4.1. Tecnología seleccionada.....	87
5.4.2. Estructura modular del código.....	87
5.4.3. Decisiones técnicas relevantes.....	89
5.4.4. Interactividad implementada.....	90
5.5. Validación.....	91
5.5.1. Pruebas realizadas.....	92
5.5.2. Limitaciones conocidas.....	92
6. Conclusiones generales del Proyecto.....	92
6.1. Conclusiones.....	92
6.2. Limitaciones del proyecto.....	95
6.3. Trabajo futuro.....	95
7. Referencias.....	97

1. Introducción.

El transporte aéreo colombiano es un sistema de alta relevancia económica, territorial y logística. La actividad de los aeropuertos no depende únicamente del volumen histórico de operaciones, sino también de la conectividad comercial, la demanda de pasajeros, la carga, la ubicación geográfica, la estacionalidad y las condiciones meteorológicas que pueden incidir en la regularidad operativa. En este contexto, el presente proyecto propone una solución analítica orientada a estudiar el comportamiento mensual de los aeropuertos colombianos y a construir modelos de Machine Learning para anticipar el nivel de operación del mes siguiente.

La unidad de análisis definida para el proyecto es el par aeropuerto-mes. Cada registro consolida información mensual asociada a un aeropuerto identificado por código ICAO, integrando datos de operaciones aéreas, tráfico origen-destino, variables meteorológicas provenientes de observaciones METAR históricas y atributos maestros del aeropuerto. Esta integración permite convertir datos operacionales dispersos en un conjunto analítico estructurado, apto para análisis exploratorio, modelado supervisado y visualización.

El objetivo general del proyecto es clasificar el nivel esperado de operaciones del siguiente mes para cada aeropuerto colombiano en categorías ordinales bajo, medio y alto. Esta clasificación busca apoyar procesos de planeación, identificación de patrones operativos y comunicación analítica para usuarios no técnicos. Adicionalmente, durante el desarrollo se exploraron visualizaciones orientadas a detectar aeropuertos con alta demanda relativa y baja competencia, entendida de forma preliminar como una combinación de pasajeros por vuelo y número de aerolíneas presentes.

El proyecto se desarrolló siguiendo una lógica cercana a CRISP-DM, metodología ampliamente utilizada en proyectos de minería de datos y analítica aplicada. En particular, se abordaron las fases de entendimiento del negocio, entendimiento de los datos, preparación, modelado, evaluación y comunicación de resultados. Desde la perspectiva tecnológica, se estructuró una arquitectura de datos por capas, con zonas bronze, silver y gold, y se migraron componentes de ingesta y preparación a PySpark para alinear el trabajo con prácticas de ingeniería de datos escalable.

El documento se organiza en siete partes. En primer lugar, se presenta la introducción general del proyecto, su contexto, objetivo y alcance. En segundo lugar, se desarrolla el marco teórico y las referencias que sustentan los conceptos de transporte aéreo, analítica de datos, meteorología aeronáutica, ingeniería de datos y Machine Learning. En tercer lugar, se expone el desarrollo metodológico de los modelos de Machine Learning, incluyendo el entendimiento del problema, el análisis exploratorio de datos, la preparación del conjunto analítico, la selección de modelos, la ingeniería de características, el entrenamiento, la evaluación y el análisis de resultados. En cuarto lugar, se describe la dimensión tecnológica del proyecto, abordando las fuentes de datos, la ingesta, el almacenamiento, el procesamiento y un escenario hipotético de despliegue. En quinto lugar, se presenta el componente de visualización y comunicación de datos,

con énfasis en el diseño e implementación de una herramienta interactiva orientada a usuarios no técnicos. Finalmente, se incluyen las conclusiones generales del proyecto y las referencias utilizadas.

2. Marco teórico y Referencias.

2.1. Analítica aeroportuaria y unidad aeropuerto-mes

La analítica aeroportuaria puede entenderse como el uso de datos operacionales, comerciales, geográficos y ambientales para describir, explicar o anticipar el comportamiento de un sistema de transporte aéreo. En este proyecto, el análisis no se realiza a nivel de vuelo individual, sino a nivel agregado mensual. Esta decisión responde a la disponibilidad de los datos abiertos consultados, a la estructura de las fuentes oficiales utilizadas y a la necesidad de construir una base comparable entre aeropuertos, meses y variables provenientes de diferentes fuentes.

La unidad de análisis definida para el proyecto es aeropuerto-mes. Esta unidad permite estudiar variaciones temporales y diferencias territoriales dentro del sistema aeroportuario colombiano. Por ejemplo, un mismo aeropuerto puede presentar meses de mayor operación por efectos de estacionalidad turística, conectividad regional, dinámica comercial, disponibilidad de rutas o condiciones particulares del mercado. Al mismo tiempo, aeropuertos con menor volumen absoluto pueden ser relevantes si muestran señales de alta demanda relativa, baja competencia o comportamiento operativo diferencial.

Esta decisión metodológica evita limitar el análisis únicamente a los aeropuertos de mayor tamaño. En cambio, permite conservar aeropuertos regionales con cobertura suficiente de datos, lo cual es importante para un país con alta diversidad territorial y con aeropuertos que cumplen funciones distintas: conexión turística, integración regional, transporte comercial, operación institucional o movilidad en zonas de difícil acceso. Por esta razón, el proyecto se enfoca en construir un dataset mensual que permita comparar aeropuertos bajo una misma estructura analítica, pero sin perder de vista que cada aeropuerto puede tener una dinámica histórica propia.

2.2. Datos abiertos aeronáuticos

El proyecto utiliza datos abiertos como base principal para la construcción del dataset analítico. La fuente de operaciones aéreas acumuladas de la Aeronáutica Civil de Colombia, disponible en el portal datos.gov.co, consolida información de operaciones registradas en aeródromos colombianos, incluyendo dimensiones como mes, aeropuerto, tipo de operación, origen, destino y tipo de tráfico [2]. Esta fuente es relevante porque permite caracterizar la actividad operacional de los aeropuertos en el tiempo y sirve como eje principal para construir el comportamiento mensual de operaciones.

Adicionalmente, se emplea el conjunto de datos de transporte aéreo comercial origen-destino, también disponible en datos.gov.co, el cual contiene información estadística sobre trayectos del transporte aéreo comercial en Colombia [3]. Esta fuente permite complementar la visión operacional con variables asociadas a movilidad, pasajeros, carga, empresas y destinos. En conjunto, las fuentes de Aerocivil permiten construir indicadores mensuales relacionados con volumen de operaciones, pasajeros transportados, carga movilizadora, conectividad y actividad comercial.

Estos conjuntos de datos son de naturaleza estructurada, tabular y batch. Es decir, no corresponden a flujos de datos en tiempo real, sino a archivos o recursos consultables periódicamente. Esta característica es coherente con el enfoque del proyecto, ya que el objetivo no es predecir eventos minuto a minuto, sino construir una base histórica mensual para análisis, visualización y modelado.

Como fuente maestra de aeropuertos se empleó OurAirports, que publica archivos CSV con información de aeropuertos, países y regiones [4]. Esta fuente permitió disponer de variables de ubicación y atributos básicos como tipo de aeropuerto, municipio, región, código IATA, latitud, longitud, elevación y servicio programado. La integración de esta fuente fue importante para normalizar códigos, filtrar aeropuertos colombianos y enriquecer el análisis con información geográfica.

2.3. Observaciones METAR e información meteorológica

Para representar condiciones meteorológicas se trabajó con observaciones METAR históricas. El Iowa Environmental Mesonet mantiene un archivo de observaciones aeroportuarias automatizadas, comúnmente asociadas a sistemas ASOS, AWOS o, de forma general, datos METAR, y ofrece documentación para solicitar dichos datos mediante su servicio 'asos.py' [5], [6]. En el proyecto se priorizó esta fuente histórica porque permite obtener registros meteorológicos observados para aeropuertos específicos y agregarlos posteriormente a nivel mensual.

En el desarrollo del proyecto se descartó el uso de TAF como fuente principal debido a su baja cobertura respecto al objetivo analítico definido. Mientras que METAR corresponde a observaciones meteorológicas reportadas, TAF corresponde a pronósticos aeronáuticos. Dado que el proyecto requiere construir variables históricas consistentes por aeropuerto y mes, la fuente METAR resultó más adecuada para el dataset final.

Las variables meteorológicas agregadas a nivel mensual incluyen medidas como temperatura, punto de rocío, humedad relativa, velocidad del viento, visibilidad, ráfagas, nubosidad y altura de capa nubosa cuando estaban disponibles. Estas variables se utilizaron como covariables contextuales. Por tanto, no deben interpretarse automáticamente como causas directas de variación operacional. Su

función dentro del proyecto es representar condiciones ambientales que podrían estar asociadas con patrones de operación, regularidad, conectividad o comportamiento regional.

Es importante señalar que el análisis desarrollado no corresponde a un estudio de inferencia causal. Por esta razón, cuando se identifiquen relaciones entre clima y operación aeroportuaria, dichas relaciones deben interpretarse como asociaciones estadísticas o patrones exploratorios. Para afirmar causalidad sería necesario diseñar una estrategia metodológica distinta, por ejemplo, con variables instrumentales, diseños cuasi-experimentales, modelos de efectos fijos, análisis contrafactual o técnicas específicas de inferencia causal.

2.4. Machine Learning supervisado y evaluación de clasificación

El enfoque central del proyecto es supervisado porque se define una variable objetivo derivada del comportamiento futuro de las operaciones aeroportuarias. El objetivo numérico base es `'target_operaciones_total_mes_siguiente'`, que representa el total de operaciones del aeropuerto en el mes siguiente. A partir de esta variable se construye una etiqueta ordinal de nivel operativo: bajo, medio o alto.

La construcción de la etiqueta bajo/medio/alto se realiza comparando cada aeropuerto contra su propia distribución histórica. Esta decisión es importante porque evita que todos los aeropuertos pequeños queden clasificados como bajo y todos los aeropuertos grandes como alto. En otras palabras, el objetivo no es comparar únicamente el tamaño absoluto de los aeropuertos, sino identificar si un aeropuerto se encuentra en un nivel operativo bajo, medio o alto respecto a su propio comportamiento histórico.

Este planteamiento convierte el problema en una tarea de clasificación supervisada ordinal. Aunque la variable objetivo tiene un orden natural, los modelos pueden abordarse inicialmente como clasificadores multiclase, dejando como posible mejora futura el uso de técnicas específicas para clasificación ordinal. En este contexto, el aprendizaje supervisado busca encontrar patrones en las variables históricas de operación, tráfico, conectividad, atributos aeroportuarios y clima que ayuden a anticipar el nivel operativo del mes siguiente.

En clasificación, la evaluación debe considerar métricas acordes con el tipo de problema. La documentación de scikit-learn incluye métricas como exactitud (accuracy), precisión, recall, F1-score, matriz de confusión, ROC-AUC y métricas basadas en curvas precision-recall para evaluar clasificadores según etiquetas, probabilidades o puntajes de decisión [7], [8], [9]. En este proyecto, estas métricas deben interpretarse con especial cuidado cuando exista desbalance de clases o cuando se derive una tarea binaria auxiliar de baja prevalencia.

La exactitud puede ser útil como medida general, pero puede resultar insuficiente si las clases no están balanceadas. Por esta razón, métricas como F1-score macro,

precision, recall y matriz de confusión permiten entender mejor el desempeño por clase. En particular, para un problema con categorías bajo, medio y alto, no basta con saber cuántas predicciones fueron correctas en total; también es necesario revisar si el modelo logra identificar adecuadamente los meses de alta operación, ya que estos pueden ser los más relevantes para la toma de decisiones.

2.5. Ingeniería de datos y procesamiento distribuido

La ingeniería de datos del proyecto se organizó mediante una arquitectura por capas. La capa bronze conserva los datos descargados desde las fuentes originales; la capa silver contiene datos normalizados, limpiados, tipificados y agregados parcialmente; y la capa gold contiene el dataset final preparado para análisis exploratorio, visualización y modelado. Esta separación reduce errores, mejora la reproducibilidad y permite auditar las transformaciones aplicadas sobre los datos.

Esta estructura es coherente con el enfoque de arquitectura medallón utilizado en entornos lakehouse. En este patrón, las capas iniciales conservan datos con menor nivel de procesamiento, mientras que las capas posteriores incrementan la calidad, estandarización y utilidad analítica de los datos. En el contexto del proyecto, bronze equivale a la zona de datos originales, silver a la zona de datos depurados y normalizados, y gold a la zona de datos listos para consumo analítico.

Para el procesamiento de datos se incorporó PySpark dentro de Databricks. Apache Spark define los DataFrames como una abstracción para procesamiento relacional y distribuido, organizada en columnas con nombre y compatible con operaciones de Spark SQL. Esta abstracción permite trabajar con archivos estructurados, aplicar transformaciones sobre grandes volúmenes de datos y construir pipelines de preparación de datos de forma escalable [10].

El uso de Databricks permitió centralizar el desarrollo del proyecto en notebooks, ejecutar transformaciones con PySpark y organizar el flujo de trabajo en etapas reproducibles. En este proyecto, Databricks se entiende como el entorno principal de desarrollo y procesamiento. Por tanto, la tecnología base del proyecto corresponde a Databricks, PySpark y archivos estructurados en las capas bronze, silver y gold.

Desde el punto de vista metodológico, esta decisión facilita la trazabilidad del pipeline. Cada etapa del proceso puede documentarse, ejecutarse y validarse dentro del mismo entorno: descarga o lectura de datos, normalización de códigos aeroportuarios, agregación mensual, integración entre fuentes, tratamiento de valores faltantes, construcción de variables objetivo y generación del dataset final. Esta organización también permite separar claramente el trabajo de ingeniería de datos del trabajo posterior de modelado y visualización.

2.6. Arquitectura analítica en Databricks

Como escenario tecnológico del proyecto, se plantea una arquitectura analítica basada en Databricks. Esta arquitectura no se formula como un despliegue productivo en AWS, sino como una implementación académica y funcional dentro de un entorno de notebooks, procesamiento distribuido y almacenamiento por capas. En este sentido, Databricks cumple el papel de plataforma central para el desarrollo, procesamiento, integración y preparación de datos.

La arquitectura propuesta se apoya en el enfoque lakehouse, donde los datos se organizan por niveles de calidad y preparación. En la capa bronze se almacenan los datos originales provenientes de las fuentes abiertas. En la capa silver se guardan datos normalizados, depurados y parcialmente agregados. En la capa gold se consolida el dataset final aeropuerto-mes y otros archivos generados en la etapa de modelado, listo para análisis exploratorio, entrenamiento de modelos y construcción de visualizaciones.

En una implementación real dentro de Databricks, el almacenamiento podría apoyarse en Delta Lake para mejorar confiabilidad, control transaccional, manejo de metadatos y evolución de esquemas. Sin embargo, para el alcance trabajado en el proyecto, el énfasis está en el uso de Databricks como ambiente de desarrollo y PySpark como motor de procesamiento. Por esta razón, la arquitectura debe describirse como una solución basada en Databricks y no como una arquitectura de servicios AWS.

El flujo general de la arquitectura puede resumirse de la siguiente manera: primero, las fuentes abiertas se descargan o consultan en modo batch; segundo, los datos se almacenan en la capa bronze; tercero, se aplican procesos de limpieza, normalización y validación en la capa silver; cuarto, se integran las fuentes por aeropuerto y mes para construir la capa gold; finalmente, el dataset gold se utiliza para modelado de Machine Learning, análisis exploratorio y visualización de resultados.

Esta arquitectura es adecuada para el proyecto porque permite trabajar con datos estructurados, históricos y periódicos. Además, facilita la reproducibilidad del análisis, ya que las transformaciones pueden organizarse en notebooks y ejecutarse por etapas. En un escenario de implementación real, la solución podría evolucionar hacia jobs programados en Databricks, tablas Delta, control de versiones de datos, monitoreo de calidad, catálogos de datos y dashboards conectados al dataset gold.

3. Desarrollo metodológico de modelos de ML.

3.1. Entendimiento del problema, pregunta de negocio o hipótesis

El sistema aeroportuario colombiano presenta una dinámica operacional heterogénea: existen aeropuertos con altos volúmenes de operación comercial y de pasajeros, aeropuertos regionales con comportamientos estacionales, terminales con baja frecuencia de vuelos, y aeropuertos que cumplen funciones estratégicas de conectividad territorial. Esta diversidad hace que el análisis aeroportuario no pueda limitarse únicamente a identificar cuáles aeropuertos tienen mayor número absoluto de operaciones, ya que un aeropuerto pequeño puede presentar meses de alta actividad respecto a su propio comportamiento histórico, mientras que un aeropuerto grande puede tener variaciones relevantes aunque conserve un volumen alto en términos absolutos.

El problema abordado en este proyecto consiste en construir un sistema analítico que permita clasificar el nivel esperado de operación mensual de los aeropuertos colombianos, utilizando información histórica de operaciones aéreas, tráfico origen-destino, características geográficas y variables meteorológicas. Para ello, se definió como unidad de análisis el par aeropuerto-mes, lo que permite observar el comportamiento de cada aeropuerto a lo largo del tiempo y comparar patrones mensuales entre aeropuertos con diferentes tamaños, ubicaciones y niveles de conectividad.

Desde el punto de vista de negocio, el problema puede formularse como la necesidad de anticipar el nivel operativo de un aeropuerto para el mes siguiente. Esta información puede ser útil para apoyar decisiones relacionadas con planeación aeroportuaria, análisis de conectividad, identificación de aeropuertos con oportunidad de crecimiento, priorización de recursos, seguimiento de demanda regional y comunicación de hallazgos mediante visualizaciones de datos. Aunque el proyecto tiene un alcance académico, el planteamiento reproduce una situación realista: una entidad aeroportuaria, aerolínea, analista sectorial o institución pública podría requerir una herramienta que le ayude a identificar aeropuertos con comportamiento operativo bajo, medio o alto en el corto plazo.

La pregunta principal del proyecto se plantea de la siguiente manera:

¿Es posible clasificar el nivel esperado de operación mensual de un aeropuerto colombiano para el mes siguiente, en categorías bajo, medio y alto, a partir de datos históricos de operaciones, tráfico comercial, atributos aeroportuarios y variables meteorológicas agregadas mensualmente?

Esta pregunta se traduce en un problema de Machine Learning supervisado. La variable objetivo numérica base corresponde a `target_operaciones_total_mes_siguiente`, la cual representa el total de operaciones del aeropuerto en el mes posterior al observado. A partir de esta variable se

construye una etiqueta categórica ordinal con tres niveles: *bajo*, *medio* y *alto*. Esta etiqueta no se define de manera global para todos los aeropuertos, sino en función de la distribución histórica de cada aeropuerto. Esta decisión metodológica es importante porque evita que el modelo clasifique automáticamente a los aeropuertos grandes como “*altos*” y a los aeropuertos pequeños como “*bajos*”. En cambio, permite evaluar si un aeropuerto se encuentra en un nivel operativo bajo, medio o alto respecto a su propio comportamiento histórico.

La hipótesis general del proyecto es que el comportamiento operativo mensual de un aeropuerto no depende únicamente de su volumen histórico de operaciones, sino también de variables asociadas a conectividad, tráfico comercial, ubicación, características del aeropuerto y condiciones meteorológicas. Por tanto, se espera que la integración de múltiples fuentes de datos permita construir un conjunto de variables suficientemente informativo para clasificar el nivel operativo esperado del mes siguiente.

De manera más específica, se plantean las siguientes hipótesis analíticas:

Hipótesis 1. Los patrones históricos de operaciones son variables relevantes para anticipar el nivel operativo del mes siguiente. Es decir, aeropuertos con tendencias recientes de aumento, disminución o estabilidad en sus operaciones pueden conservar señales predictivas útiles.

Hipótesis 2. Las variables de tráfico comercial, como pasajeros, carga, destinos, empresas y registros de tráfico, aportan información complementaria para explicar diferencias entre aeropuertos y variaciones mensuales en la actividad aeroportuaria.

Hipótesis 3. Las características propias del aeropuerto, como ubicación geográfica, tipo de aeropuerto, región, elevación y servicio programado, ayudan a contextualizar el comportamiento operacional y pueden mejorar la capacidad descriptiva del dataset.

Hipótesis 4. Las variables meteorológicas agregadas mensualmente pueden estar asociadas con el comportamiento operativo, especialmente en aeropuertos donde las condiciones ambientales influyen en la regularidad, visibilidad, operación regional o conectividad. Sin embargo, estas variables se interpretan como covariables contextuales y no como causas directas, ya que el proyecto no desarrolla un diseño de inferencia causal.

El alcance del problema se limita al análisis histórico mensual de aeropuertos colombianos con cobertura suficiente de datos. Para ello, se integraron fuentes abiertas y estructuradas: información de aeropuertos desde OurAirports, operaciones aéreas acumuladas de Aerocivil, tráfico origen-destino de Aerocivil y observaciones meteorológicas históricas METAR desde IEM. La fuente AviationWeather fue revisada en etapas previas, pero para la versión final del

dataset histórico se priorizó IEM y se descartó TAF por baja cobertura frente al objetivo analítico.

El proyecto no busca predecir vuelos individuales, retrasos específicos, ocupación real por vuelo ni demanda en tiempo real. Tampoco pretende construir un sistema operacional productivo para toma de decisiones críticas. El objetivo es construir una base analítica reproducible, organizada en capas bronze, silver y gold, que permita realizar análisis exploratorio, visualización y modelado supervisado sobre el nivel operativo mensual de los aeropuertos.

Desde la perspectiva de datos, el problema también implica varios retos metodológicos.

El primero es la integración de fuentes con granularidades distintas, ya que las operaciones, el tráfico comercial, los atributos aeroportuarios y las observaciones meteorológicas no tienen la misma estructura original. Por esta razón, fue necesario normalizar códigos aeroportuarios, agregar información a nivel mensual y consolidar una llave común basada en aeropuerto y mes.

El segundo reto corresponde al manejo de valores faltantes. No todos los aeropuertos cuentan con la misma cobertura meteorológica ni con el mismo nivel de información en todas las fuentes. Para evitar una reducción excesiva del dataset, el tratamiento de datos faltantes debe diferenciar entre ausencia estructural de información, falta parcial de registros y variables no aplicables. En particular, la cobertura METAR se utilizó como criterio importante para decidir qué aeropuertos podían permanecer en el dataset final.

El tercer reto es evitar fuga de información o data leakage. Debido a que la variable objetivo representa el nivel operativo del mes siguiente, las variables predictoras deben construirse únicamente con información disponible hasta el mes observado. Por tanto, no deben incluirse columnas que contengan directamente el valor futuro que se pretende predecir, ni variables derivadas de la etiqueta objetivo. Esta validación es fundamental para que la evaluación del modelo sea metodológicamente válida.

Finalmente, el proyecto se entiende como un ejercicio integral de analítica de datos, ingeniería de datos, Machine Learning y visualización. El valor del trabajo no está únicamente en entrenar modelos, sino en construir un pipeline ordenado y justificable: obtención de fuentes abiertas, almacenamiento en capa bronze, limpieza y normalización en silver, construcción del dataset final en gold, análisis exploratorio, definición de variable objetivo, entrenamiento de modelos y comunicación de resultados. Desde esta perspectiva, el entendimiento del problema guía todas las decisiones posteriores del proyecto, desde la selección de variables hasta la forma de evaluar y presentar los hallazgos.

3.2. Análisis Exploratorio de Datos

3.2.1. Entendimiento de los datos

Durante el proyecto se integraron fuentes de naturaleza heterogénea, pero todas fueron llevadas a estructuras tabulares mensuales. Las fuentes principales fueron: maestro de aeropuertos de OurAirports; operaciones aéreas acumuladas de Aerocivil; tráfico origen-destino de Aerocivil; y observaciones meteorológicas históricas METAR desde IEM. La fuente AviationWeather fue revisada durante etapas previas, pero para la versión final del dataset histórico se priorizó IEM y se descartó TAF por baja cobertura.

Tabla 1. Fuentes de datos y naturaleza dentro del proyecto

Fuente	Naturaleza	Uso dentro del proyecto	Estado trabajado
OurAirports Colombia	CSV estructurado, batch	Dimensión maestra de aeropuertos: códigos, ubicación y atributos básicos.	Usado para construir dim_airports_colombia_skxx.parquet.
Aerocivil - Operaciones aéreas acumuladas	Datos abiertos estructurados, batch	Operaciones mensuales por aeropuerto y variables operacionales.	Usado para construir agregados mensuales de operaciones.
Aerocivil - Tráfico origen-destino	Datos abiertos estructurados, batch	Pasajeros, carga, destinos, empresas y registros de tráfico comercial.	Usado para construir od_airport_month.parquet.
IEM ASOS/METAR	Datos meteorológicos históricos, batch vía servicio web	Variables meteorológicas mensuales por aeropuerto.	Usado para construir weather_monthly_iem.parquet.

Los conteos consolidados trabajados durante el desarrollo fueron los siguientes. Estos valores deben conservarse como trazabilidad del pipeline y verificarse de nuevo en la versión final del notebook antes de entregar el informe definitivo.

Tabla 2. Artefactos y dataset.

Artefacto o dataset	Conteo trabajado	Comentario
dim_airports_colombia_skxx.parquet	146 aeropuertos SKXX válidos	Dimensión de aeropuertos filtrada para Colombia.
tráfico bronze	455.787 filas	Registros originales de tráfico antes de agregación final.

Artefacto o dataset	Conteo trabajado	Comentario
operaciones bronze	550.724 filas	Registros originales de operaciones antes de agregación final.
operations_airport_month.parquet	8.673 filas	Agregado mensual de operaciones por aeropuerto.
od_airport_month.parquet	5.898 filas	Agregado mensual de tráfico origen-destino por aeropuerto.
weather_monthly_iem.parquet	2.495 filas	Agregado meteorológico mensual para 46 aeropuertos.
monthly_airport_dataset.parquet	3.300 filas y 63 columnas	Dataset gold integrado.
monthly_airport_model_dataset.parquet	3.254 filas	Dataset filtrado para modelado.
Aeropuertos finales	46 aeropuertos	Aeropuertos con cobertura mensual válida para el análisis final.



Figura 1. Registros aeropuerto-mes por año. Permite validar la continuidad temporal del dataset final y observar la reducción de 2025 por corte de disponibilidad de datos.

Interpretación: el dataset mantiene una cobertura anual estable entre 2020 y 2024. La menor cantidad de registros en 2025 debe explicarse como consecuencia del corte temporal de los datos, no como error metodológico.

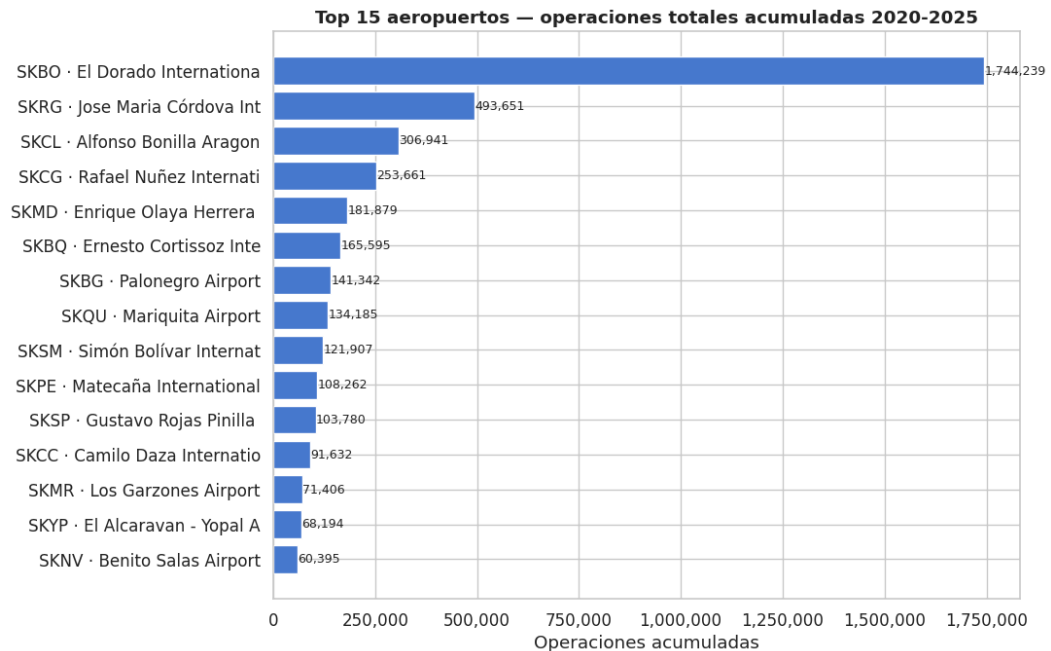


Figura 2. Top 15 aeropuertos por operaciones acumuladas 2020-2025. Muestra la concentración operacional en los principales aeropuertos del país.

Interpretación: SKBO concentra el mayor volumen de operaciones, seguido por aeropuertos principales como SKRG, SKCL, SKCG y SKMD. Esta figura justifica complementar el análisis absoluto con indicadores relativos por aeropuerto.

3.2.2. Preparación de los datos.

La preparación de los datos fue una de las etapas centrales del proyecto, debido a que las fuentes utilizadas tenían estructuras, niveles de granularidad, nombres de variables y códigos aeroportuarios diferentes. Aunque todas las fuentes eran estructuradas y de naturaleza tabular, no estaban listas para ser integradas directamente en un único dataset analítico. Por esta razón, fue necesario diseñar un proceso de preparación por capas, orientado a conservar la trazabilidad de los datos originales, mejorar la calidad de la información y construir una tabla final consistente a nivel aeropuerto-mes.

El proceso se organizó bajo una arquitectura de datos tipo medallón, compuesta por las capas bronze, silver y gold. En la capa bronze se conservaron los archivos descargados desde las fuentes originales, sin aplicar transformaciones analíticas profundas. Esta capa permitió mantener una copia de referencia de los datos tal como fueron obtenidos desde OurAirports, Aerocivil e IEM. En la capa silver se realizaron procesos de limpieza, normalización, tipificación, filtrado geográfico, estandarización de códigos y agregaciones preliminares. Finalmente, en la capa gold se consolidó el dataset mensual final, integrando información de operaciones, tráfico origen-destino, clima y dimensión maestra de aeropuertos.

La primera etapa de preparación consistió en construir una dimensión maestra de aeropuertos colombianos. Para ello, se utilizó la información de OurAirports y se filtraron los aeropuertos asociados a Colombia. Posteriormente, se priorizaron los aeropuertos con código ICAO colombiano válido, es decir, aquellos identificados bajo el patrón SKXX. Esta decisión fue importante porque las demás fuentes podían contener referencias a aeropuertos mediante códigos IATA, nombres escritos de forma no homogénea o descripciones textuales, lo que dificultaba la unión directa entre tablas. La dimensión maestra permitió centralizar atributos como `icao_code`, `iata_code`, `name`, `municipality`, `region_name`, `latitude_deg`, `longitude_deg`, `elevation_ft`, `type` y `scheduled_service`.

Una transformación crítica fue la normalización de códigos aeroportuarios. Los datos provenientes de Aerocivil y de las fuentes meteorológicas no siempre utilizan el mismo identificador. En algunos casos, los registros pueden venir asociados a códigos IATA; en otros, a códigos ICAO; y en otros, a nombres o campos textuales que requieren limpieza previa. Por esta razón, se trabajaron funciones de normalización como `code_to_icao_expr`, `normalizar_codigo_expr` y `es_codigo_skxx_valido_expr`. Estas funciones permitieron transformar códigos a un formato común, validar si correspondían a aeropuertos colombianos SKXX y reducir errores en las uniones posteriores. Sin esta normalización, el dataset final habría tenido duplicados, pérdidas de registros o asociaciones incorrectas entre aeropuertos.

Después de construir la dimensión de aeropuertos, se preparó la información de operaciones aéreas. Esta fuente fue agregada a nivel mensual por aeropuerto, utilizando como llave principal `icao_code` y `fecha_mes`. En esta etapa se consolidaron variables como operaciones de llegada, operaciones de salida, otras operaciones y operaciones totales. El propósito fue transformar una fuente operacional con mayor detalle en una tabla mensual comparable entre aeropuertos. De esta manera, cada fila representaba el comportamiento operativo de un aeropuerto en un mes determinado.

De forma complementaria, se preparó la fuente de tráfico origen-destino de Aerocivil. Esta fuente permitió construir variables relacionadas con demanda y conectividad, tales como pasajeros de salida, carga de salida en kilogramos, número de destinos, número de empresas y número de registros mensuales de tráfico comercial. Al igual que con operaciones, esta información se agregó a nivel aeropuerto-mes. La integración de esta fuente fue importante porque las operaciones por sí solas no describen completamente la dinámica aeroportuaria. Dos aeropuertos pueden tener un número similar de operaciones, pero diferenciarse en cantidad de pasajeros, volumen de carga, número de rutas o cantidad de empresas operadoras.

La preparación de la información meteorológica requirió un tratamiento particular. Para esta parte del proyecto se priorizaron observaciones históricas

METAR desde IEM, ya que esta fuente permitió construir variables observadas y agregadas mensualmente. La fuente AviationWeather fue revisada en etapas previas, pero para la versión final del dataset histórico se priorizó IEM y se descartó TAF por baja cobertura frente al objetivo del proyecto. En la versión trabajada se aplicó un criterio mínimo de cobertura con `MIN_IEM_ROWS = 500`, con el fin de conservar únicamente aeropuertos con suficiente información meteorológica histórica.

A partir de los registros METAR se construyeron variables mensuales como número de observaciones METAR disponibles en el mes, temperatura media, punto de rocío medio, humedad relativa media, velocidad media del viento, visibilidad media, ráfaga media, modo de cobertura nubosa y altura media de la primera capa nubosa cuando estaba disponible. Estas variables fueron agregadas a nivel aeropuerto-mes para que pudieran integrarse con las fuentes operacionales y de tráfico. En este punto fue necesario considerar que no todos los aeropuertos tenían la misma cobertura meteorológica. Por ello, se diferenciaron tres grupos: aeropuertos con cobertura completa, aeropuertos con cobertura parcial e imputación posible, y aeropuertos sin cobertura meteorológica útil. Los aeropuertos sin cobertura suficiente fueron excluidos del dataset final para evitar introducir ruido excesivo o variables meteorológicas completamente vacías.

El tratamiento de valores faltantes fue abordado de forma diferenciada según el tipo de variable. En las variables meteorológicas, la ausencia de registros podía reflejar falta de cobertura de la estación y no necesariamente un valor físico igual a cero. Por tanto, no se asumió que los valores faltantes fueran equivalentes a ausencia del fenómeno. En cambio, se revisó la cobertura disponible por aeropuerto y se conservó información auxiliar como `n_metar_mes`, que permite identificar cuántas observaciones respaldan los agregados meteorológicos mensuales. Para variables operacionales y de tráfico, el tratamiento dependió del significado del campo: algunos valores ausentes podían indicar ausencia de registros en la fuente para un aeropuerto-mes específico, mientras que otros requerían conservarse como faltantes para no inventar información.

La construcción del dataset final se realizó mediante uniones sucesivas usando las llaves `icao_code` y `fecha_mes`. Primero, se consolidó la tabla de operaciones mensuales. Luego, se integró el agregado mensual de tráfico origen-destino. Posteriormente, se incorporó la meteorología mensual proveniente de IEM. Finalmente, se adicionaron los atributos estáticos de la dimensión maestra de aeropuertos. Esta secuencia permitió mantener una lógica clara: las operaciones funcionaron como eje principal de análisis; el tráfico complementó la dimensión de demanda y conectividad; la meteorología agregó contexto ambiental; y la dimensión de aeropuertos aportó atributos geográficos y descriptivos.

En la versión trabajada del proyecto, el proceso permitió construir un dataset final mensual con 46 aeropuertos y 3.300 registros en la capa gold. Posteriormente, para la etapa de modelado, se generó una versión depurada del dataset con 3.254

registros, excluyendo observaciones no utilizables para entrenamiento por ausencia de variable objetivo u otros criterios metodológicos. Esta diferencia entre dataset analítico y dataset de modelado es importante, porque no todas las filas disponibles para análisis descriptivo necesariamente pueden utilizarse para entrenar modelos supervisados.

Una etapa fundamental de la preparación fue la construcción de la variable objetivo. El objetivo numérico base se definió como `target_operaciones_total_mes_siguiente`, que representa el total de operaciones del aeropuerto en el mes siguiente. Para construirlo, fue necesario aplicar un desplazamiento temporal dentro de cada aeropuerto, de manera que las variables del mes actual se utilizaran para anticipar el comportamiento del mes posterior. Esta transformación convirtió el problema en una tarea supervisada de predicción del nivel operativo futuro.

A partir del objetivo numérico se construyó una etiqueta ordinal de tres categorías: bajo, medio y alto. Esta clasificación se realizó con base en la distribución histórica de cada aeropuerto, y no mediante umbrales globales para todo el sistema. Esta decisión evitó que los aeropuertos grandes fueran clasificados casi siempre como altos y que los aeropuertos pequeños quedaran clasificados casi siempre como bajos. En cambio, permitió analizar si un aeropuerto se encontraba en un nivel bajo, medio o alto respecto a su propio comportamiento histórico.

También se aplicaron controles para evitar fuga de información o data leakage. Dado que el objetivo consiste en anticipar el nivel operativo del mes siguiente, las variables predictoras deben representar información disponible hasta el mes observado. Por tanto, se verificó que las columnas usadas como características no incluyeran directamente la variable objetivo, etiquetas derivadas del futuro ni variables construidas con información posterior al mes de análisis. Esta validación es esencial para que la evaluación de los modelos sea metodológicamente válida y no presente resultados artificialmente optimistas.

La preparación de datos también implicó revisar la consistencia de las variables categóricas y numéricas. Las variables geográficas, como latitud, longitud y elevación, fueron tratadas como atributos estáticos del aeropuerto. Las variables operacionales, de tráfico y meteorológicas fueron tratadas como variables dinámicas mensuales. Las variables categóricas, como tipo de aeropuerto o servicio programado, se conservaron para su posterior codificación en la etapa de modelado. Esta separación entre variables estáticas y variables temporales permitió organizar mejor el dataset y evitar interpretaciones incorrectas.

En términos metodológicos, la preparación de los datos permitió pasar de fuentes independientes y heterogéneas a una estructura analítica común. El resultado fue una tabla donde cada fila representa un aeropuerto colombiano en un mes específico, y cada columna describe una característica operacional, comercial,

meteorológica, geográfica o de clasificación objetivo. Esta estructura fue la base para el análisis exploratorio, la construcción de visualizaciones, el entrenamiento de modelos de Machine Learning y la interpretación de resultados.

La siguiente tabla resume los principales bloques de variables trabajados durante la preparación del dataset:

Tabla 2. Bloques de variables del dataset final

Bloque de variables	Ejemplos trabajados	Tratamiento metodológico
Identificación	icao_code, iata_code, name, municipality, region_name	Normalización, validación de códigos SKXX y deduplicación por aeropuerto.
Operaciones	operaciones_llegada, operaciones_salida, operaciones_otras, operaciones_total	Agregación mensual desde tráfico origen-destino.
Demanda y tráfico	pasajeros_salida, carga_salida_kg, n_destinos_total, n_empresas_salida, n_registros_mes	Agregación mensual desde tráfico origen-destino.
Meteorología	n_metar_mes, tmpf_mean, dwpf_mean, relh_mean, sknt_mean, vsby_mean, gust_mean, skycl_mode, skycl_mean	Agregación mensual de observaciones METAR históricas desde IEM.
Geografía	latitude_deg, longitude_deg, elevation_ft	Incorporación de atributos estáticos desde la dimensión maestra de aeropuertos.
Servicio	type, scheduled_service	Conservación de variables categóricas para describir tipo de aeropuerto y disponibilidad de servicio programado.
Objetivo	target_operaciones_total_mes_siguiente, nivel bajo/medio/alto	Desplazamiento temporal hacia el mes siguiente y discretización ordinal por distribución histórica de cada aeropuerto.

En síntesis, la preparación de los datos no consistió únicamente en limpiar columnas o eliminar valores faltantes. Fue un proceso de integración, estandarización y diseño analítico que permitió construir una base confiable para responder la pregunta del proyecto. La calidad de esta etapa condiciona directamente la validez del análisis exploratorio, la interpretación de las visualizaciones y el desempeño de los modelos de Machine Learning.

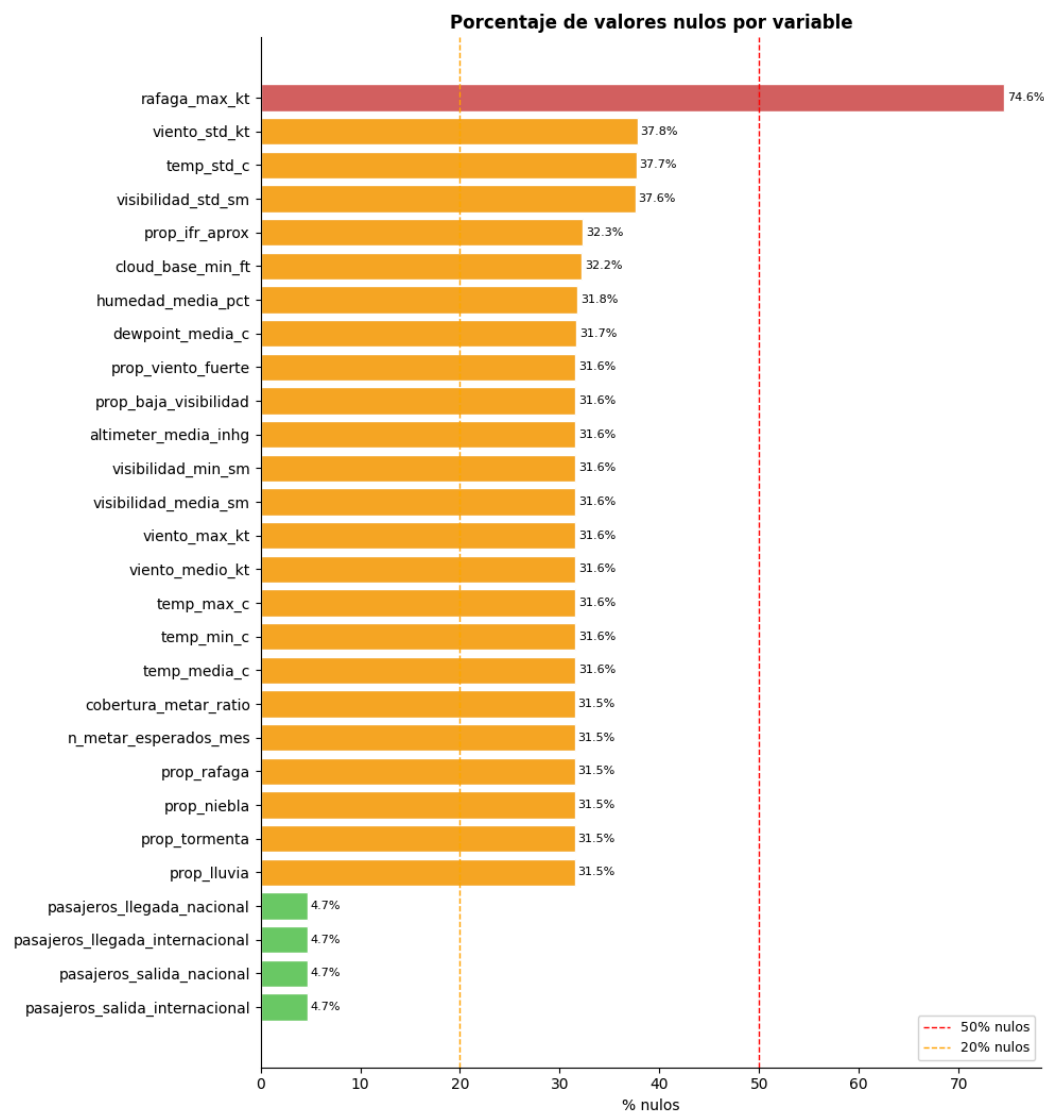


Figura 3. Porcentaje de valores nulos por variable. Evidencia que la nulidad se concentra principalmente en variables meteorológicas.

Interpretación: la variable `rafaga_max_kt` tiene el mayor porcentaje de nulos. La mayoría de variables meteorológicas se ubica alrededor del 31 % de nulidad, mientras que algunas variables de pasajeros presentan valores nulos cercanos al 4,7 %. Esto justifica un tratamiento diferenciado por tipo de variable.

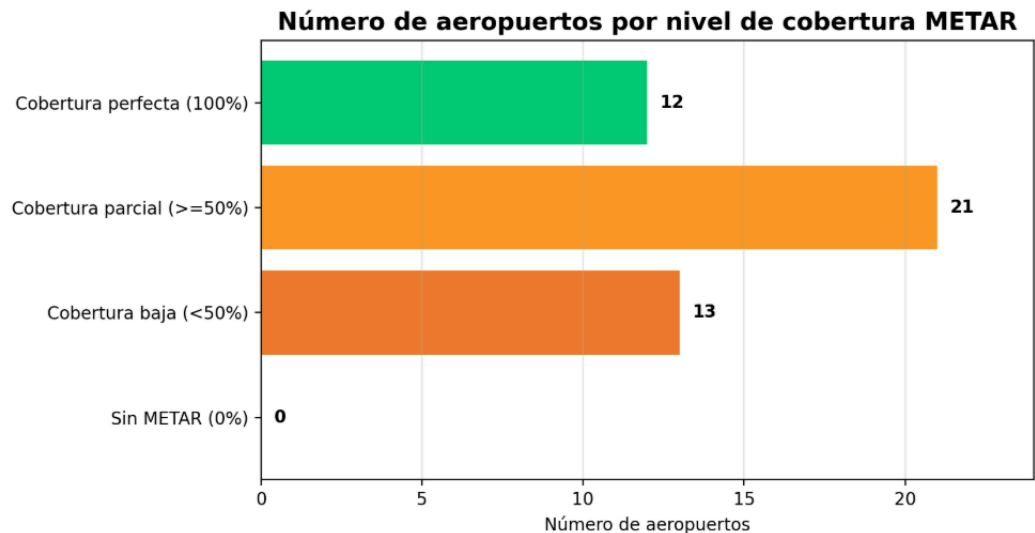


Figura 4. Número de aeropuertos por nivel de cobertura METAR. Resume la disponibilidad de información meteorológica dentro del dataset final.

Interpretación: existen 12 aeropuertos con cobertura perfecta, 21 con cobertura parcial mayor o igual al 50 %, 13 con cobertura baja y 0 sin METAR dentro del universo final revisado. Esta figura justifica la estrategia de control de calidad meteorológica.

3.2.3. Análisis descriptivo e insights importantes (correlación, causa-efecto, etc)

El análisis descriptivo permitió identificar que no todos los aeropuertos colombianos tienen cobertura suficiente en todas las fuentes utilizadas. Aunque la dimensión maestra de aeropuertos contiene 146 aeropuertos colombianos con código ICAO válido bajo el patrón SKXX, el dataset final de modelado quedó concentrado en 46 aeropuertos con información mensual operativa, comercial y meteorológica suficientemente consistente. Esta reducción no debe interpretarse como una pérdida arbitraria de datos, sino como una decisión metodológica orientada a conservar registros comparables y evitar modelos entrenados con información incompleta, no homogénea o sin respaldo suficiente en alguna de las fuentes principales.

Uno de los hallazgos más importantes del análisis exploratorio fue la diferencia entre volumen absoluto y oportunidad relativa. Los aeropuertos principales concentran una mayor cantidad de operaciones y pasajeros debido a su papel estructural dentro de la red aérea nacional. Sin embargo, algunos aeropuertos regionales pueden presentar señales relevantes cuando se analizan indicadores relativos, como pasajeros por vuelo, número de aerolíneas, conectividad, destinos atendidos o comportamiento histórico frente a su propio promedio mensual. Esta lectura es importante porque evita que el análisis se limite únicamente a los aeropuertos de mayor tamaño y permite identificar dinámicas regionales que podrían pasar desapercibidas en una revisión basada solo en volumen total.

En una visualización preliminar construida para el año 2025 se revisaron aeropuertos con señales de demanda relativa y competencia limitada o moderada. Entre los aeropuertos destacados se encontraron ADZ, RCH, PSO, IPI, BSC y PUU. Los valores observados en dicha exploración fueron: ADZ con 124 pasajeros por vuelo y 7 aerolíneas; RCH con 122 pasajeros por vuelo y 4 aerolíneas; PSO con 96 pasajeros por vuelo y 5 aerolíneas; IPI con 85 pasajeros por vuelo y 4 aerolíneas; BSC con 40 pasajeros por vuelo y 7 aerolíneas; y PUU con 38 pasajeros por vuelo y 4 aerolíneas. Estos resultados son útiles para comunicación visual y análisis exploratorio, pero deben interpretarse como señales preliminares. Para convertirlos en una conclusión formal sería necesario definir explícitamente un indicador compuesto de oportunidad, por ejemplo, combinando demanda relativa, número de competidores, conectividad, volumen de pasajeros y estabilidad temporal.

El análisis también permitió evidenciar que la cobertura meteorológica es un criterio determinante para la construcción del dataset final. No todos los aeropuertos con información operacional o comercial cuentan con observaciones METAR suficientes para construir variables mensuales confiables. Por esta razón, la disponibilidad de datos meteorológicos desde IEM fue utilizada como criterio de calidad y consistencia. Esta decisión permitió conservar aeropuertos con información más completa, aunque redujo el número total de aeropuertos disponibles para modelado. En este sentido, la reducción del universo inicial no representa simplemente una exclusión de aeropuertos, sino una forma de garantizar que las variables empleadas en el análisis tengan una base comparable.

Otro aspecto relevante fue la revisión de posibles valores faltantes y diferencias de cobertura entre fuentes. Las variables operacionales, comerciales y meteorológicas no tienen necesariamente la misma disponibilidad para todos los aeropuertos y meses. En consecuencia, el análisis descriptivo ayudó a identificar qué columnas podían utilizarse directamente, cuáles requerían tratamiento adicional y cuáles debían interpretarse con precaución. En particular, variables como `n_metar_mes` son importantes porque permiten conocer la cantidad de observaciones meteorológicas que respaldan los promedios mensuales calculados.

También se revisó el riesgo de fuga de información o data leakage. En una validación preliminar de columnas se reportó que no había leakage directo por presencia de columnas target o label dentro del conjunto final de variables explicativas. Esta validación es fundamental porque el objetivo del modelo es anticipar el nivel operativo del mes siguiente, no reconstruir una etiqueta usando información futura o variables derivadas directamente del objetivo. Por tanto, las variables utilizadas como características deben corresponder únicamente a información disponible hasta el mes observado.

Desde el punto de vista temporal, el análisis aeropuerto-mes permite estudiar patrones de variación mensual y posibles comportamientos estacionales. Algunos

aeropuertos pueden aumentar su operación en determinados meses por turismo, conectividad regional, temporadas vacacionales o dinámicas comerciales específicas. Esta estructura temporal justifica la construcción de la variable `target_operaciones_total_mes_siguiente`, ya que el interés del proyecto no es explicar únicamente el estado actual del aeropuerto, sino clasificar el nivel operativo esperado para el mes siguiente.

En cuanto a correlación y causa-efecto, el análisis debe ser prudente. La correlación entre operaciones, pasajeros, destinos o número de empresas puede reflejar relaciones estructurales del sistema aéreo, pero no necesariamente causalidad. Por ejemplo, una mayor cantidad de empresas puede coexistir con una mayor demanda, pero el dato agregado mensual no permite concluir por sí solo si las empresas generan la demanda o si ingresan al mercado porque la demanda ya existe. Lo mismo ocurre con las variables meteorológicas: una asociación entre visibilidad, viento, nubosidad u operación mensual no implica causalidad operacional sin un diseño metodológico adicional.

Por esta razón, los resultados del análisis descriptivo deben entenderse como una base para formular hipótesis, orientar la selección de variables y apoyar la construcción de visualizaciones. El proyecto no desarrolla un diseño de inferencia causal, sino un enfoque de analítica descriptiva y Machine Learning supervisado. En consecuencia, los patrones encontrados permiten identificar relaciones, comportamientos diferenciales y señales útiles para el modelado, pero no deben presentarse como efectos causales comprobados.

En síntesis, el análisis descriptivo mostró tres ideas principales. Primero, la calidad y cobertura de los datos condiciona directamente el universo de aeropuertos que pueden incluirse en el modelado. Segundo, el análisis relativo por aeropuerto permite identificar oportunidades y comportamientos que no son visibles cuando solo se revisan volúmenes absolutos. Tercero, la integración de operaciones, tráfico, meteorología y atributos aeroportuarios proporciona una visión más completa del sistema, siempre que las relaciones encontradas se interpreten con cautela metodológica.

ESPACIO DEJADO EN BLANCO INTENCIONALMENTE

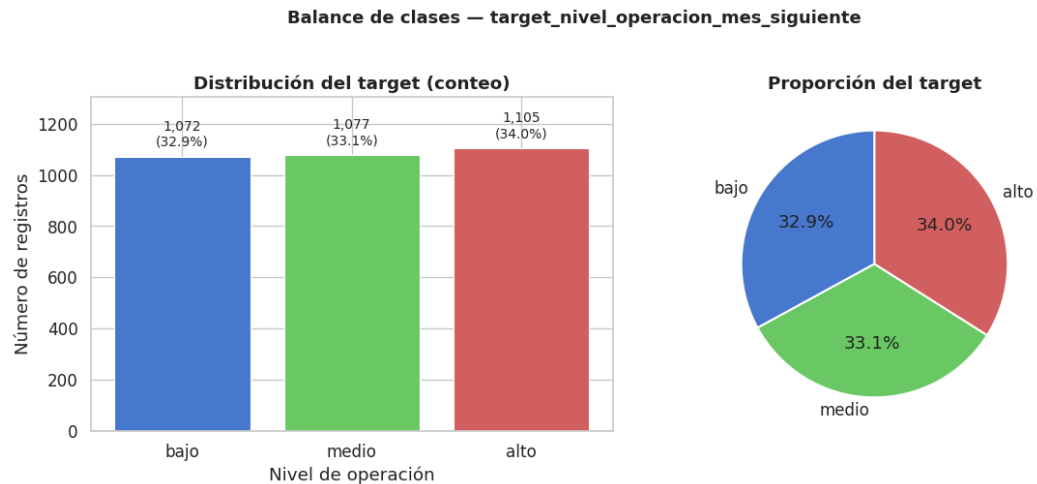


Figura 5. Balance de clases del target target_nivel_operacion_mes_siguiente. La distribución de bajo, medio y alto es aproximadamente uniforme.

Interpretación: el target quedó balanceado, con bajo = 32,9 %, medio = 33,1 % y alto = 34,0 %. Este resultado favorece la evaluación de modelos multiclasa porque no existe una clase dominante evidente.

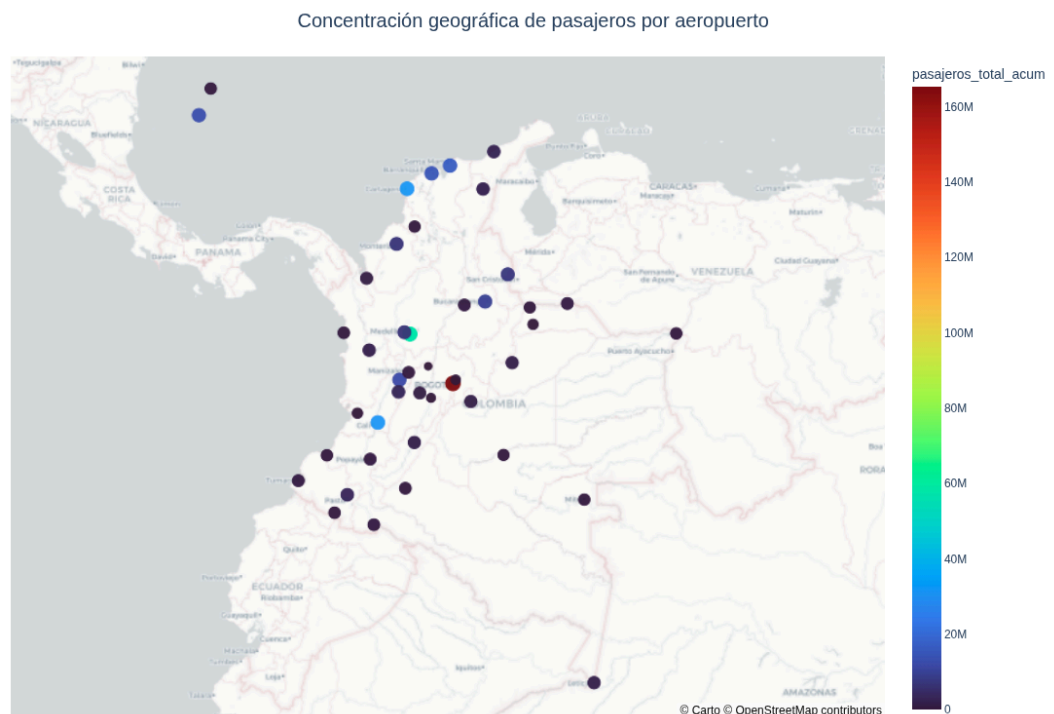


Figura 6. Concentración geográfica de pasajeros por aeropuerto. La figura presenta la distribución espacial del volumen acumulado de pasajeros en los aeropuertos incluidos en el dataset.

Interpretación: la figura permite identificar los principales nodos de demanda aérea dentro del sistema aeroportuario analizado. Se observa que la concentración de pasajeros no es homogénea en el territorio, sino que se agrupa principalmente en aeropuertos con mayor conectividad comercial, ubicación estratégica o relevancia regional. Esta visualización complementa el análisis de operaciones, ya que permite diferenciar entre aeropuertos con alto movimiento operacional y aeropuertos con mayor concentración de pasajeros acumulados.

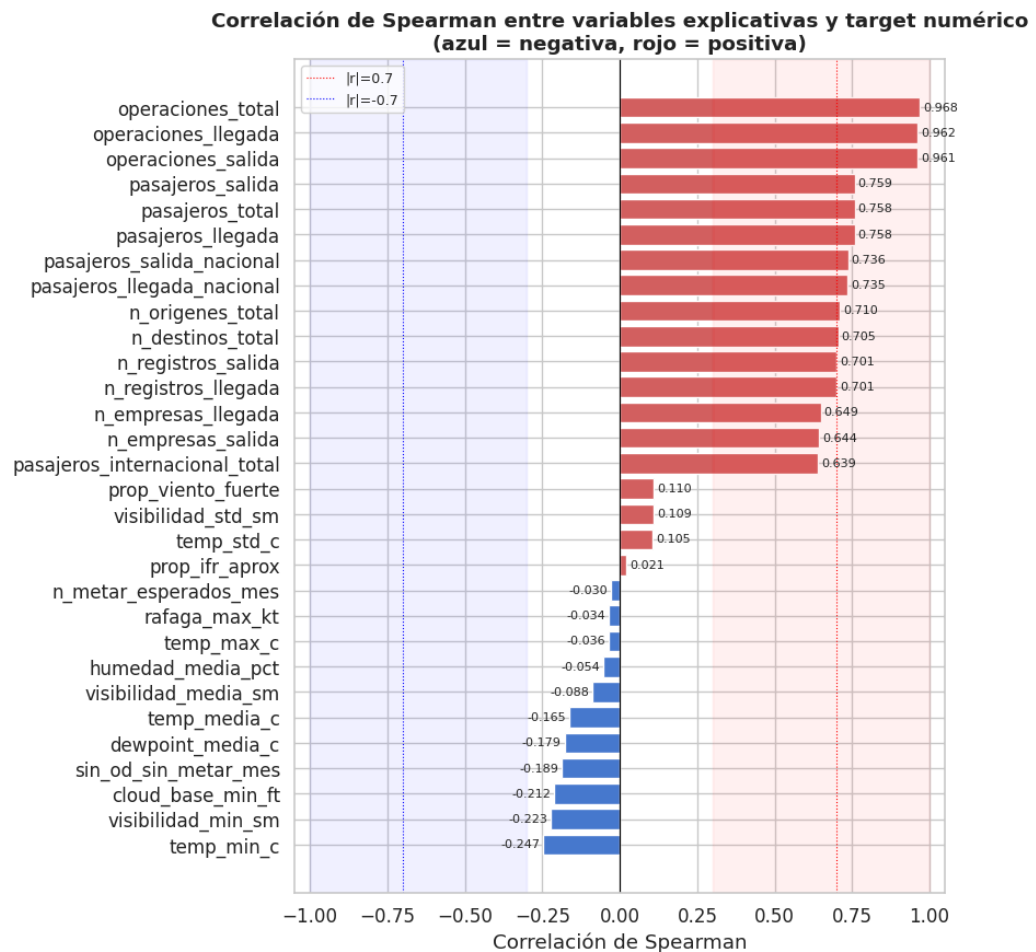


Figura 7. Correlación de Spearman entre variables explicativas y target numérico. Resalta asociaciones monotónicas con las operaciones del mes siguiente.

Interpretación: las variables operacionales, de pasajeros, destinos, registros y empresas muestran las correlaciones positivas más fuertes con el target numérico. Las variables meteorológicas tienen asociaciones más débiles, por lo que deben interpretarse como covariables contextuales.

3.3. Selección de modelos (supervisados y no supervisados), Ingeniería de Características, Entrenamiento, Evaluación.

3.3.1. Modelos

Para la construcción del sistema predictivo se abordó un problema de aprendizaje supervisado de clasificación multiclase, cuyo objetivo consiste en predecir el nivel de operación esperado para el mes siguiente, categorizado en tres niveles: bajo, medio y alto.

Con el fin de identificar el algoritmo con mejor capacidad predictiva, se implementaron y compararon distintos modelos de clasificación pertenecientes a diferentes familias de aprendizaje automático. Los modelos considerados fueron:

- Regresión Logística Multinomial (Logistic Regression).
- Random Forest.
- XGBoost (Extreme Gradient Boosting).
- LightGBM (Light Gradient Boosting Machine).
- Extra Trees.
- K-Nearest Neighbors (KNN).

La Regresión Logística Multinomial fue utilizada como modelo base debido a su simplicidad, interpretabilidad y capacidad para establecer una línea de referencia en problemas de clasificación multiclase.

Random Forest fue seleccionado por su capacidad para modelar relaciones no lineales entre las variables, así como por su robustez frente al sobreajuste mediante la construcción de múltiples árboles de decisión entrenados sobre subconjuntos aleatorios de los datos.

XGBoost y LightGBM fueron incorporados debido a su alto desempeño en problemas de clasificación tabular. Ambos algoritmos se basan en técnicas de Gradient Boosting que permiten mejorar progresivamente el rendimiento del modelo mediante la corrección iterativa de errores cometidos por modelos anteriores.

Extra Trees fue considerado como una alternativa basada en ensamblado de árboles, caracterizada por introducir un mayor nivel de aleatoriedad durante el proceso de entrenamiento, lo que puede contribuir a mejorar la capacidad de generalización.

Finalmente, K-Nearest Neighbors (KNN) fue incluido como método basado en instancias, permitiendo comparar el desempeño de algoritmos basados en distancia frente a enfoques de ensamblado y aprendizaje probabilístico.

Taxonomía de algoritmos evaluados en el proceso de modelado (métodos lineales, distancia y ensambles de árboles) .

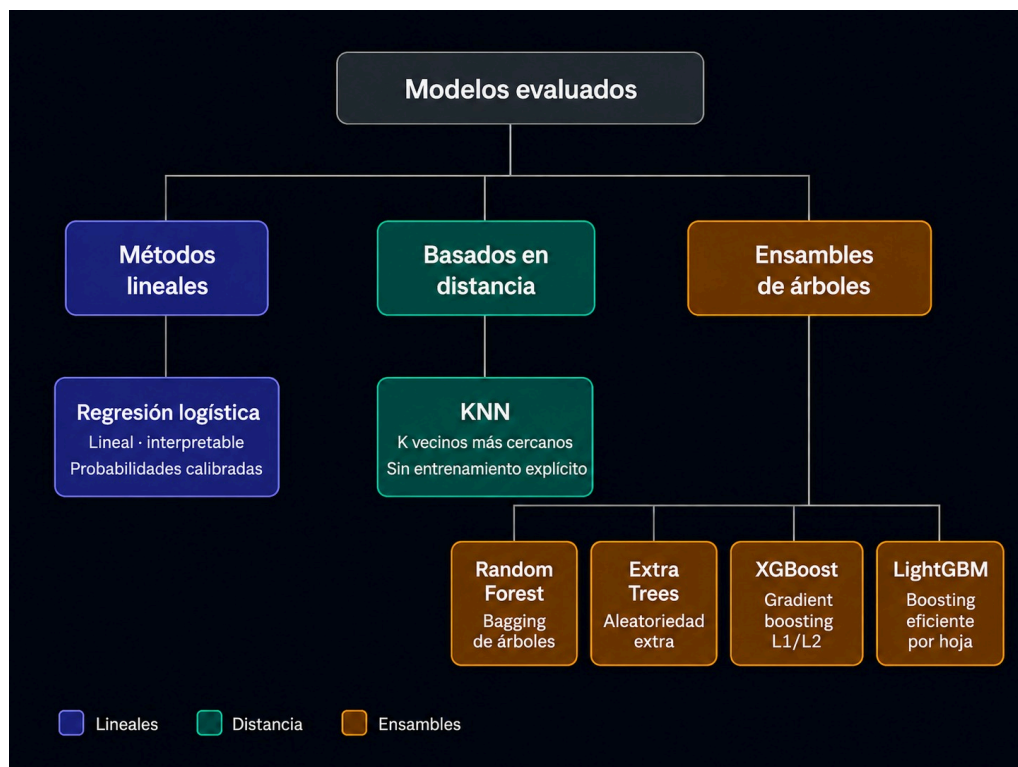


Figura 8. Taxonomía de algoritmos de machine learning evaluados. Se consideraron tres familias: métodos lineales, basados en distancia y ensambles de árboles, estos últimos representados por Random Forest, Extra Trees, XGBoost y LightGBM.

La selección de modelos buscó incorporar algoritmos pertenecientes a diferentes paradigmas de aprendizaje automático, incluyendo métodos lineales, modelos basados en distancia y técnicas de ensamblado mediante bagging y boosting. Esta estrategia permitió comparar distintos enfoques de modelado y determinar cuál ofrecía la mejor capacidad predictiva para el problema planteado.

3.3.2. Características e Ingeniería de Características

La ingeniería de características se orientó a transformar el dataset mensual aeropuerto-mes en una matriz de variables explicativas adecuada para modelos de Machine Learning supervisado. El objetivo fue construir predictores disponibles antes del mes que se desea clasificar, evitando el uso de información futura y reduciendo el riesgo de fuga de información o data leakage. Dado que la variable

objetivo representa el nivel de operación del mes siguiente, las características utilizadas por el modelo deben corresponder al mes actual o a meses anteriores.

En términos generales, las variables candidatas se agruparon en cinco familias principales: variables operacionales históricas, variables de demanda y conectividad, variables meteorológicas mensuales, atributos estáticos del aeropuerto y variables temporales. A estas familias se adicionaron variables derivadas construidas mediante ratios, rezagos temporales, promedios móviles, cambios relativos y banderas de disponibilidad histórica. Esta estrategia permitió enriquecer el dataset con información de tendencia, intensidad operacional, estacionalidad y contexto meteorológico.

La selección final de variables debe excluir cualquier columna que contenga directamente el objetivo futuro, etiquetas derivadas del objetivo o información generada después del mes de predicción. Por ejemplo, variables como `operaciones_total` del mes actual pueden ser válidas para anticipar el mes siguiente, siempre que correspondan al mes observado. En cambio, columnas como `target_operaciones_total_mes_siguiente`, `target_nivel_operacion_mes_siguiente` o cualquier variable derivada directamente de la etiqueta no deben ingresar como predictores.

La ingeniería de características se implementó en PySpark, aprovechando operaciones de columnas, funciones de ventana y transformaciones distribuidas. Esto permitió mantener consistencia con el enfoque tecnológico del proyecto en Databricks y facilitar el procesamiento del dataset en las capas analíticas definidas.

Tabla 3. Familias principales de características consideradas.

Familia de características	Ejemplos	Riesgo y control metodológico
Históricas operativas	<code>operaciones_total</code> y agregados mensuales	Controlar que las variables correspondan al mes observado o a meses anteriores, no al mes objetivo futuro.
Demanda y conectividad	<code>pasajeros_salida</code> , <code>carga_salida_kg</code> , <code>n_destinos_total</code> , <code>n_empresas_salida</code>	Revisar disponibilidad temporal y consistencia por aeropuerto antes de usarlas como predictores.
Meteorológicas	<code>temp_media_c</code> , <code>humedad_media_pct</code> , <code>viento_medio_kt</code> , <code>visibilidad_media_sm</code> , <code>prop_lluvia</code>	Imputar, marcar cobertura parcial o excluir variables con ausencia excesiva; no asumir causalidad.

Familia de características	Ejemplos	Riesgo y control metodológico
Atributos del aeropuerto	type, region_name, elevation_ft, scheduled_service	Codificar variables categóricas y controlar cardinalidad.
Temporales	anio, mes, trimestre, semestre, mes_sin, mes_cos	Evitar codificaciones que memoricen periodos específicos sin capacidad de generalización.

3.3.2.1. Variables temporales.

El primer bloque de ingeniería correspondió a la generación de variables temporales a partir de fecha_mes. Estas variables buscan capturar patrones de calendario, estacionalidad y comportamiento histórico asociado al tiempo. En un problema aeroportuario mensual, el tiempo es una dimensión relevante porque la operación puede variar por temporadas vacacionales, efectos de recuperación pospandemia, meses de alta demanda turística o cambios estructurales del sistema.

La inclusión de variables sinusoidales y cosenoidales es importante porque el mes del año no debe interpretarse únicamente como un número lineal. Por ejemplo, diciembre y enero son meses consecutivos en términos estacionales, aunque sus valores numéricos sean 12 y 1. La codificación cíclica permite representar esta continuidad temporal de forma más adecuada para los modelos.

Después de agregar las variables temporales, los tres conjuntos de datos conservaron el mismo número de columnas. El conjunto de entrenamiento quedó con 2.572 registros y 77 columnas; el conjunto de validación con 319 registros y 77 columnas; y el conjunto de prueba con 363 registros y 77 columnas. Esto confirma que la transformación fue aplicada de manera consistente sobre los tres subconjuntos.

Tabla 4. Variables temporales construidas desde fecha_mes.

Variable temporal	Descripción	Utilidad metodológica
anio	Año del registro mensual.	Permite capturar diferencias estructurales entre años.
mes	Mes calendario del registro.	Representa estacionalidad mensual.
trimestre	Trimestre del año.	Resume patrones de calendario por bloques de tres meses.
semestre	Primer o segundo semestre.	Captura diferencias generales entre mitades del año.

Variable temporal	Descripción	Utilidad metodológica
mes_idx	Índice secuencial de meses desde el primer año observado.	Representa tendencia temporal acumulada.
mes_sin y mes_cos	Transformaciones cíclicas del mes.	Codifican continuidad entre meses como diciembre y enero.
trimestre_sin y trimestre_cos	Transformaciones cíclicas del trimestre.	Codifican ciclos trimestrales.
anio_covid	Indicador binario para registros de 2020.	Marca el año de choque operacional por pandemia.
post_covid	Indicador binario para años desde 2021.	Representa el período posterior al choque inicial de 2020.

Tabla 5. Resultado de la creación de variables temporales por split.

Split	Registros	Columnas después de variables temporales
Train	2.572	77
Validation	319	77
Test	363	77

Como control metodológico, el índice temporal mes_idx debe construirse con una referencia común para todos los subconjuntos. En una implementación final, es recomendable fijar el año mínimo a partir del conjunto de entrenamiento y aplicar esa misma referencia a validación y prueba, evitando que cada subconjunto calcule una escala temporal diferente.

3.3.2.2. Ratios operacionales.

El segundo bloque correspondió a ratios operacionales y comerciales. Estos indicadores permiten normalizar magnitudes absolutas y comparar aeropuertos con tamaños diferentes. En un sistema aeroportuario altamente concentrado, las variables absolutas pueden estar dominadas por aeropuertos grandes. Por ello, los ratios ayudan a construir señales relativas de eficiencia, intensidad o uso.

Estas variables son útiles porque permiten diferenciar aeropuertos que tienen un volumen operacional similar pero composiciones distintas. Por ejemplo, dos aeropuertos pueden registrar una cantidad parecida de operaciones, pero uno puede movilizar más pasajeros por operación o atender menos destinos con mayor intensidad. Este tipo de señal puede ser relevante para clasificar el nivel operativo del mes siguiente.

Para evitar errores por divisiones entre cero o valores faltantes, estas variables fueron calculadas mediante una función de división segura, representada en el código como `safe_divide_expr`. Esta práctica evita producir valores infinitos o inconsistentes cuando el denominador no está disponible o toma valor cero.

Tabla 4. Ratios operacionales y comerciales generados. Variables meteorológicas derivadas

Variable derivada	Fórmula conceptual	Interpretación
pasajeros_por_operacion	pasajeros totales / operaciones totales	Aproxima la intensidad de pasajeros por operación.
carga_kg_por_operacion	carga total kg / operaciones totales	Aproxima la intensidad de carga por operación.
operaciones_por_destino	operaciones totales / número de destinos	Mide la concentración de operaciones por destino atendido.

3.3.2.3. Variables meteorológicas derivadas

El tercer bloque correspondió a variables meteorológicas derivadas. Las observaciones METAR originales ya habían sido agregadas mensualmente, pero se construyeron indicadores adicionales para representar rangos, diferencias y condiciones adversas de forma más sintética.

El indicador `meteo_adverso_score` se construyó como un promedio de variables meteorológicas adversas disponibles, incluyendo proporción de lluvia, tormenta, niebla, baja visibilidad, viento fuerte, ráfaga e IFR aproximado. Esta variable busca resumir en un solo indicador la presencia relativa de condiciones meteorológicas potencialmente adversas durante el mes.

Es importante aclarar que estas variables se interpretan como covariables contextuales y no como evidencia causal. Un mayor puntaje de meteorología adversa puede estar asociado con ciertos patrones operacionales, pero el proyecto no desarrolla un diseño de inferencia causal que permita afirmar que dichas condiciones causan directamente un cambio en el nivel de operación.

Tabla 5. Variables meteorológicas derivadas.

Variable derivada	Descripción	Interpretación
temp_rango_c	Diferencia entre temperatura máxima y mínima mensual.	Aproxima variabilidad térmica mensual.

Variable derivada	Descripción	Interpretación
dewpoint_depression_c	Diferencia entre temperatura media y punto de rocío medio.	Indica separación entre temperatura y humedad atmosférica.
visibilidad_rango_sm	Diferencia entre visibilidad media y visibilidad mínima.	Resume variabilidad de visibilidad durante el mes.
meteo_adverso_score	Promedio de proporciones de fenómenos adversos.	Indicador agregado de condiciones meteorológicas adversas.

3.3.2.4. Rezagos temporales por aeropuerto.

El cuarto bloque consistió en construir variables rezagadas por aeropuerto. Los rezagos permiten incorporar memoria histórica al modelo, utilizando información de meses anteriores para anticipar el comportamiento del mes siguiente. Esta estrategia es especialmente importante porque el comportamiento aeroportuario suele presentar persistencia temporal: un aeropuerto con alta operación reciente puede mantener niveles altos en el corto plazo, mientras que un aeropuerto con baja operación puede conservar esa condición durante varios meses.

Los rezagos se calcularon particionando por icao_code y ordenando por fecha_mes. Para cada aeropuerto, se generaron rezagos de 1, 2, 3 y 12 meses sobre variables seleccionadas. Los rezagos de corto plazo capturan continuidad reciente, mientras que el rezago de 12 meses aproxima comportamiento estacional anual.

La generación de rezagos produjo variables como operaciones_total_lag_1, operaciones_total_lag_2, operaciones_total_lag_3, operaciones_total_lag_12, y equivalentes para pasajeros, carga, destinos, empresas, registros y meteorología. Estas columnas permiten que el modelo incorpore información sobre el comportamiento pasado del aeropuerto sin utilizar directamente el valor futuro que se quiere predecir.

El control más importante en esta etapa fue garantizar que los rezagos siempre miraran hacia atrás en el tiempo. Para ello, se usó una ventana ordenada por fecha_mes dentro de cada icao_code. De esta manera, una fila correspondiente a un mes determinado solo puede recibir información de meses anteriores del mismo aeropuerto.

Tabla 6. Variables utilizadas como fuente para rezagos temporales.

Familia	Variables fuente de rezagos
Operación	operaciones total

Familia	Variables fuente de rezagos
Demanda	pasajeros total, carga total kg
Conectividad	n_destinos_total, n_empresas_salida, n_registros_salida
Meteorología	cobertura metar ratio, meteo_adverso_score
Ratios derivados	pasajeros por operacion, carga kg por operacion

3.3.2.5. Promedios móviles

Además de los rezagos puntuales, se construyeron promedios móviles por aeropuerto. Estos indicadores permiten suavizar la variabilidad mensual y representar tendencias recientes o anuales. A diferencia de un rezago específico, un promedio móvil resume varios meses anteriores, lo que puede hacer que el modelo capture patrones más estables.

Metodológicamente, los promedios móviles se calcularon usando únicamente meses anteriores al mes observado. En el código, esto se logró mediante ventanas con el rango `rowsBetween(-window, -1)`, lo que excluye la fila actual y evita incorporar información del propio mes cuando se desea representar únicamente el historial previo.

Este control es relevante porque, en problemas temporales, un promedio móvil mal construido puede introducir información no disponible en el momento de predicción. En este caso, la construcción busca mantener coherencia con un escenario donde el modelo se alimenta de información histórica disponible antes de clasificar el mes siguiente.

Tabla 7. Promedios móviles generados por aeropuerto.

Variable fuente	Promedio móvil generado	Interpretación
operaciones_total	operaciones_total_roll_mean_3, operaciones_total_roll_mean_12	Tendencia operacional reciente y anual.
pasajeros_total	pasajeros_total_roll_mean_3, pasajeros_total_roll_mean_12	Tendencia de demanda de pasajeros.
carga_total_kg	carga_total_kg_roll_mean_3, carga_total_kg_roll_mean_12	Tendencia de carga movilizada.
meteo_adverso_score	meteo_adverso_score_roll_mean_3, meteo_adverso_score_roll_mean_12	Tendencia de condiciones meteorológicas adversas.

3.3.2.6. Cambios operacionales

El sexto bloque de ingeniería de características calculó variables de cambio operacional. Estas variables permiten medir no solo el nivel actual de operación, sino también la dirección e intensidad del cambio respecto a períodos previos. En problemas de predicción mensual, las variaciones recientes pueden ser señales importantes para anticipar transiciones entre niveles bajo, medio y alto.

Estas variables son útiles porque dos aeropuertos pueden tener el mismo nivel actual de operaciones, pero encontrarse en trayectorias distintas. Uno puede estar creciendo respecto a su promedio reciente, mientras otro puede estar disminuyendo. Esta diferencia puede ser relevante para clasificar el nivel operativo esperado del mes siguiente.

Tabla 8. Variables de cambio operacional.

Variable	Descripción	Interpretación
operaciones_delta_1	Diferencia entre operaciones actuales y operaciones del mes anterior.	Cambio absoluto mensual.
operaciones_pct_change_1	Cambio porcentual frente al mes anterior.	Variación relativa reciente.
operaciones_vs_roll_mean_3	Operaciones actuales frente al promedio móvil de 3 meses.	Comparación contra tendencia reciente.
operaciones_vs_roll_mean_12	Operaciones actuales frente al promedio móvil de 12 meses.	Comparación contra referencia anual.

3.3.2.7. Flags de disponibilidad histórica

El séptimo bloque consistió en crear banderas binarias de disponibilidad histórica. Estas variables indican si un aeropuerto-mes cuenta con rezagos operacionales disponibles para 1, 3 o 12 meses.

Estas banderas son importantes porque los primeros meses de la serie temporal de cada aeropuerto no tienen historial completo. Por ejemplo, un registro de enero de 2020 no puede tener rezago de 12 meses si el dataset inicia en 2020. En lugar de ignorar completamente esta condición, las banderas permiten que el modelo identifique si ciertos valores históricos están disponibles o no.

Tabla 10. Banderas de disponibilidad histórica.

Variable	Descripción
historial_lag_1_disponible	Indica si existe información del mes anterior.
historial_lag_3_disponible	Indica si existe información con tres meses de rezago.
historial_lag_12_disponible	Indica si existe información del mismo mes del año anterior.

3.3.2.8. Resultado de la ingeniería de características

Después de aplicar la ingeniería de características adicionales, los conjuntos de entrenamiento, validación y prueba quedaron con 129 columnas cada uno. El número de registros se mantuvo igual respecto a los conjuntos originales, lo que indica que la generación de características no eliminó observaciones.

En total, se generaron 63 columnas nuevas respecto al dataset base de modelado. Estas variables incluyen el identificador de split, ratios operacionales, variables meteorológicas derivadas, rezagos, promedios móviles, cambios operacionales y banderas de disponibilidad histórica.

Tabla 11. Resultado de la ingeniería adicional por split.

Split	Registros	Columnas después de ingeniería adicional
Train	2.572	129
Validation	319	129
Test	363	129

Tabla 12. Resumen de grupos de variables nuevas generadas.

Grupo de variables nuevas	Ejemplos de columnas generadas	Propósito
Ratios operacionales	pasajeros_por_operacion, carga_kg_por_operacion, operaciones_por_destino	Normalizar magnitudes absolutas y comparar aeropuertos de diferente tamaño.
Meteorología derivada	temp_rango_c, dewpoint_depression_c, visibilidad_rango_sm, meteo_adverso_score	Representar variabilidad y condiciones meteorológicas adversas.

Grupo de variables nuevas	Ejemplos de columnas generadas	Propósito
Rezagos de corto plazo	operaciones_total_lag_1, pasajeros_total_lag_1, carga_total_kg_lag_1	Capturar continuidad reciente del comportamiento aeroportuario.
Rezagos de mediano plazo	operaciones_total_lag_2, operaciones_total_lag_3	Incorporar memoria operacional de los meses previos.
Rezagos anuales	operaciones_total_lag_12, pasajeros_total_lag_12, meteo_adverso_score_lag_12	Capturar patrones estacionales anuales.
Promedios móviles	operaciones_total_roll_mean_3, operaciones_total_roll_mean_12	Suavizar variaciones mensuales y representar tendencias.
Cambios operacionales	operaciones_delta_1, operaciones_pct_change_1, operaciones_vs_roll_mean_3, operaciones_vs_roll_mean_12	Medir crecimiento, caída o desviación frente al historial.
Flags de historial	historial_lag_1_disponible, historial_lag_3_disponible, historial_lag_12_disponible	Indicar disponibilidad de información histórica.

3.3.2.9. Control de fuga de información

El principal control metodológico de esta etapa fue asegurar que las variables construidas no incorporarán información futura. Para ello, las variables temporales se calcularon con base en la fecha del registro, los ratios utilizaron variables del mes observado, los rezagos utilizaron meses anteriores y los promedios móviles fueron calculados excluyendo la fila actual. Además, la variable objetivo y las etiquetas derivadas del mes siguiente no deben formar parte del conjunto de predictores.

La concatenación de los conjuntos de entrenamiento, validación y prueba antes de calcular rezagos y promedios móviles permitió conservar continuidad temporal entre particiones. Esto es útil cuando la división de los datos respeta el orden temporal y simula un escenario real en el que, para predecir un mes de validación o prueba, se dispone de información histórica previa. Sin embargo, si la división de datos fuera aleatoria, este procedimiento debería revisarse cuidadosamente para evitar contaminación entre particiones. En un escenario final, la recomendación metodológica es que la separación de datos y la generación de variables temporales estén alineadas con la lógica de predicción temporal.

3.3.2.10. Síntesis de la ingeniería de características

En síntesis, la ingeniería de características permitió enriquecer el dataset original con variables que representan calendario, estacionalidad, intensidad operacional, conectividad, meteorología derivada, memoria histórica, tendencias recientes y cambios respecto al pasado. Este proceso transformó una tabla mensual descriptiva en una matriz de predictores más adecuada para modelos de clasificación supervisada.

La mayor fortaleza de esta etapa fue incorporar información histórica por aeropuerto mediante rezagos y promedios móviles, lo cual es coherente con la naturaleza temporal del problema. La principal precaución metodológica es mantener la separación entre variables disponibles en el mes observado y variables asociadas al mes objetivo futuro. De esta forma, el modelo se entrena bajo una lógica más realista: utilizar información conocida hasta el mes actual para clasificar el nivel operativo esperado del mes siguiente.

3.3.3. Entrenamiento

El proceso de entrenamiento tuvo como objetivo identificar el modelo supervisado con mejor capacidad para clasificar el nivel mensual de operación aeroportuaria en las categorías bajo, medio y alto. Para ello se diseñó una estrategia de entrenamiento progresiva compuesta por cuatro etapas: entrenamiento base, ajuste manual de hiperparámetros, optimización automática mediante Optuna y validación cruzada interna.

3.3.3.1. Entrenamiento inicial de modelos base.

Inicialmente se definió un catálogo de modelos clásicos de clasificación compuesto por:

- Logistic Regression (B01)
- Random Forest (M01)
- XGBoost (M02)
- LightGBM (M03)
- Extra Trees (M04)
- K-Nearest Neighbors (M05)

Cada modelo fue entrenado utilizando tres configuraciones distintas de variables predictoras:

- corr_060_070
- corr_060_080
- corr_060_090

Estas configuraciones provenían del proceso previo de selección de variables basado en niveles de correlación.

Para cada combinación modelo-configuración se utilizó el conjunto de entrenamiento (**train**) para ajustar el modelo y posteriormente se evaluó sobre el conjunto de validación (**validation**). Los resultados obtenidos fueron almacenados para su comparación posterior.

3.3.3.2. Entrenamiento Base: Regresión Logística Multinomial (B01)

Como primera etapa del proceso de modelado se implementó un modelo base de Regresión Logística Multinomial (B01). Este modelo se utilizó como referencia inicial debido a su simplicidad, rapidez de entrenamiento y capacidad para establecer una línea base de desempeño frente a modelos más complejos.

Para garantizar la comparabilidad de los experimentos, se mantuvieron los mismos hiperparámetros en las tres configuraciones de variables evaluadas:

Tabla 13. Hiperparámetros del modelo B01: Regresión Logística Multinomial

Hiperparámetro	Valor
max iter	100
solver	lbfgs
multi class	multinomial
random state	42

El entrenamiento se realizó sobre tres conjuntos de variables obtenidos durante la etapa de selección de características, correspondientes a diferentes umbrales de correlación.

Configuración corr_060_070

En esta configuración se utilizaron las variables seleccionadas bajo el criterio de correlación entre 0.60 y 0.70.

Tabla 14. Métricas del modelo B01 con configuración corr_060_070

Métrica	Valor
Accuracy	0.6426
Balanced Accuracy	0.6438
Macro F1	0.6371
Tiempo de ejecución	0.15 s

Los resultados muestran un desempeño inicial aceptable, logrando una clasificación relativamente equilibrada entre las diferentes categorías de la variable objetivo.

Configuración corr_060_080

Posteriormente se entrenó el mismo modelo utilizando una configuración menos restrictiva, permitiendo correlaciones de hasta 0.80.

Tabla 15. Métricas del modelo B01 con configuración corr_060_080

Métrica	Valor
Accuracy	0.6552
Balanced Accuracy	0.6684
Macro F1	0.6553
Tiempo de ejecución	0.32 s

En comparación con la configuración anterior, se observa una mejora en todas las métricas evaluadas, lo que indica que la incorporación de variables adicionales aportó información relevante para la clasificación.

Configuración corr_060_090

Finalmente se evaluó la configuración con mayor cantidad de variables, permitiendo correlaciones de hasta 0.90.

Tabla 16. Métricas del modelo B01 con configuración corr_060_090

Métrica	Valor
Accuracy	0.6708
Balanced Accuracy	0.6780
Macro F1	0.6725
Tiempo de ejecución	1.39 s

Esta configuración obtuvo los mejores resultados entre las tres variantes evaluadas, alcanzando el mayor valor de Accuracy, Balanced Accuracy y Macro F1. Aunque el tiempo de entrenamiento aumentó debido al mayor número de variables incluidas, la mejora en el desempeño justificó continuar con esta configuración en las siguientes etapas del modelado.

Comparación de Resultados

Tabla 17. Comparación de resultados del modelo B01 por configuración

Configuración	Accuracy	Balanced Accuracy	Macro F1	Runtime (s)
corr_060_070	0.6426	0.6438	0.6371	0.15
corr_060_080	0.6552	0.6684	0.6553	0.32
corr_060_090	0.6708	0.6780	0.6725	1.39

Análisis de resultados

Los resultados evidencian una tendencia positiva a medida que aumenta la cantidad de variables disponibles para el modelo. La configuración **corr_060_090** presentó el mejor desempeño global, logrando un incremento de aproximadamente 3 puntos porcentuales en Accuracy y más de 3.5 puntos porcentuales en Macro F1 respecto a la configuración más restrictiva (**corr_060_070**).

Estos hallazgos sugieren que las variables adicionales incorporadas en la configuración **corr_060_090** contienen información predictiva relevante y no generan un impacto negativo significativo por multicolinealidad. Por esta razón, esta configuración fue considerada la más prometedora para continuar con la evaluación de otros algoritmos de clasificación y posteriores procesos de optimización.

3.3.3.3. Entrenamiento Base: Random Forest (M01)

Como segundo modelo evaluado dentro de la etapa de entrenamiento base, se implementó un algoritmo **Random Forest**, una técnica de aprendizaje supervisado basada en ensamblado de árboles de decisión. Este algoritmo combina múltiples árboles entrenados sobre subconjuntos aleatorios de datos y variables, reduciendo el riesgo de sobreajuste y mejorando la capacidad de generalización respecto a un único árbol de decisión.

El modelo fue entrenado utilizando la implementación de Scikit-Learn bajo un entorno local y manteniendo la misma configuración de hiperparámetros para las tres versiones del conjunto de variables.

Hiperparámetros utilizados

Tabla 18. Hiperparámetros del modelo M01: Random Forest

Hiperparámetro	Valor
n_estimators	100
max_depth	10
random_state	42
n_jobs	-1

Donde:

- **n_estimators = 100** indica que el bosque está compuesto por 100 árboles de decisión.
- **max_depth = 10** limita la profundidad máxima de cada árbol para evitar sobreajuste.
- **random_state = 42** garantiza la reproducibilidad de los experimentos.

- **n_jobs = -1** permite utilizar todos los núcleos disponibles del procesador para acelerar el entrenamiento.

Configuración corr_060_070

En esta primera ejecución se utilizaron las variables seleccionadas bajo el criterio de correlación entre 0.60 y 0.70.

Tabla 19. Métricas del modelo M01 con configuración corr_060_070

Métrica	Valor
Accuracy	0.8433
Balanced Accuracy	0.8454
Macro F1	0.8447
Tiempo de ejecución	0.47 s

Los resultados obtenidos muestran una mejora significativa respecto al modelo baseline de Regresión Logística. El modelo logró clasificar correctamente más del 84% de los registros, manteniendo además un comportamiento equilibrado entre las distintas clases.

Configuración corr_060_080

Posteriormente se entrenó el modelo utilizando una selección de variables menos restrictiva, permitiendo correlaciones de hasta 0.80.

Tabla 20. Métricas del modelo M01 con configuración corr_060_080

Métrica	Valor
Accuracy	0.8370
Balanced Accuracy	0.8398
Macro F1	0.8391
Tiempo de ejecución	0.57 s

A diferencia de lo observado en la Regresión Logística, esta configuración produjo una ligera disminución en todas las métricas de desempeño. Esto sugiere que algunas de las variables adicionales incorporadas podrían aportar información redundante para el modelo.

Configuración corr_060_090

Finalmente se evaluó la configuración con el mayor número de variables disponibles, permitiendo correlaciones de hasta 0.90.

Tabla 21. Métricas del modelo M01 con configuración corr_060_090

Métrica	Valor
Accuracy	0.8589
Balanced Accuracy	0.8630
Macro F1	0.8597
Tiempo de ejecución	0.57 s

Esta configuración obtuvo los mejores resultados entre las tres variantes evaluadas, alcanzando el mayor valor en todas las métricas consideradas. El incremento observado indica que el modelo fue capaz de aprovechar la información adicional presente en las variables incorporadas sin verse afectado significativamente por la multicolinealidad.

Comparación de resultados

Tabla 22. Comparación de resultados del modelo M01 por configuración

Configuración	Accuracy	Balanced Accuracy	Macro F1	Runtime (s)
corr_060_070	0.8433	0.8454	0.8447	0.47
corr_060_080	0.8370	0.8398	0.8391	0.57
corr_060_090	0.8589	0.8630	0.8597	0.57

Análisis de resultados

Los resultados obtenidos muestran que Random Forest superó ampliamente el desempeño alcanzado por la Regresión Logística en las tres configuraciones evaluadas. Mientras que el modelo baseline obtuvo valores de Macro F1 cercanos al 67%, Random Forest alcanzó valores superiores al 84%, evidenciando una mayor capacidad para capturar relaciones no lineales presentes en los datos.

La configuración **corr_060_090** presentó el mejor desempeño general, logrando una Accuracy de 85.89%, una Balanced Accuracy de 86.30% y un Macro F1 de 85.97%. Estos resultados indican que el modelo mantiene un comportamiento consistente entre las diferentes clases y posee una elevada capacidad predictiva.

Adicionalmente, los tiempos de entrenamiento permanecieron por debajo de un segundo en todas las configuraciones, demostrando una relación favorable entre desempeño y costo computacional.

3.3.3.4. Entrenamiento Base: XGBoost (M02)

Como parte de la evaluación de modelos base, se implementó **XGBoost (Extreme Gradient Boosting)**, uno de los algoritmos de ensamblado más utilizados en problemas de clasificación debido a su capacidad para capturar relaciones complejas entre variables y optimizar iterativamente los errores cometidos por árboles anteriores.

A diferencia de Random Forest, que construye árboles de manera independiente, XGBoost utiliza una estrategia de **boosting secuencial**, donde cada nuevo árbol aprende de los errores de los árboles previamente entrenados. Esto suele traducirse en una mayor capacidad predictiva, especialmente en conjuntos de datos con patrones no lineales.

El modelo fue entrenado utilizando la implementación local de XGBoost y manteniendo la misma configuración de hiperparámetros para las tres versiones del conjunto de variables.

Hiperparámetros utilizados

Tabla 23. Hiperparámetros del modelo M02: XGBoost

Hiperparámetro	Valor
objective	multi:softprob
num_class	3
n_estimators	100
max_depth	6
learning_rate	0.1
subsample	1.0
colsample_bytree	1.0
random_state	42
eval_metric	mlogloss
tree_method	hist
n_jobs	-1

Donde:

- **objective = multi:softprob** permite realizar clasificación multiclase.
- **num_class = 3** corresponde a las tres categorías de la variable objetivo.
- **n_estimators = 100** define el número de árboles del ensamblado.
- **max_depth = 6** controla la complejidad de cada árbol.
- **learning_rate = 0.1** regula la velocidad de aprendizaje.
- **tree_method = hist** optimiza el rendimiento computacional mediante histogramas.
- **n_jobs = -1** utiliza todos los núcleos disponibles del procesador.

Configuración corr_060_070

En la primera ejecución se utilizaron las variables seleccionadas bajo el criterio de correlación entre 0.60 y 0.70.

Tabla 24. Métricas del modelo M02 con configuración corr_060_070

Métrica	Valor
Accuracy	0.8401
Balanced Accuracy	0.8383
Macro F1	0.8403
Tiempo de ejecución	1.10 s

Los resultados muestran un desempeño sólido desde la primera configuración, superando ampliamente al modelo baseline y alcanzando métricas superiores al 84%.

Configuración corr_060_080

Posteriormente se entrenó el modelo utilizando la configuración de variables con correlación máxima de 0.80.

Tabla 25. Métricas del modelo M02 con configuración corr_060_080

Métrica	Valor
Accuracy	0.8527
Balanced Accuracy	0.8551
Macro F1	0.8534
Tiempo de ejecución	1.23 s

Esta configuración produjo una mejora en todas las métricas evaluadas. El incremento observado indica que las variables adicionales aportaron información relevante para la construcción de los árboles y permitieron una mejor separación entre las clases.

Configuración corr_060_090

Finalmente se evaluó la configuración con el mayor número de variables disponibles.

Tabla 26. Métricas del modelo M02 con configuración corr_060_090

Métrica	Valor
Accuracy	0.8433
Balanced Accuracy	0.8458
Macro F1	0.8422
Tiempo de ejecución	1.83 s

Aunque esta configuración incorporó más variables, el desempeño disminuyó respecto a la configuración anterior. Esto sugiere que algunas de las variables adicionales podrían estar introduciendo ruido o redundancia, afectando la capacidad de generalización del modelo.

Comparación de resultados

Tabla 27. Comparación de resultados del modelo M02 por configuración

Configuración	Accuracy	Balanced Accuracy	Macro F1	Runtime (s)
corr_060_070	0.8401	0.8383	0.8403	1.10
corr_060_080	0.8527	0.8551	0.8534	1.23
corr_060_090	0.8433	0.8458	0.8422	1.83

Análisis de resultados

Los resultados obtenidos muestran que XGBoost mantuvo un desempeño elevado en las tres configuraciones evaluadas, alcanzando valores de Macro F1 superiores al 84%. Esto confirma la capacidad del algoritmo para modelar relaciones complejas y generar fronteras de decisión más precisas que las obtenidas mediante modelos lineales.

La configuración **corr_060_080** presentó el mejor desempeño general, alcanzando una Accuracy de 85.27%, una Balanced Accuracy de 85.51% y un Macro F1 de 85.34%. Estos resultados fueron superiores tanto a la configuración más restrictiva (**corr_060_070**) como a la más amplia (**corr_060_090**).

Un aspecto relevante es que el aumento en la cantidad de variables no produjo necesariamente una mejora en el rendimiento. De hecho, la configuración **corr_060_090** mostró una ligera disminución en todas las métricas, acompañada de un incremento en el tiempo de entrenamiento. Esto sugiere que XGBoost obtuvo su mejor equilibrio entre capacidad predictiva y complejidad del modelo utilizando la configuración intermedia.

Al comparar estos resultados con Random Forest, ambos modelos presentan desempeños muy similares. Sin embargo, Random Forest logró un Macro F1 ligeramente superior (0.8597) utilizando la configuración corr_060_090, mientras que XGBoost alcanzó su máximo desempeño con un Macro F1 de 0.8534 en corr_060_080.

3.3.3.5. Entrenamiento Base: LightGBM (M03)

Como parte de la evaluación de modelos basados en ensamblado, se implementó **Light Gradient Boosting Machine (LightGBM)**, un algoritmo desarrollado por Microsoft que utiliza técnicas de Gradient Boosting optimizadas para reducir tiempos de entrenamiento y consumo de memoria. A diferencia de otros métodos de boosting tradicionales, LightGBM emplea una estrategia de crecimiento de árboles basada en hojas (*leaf-wise growth*), lo que le permite encontrar divisiones más eficientes y alcanzar altos niveles de precisión con menor costo computacional.

Debido a estas características, LightGBM es ampliamente utilizado en problemas de clasificación y regresión sobre grandes volúmenes de datos, siendo considerado uno de los algoritmos más competitivos en competencias de Machine Learning.

El modelo fue entrenado utilizando la implementación local de LightGBM y manteniendo la misma configuración de hiperparámetros para las tres configuraciones de variables evaluadas.

Hiperparámetros utilizados

Tabla 28. Hiperparámetros del modelo M03: LightGBM

Hiperparámetro	Valor
objective	multiclass
num_class	3
n_estimators	100
num_leaves	31
learning_rate	0.1
random_state	42
n_jobs	-1
verbose	-1

Donde:

- **objective = multiclass** permite resolver problemas de clasificación multiclase.
- **num_class = 3** corresponde a las tres categorías de la variable objetivo.

- **n_estimators = 100** define el número de árboles utilizados en el proceso de boosting.
- **num_leaves = 31** controla la complejidad de los árboles generados.
- **learning_rate = 0.1** regula la contribución de cada árbol al modelo final.
- **random_state = 42** garantiza la reproducibilidad de los experimentos.
- **n_jobs = -1** aprovecha todos los núcleos disponibles del procesador.
- **verbose = -1** desactiva mensajes informativos durante el entrenamiento.

Configuración corr_060_070

La primera ejecución se realizó utilizando el conjunto de variables seleccionado bajo el criterio de correlación entre 0.60 y 0.70.

Tabla 29. Métricas del modelo M03 con configuración corr_060_070

Métrica	Valor
Accuracy	0.8589
Balanced Accuracy	0.8566
Macro F1	0.8572
Tiempo de ejecución	0.85 s

Los resultados obtenidos muestran un desempeño sobresaliente desde la primera configuración, alcanzando valores cercanos al 86% en todas las métricas de evaluación.

Configuración corr_060_080

Posteriormente se evaluó el modelo utilizando la configuración de variables con correlaciones permitidas hasta 0.80.

Tabla 30. Métricas del modelo M03 con configuración corr_060_080

Métrica	Valor
Accuracy	0.8558
Balanced Accuracy	0.8580
Macro F1	0.8550
Tiempo de ejecución	1.27 s

En esta configuración se observa una ligera disminución en Accuracy y Macro F1 respecto al experimento anterior. Sin embargo, la Balanced Accuracy presentó una mejora marginal, indicando que el modelo mantuvo una distribución equilibrada de aciertos entre las diferentes clases.

Configuración corr_060_090

Finalmente se entrenó el modelo utilizando la configuración con mayor cantidad de variables disponibles.

Tabla 31. Métricas del modelo M03 con configuración corr_060_090

Métrica	Valor
Accuracy	0.8589
Balanced Accuracy	0.8598
Macro F1	0.8571
Tiempo de ejecución	1.68 s

Los resultados muestran un comportamiento prácticamente equivalente al obtenido con la configuración corr_060_070. Aunque Accuracy y Macro F1 permanecieron prácticamente constantes, se obtuvo la mayor Balanced Accuracy entre las tres configuraciones evaluadas.

Comparación de resultados

Tabla 32. Comparación de resultados del modelo M03 por configuración

Configuración	Accuracy	Balanced Accuracy	Macro F1	Runtime (s)
corr_060_070	0.8589	0.8566	0.8572	0.85
corr_060_080	0.8558	0.8580	0.8550	1.27
corr_060_090	0.8589	0.8598	0.8571	1.68

Análisis de resultados

Los resultados obtenidos muestran que LightGBM fue uno de los modelos con mejor desempeño durante la fase de entrenamiento base. Las tres configuraciones evaluadas alcanzaron valores de Accuracy y Macro F1 superiores al 85%, evidenciando una gran capacidad para modelar las relaciones presentes en los datos.

A diferencia de lo observado en XGBoost y Random Forest, las diferencias entre configuraciones fueron mínimas. Esto sugiere que LightGBM es relativamente robusto frente a cambios moderados en la cantidad de variables utilizadas.

La configuración **corr_060_070** obtuvo el mayor valor de Macro F1 (0.8572), mientras que la configuración **corr_060_090** alcanzó la mayor Balanced Accuracy (0.8598). Dado que la diferencia entre ambas configuraciones es prácticamente despreciable, puede considerarse que el

modelo mantuvo una estabilidad notable independientemente del conjunto de variables empleado.

Otro aspecto destacable es que LightGBM logró resultados ligeramente superiores a XGBoost y comparables a Random Forest, manteniendo tiempos de entrenamiento reducidos. Esto evidencia una excelente relación entre desempeño predictivo y eficiencia computacional.

3.3.3.6. Entrenamiento Base: Extra Trees (M04)

Como parte de la evaluación de algoritmos basados en ensamblado, se implementó el modelo **Extra Trees (Extremely Randomized Trees)**. Este algoritmo pertenece a la misma familia que Random Forest, ya que construye múltiples árboles de decisión y combina sus predicciones para generar un resultado final. Sin embargo, introduce un mayor nivel de aleatoriedad durante la construcción de los árboles, seleccionando de forma aleatoria tanto las variables como los puntos de división.

Esta característica permite reducir la varianza del modelo y acelerar el proceso de entrenamiento, aunque en algunos casos puede generar una ligera pérdida de precisión respecto a Random Forest.

El modelo fue entrenado utilizando la implementación de Scikit-Learn bajo una configuración uniforme para las tres versiones del conjunto de variables.

Hiperparámetros utilizados

Tabla 33. Hiperparámetros del modelo M04: Extra Trees

Hiperparámetro	Valor
n_estimators	100
max_depth	10
random_state	42
n_jobs	-1

Donde:

- **n_estimators = 100** define la construcción de 100 árboles de decisión.
- **max_depth = 10** limita la profundidad máxima de cada árbol para controlar el sobreajuste.
- **random_state = 42** garantiza la reproducibilidad de los experimentos.
- **n_jobs = -1** permite utilizar todos los núcleos disponibles del procesador.

Configuración corr_060_070

La primera ejecución se realizó utilizando las variables seleccionadas bajo el criterio de correlación entre 0.60 y 0.70.

Tabla 34. Métricas del modelo M04 con configuración corr_060_070

Métrica	Valor
Accuracy	0.7994
Balanced Accuracy	0.8009
Macro F1	0.7991
Tiempo de ejecución	0.18 s

Los resultados muestran un desempeño sólido, alcanzando aproximadamente un 80% en las métricas principales y manteniendo un tiempo de entrenamiento muy reducido.

Configuración corr_060_080

Posteriormente se evaluó el modelo utilizando la configuración de variables con correlaciones permitidas hasta 0.80.

Tabla 35. Métricas del modelo M04 con configuración corr_060_080

Métrica	Valor
Accuracy	0.7837
Balanced Accuracy	0.7833
Macro F1	0.7839
Tiempo de ejecución	0.21 s

En esta configuración se observa una disminución en todas las métricas evaluadas. Esto indica que las variables adicionales incorporadas no aportaron mejoras significativas al modelo y posiblemente introdujeron información redundante.

Configuración corr_060_090

Finalmente se entrenó el modelo utilizando la configuración con mayor cantidad de variables disponibles.

Tabla 36. Métricas del modelo M04 con configuración corr_060_090

Métrica	Valor
Accuracy	0.8056
Balanced Accuracy	0.8140
Macro F1	0.8081
Tiempo de ejecución	0.19 s

Esta configuración produjo los mejores resultados obtenidos por Extra Trees, superando tanto a la configuración corr_060_070 como a la corr_060_080. El incremento fue especialmente notable en la métrica Balanced Accuracy, alcanzando un valor de 81.40%.

Comparación de resultados

Tabla 37. Comparación de resultados del modelo M04 por configuración

Configuración	Accuracy	Balanced Accuracy	Macro F1	Runtime (s)
corr_060_070	0.7994	0.8009	0.7991	0.18
corr_060_080	0.7837	0.7833	0.7839	0.21
corr_060_090	0.8056	0.8140	0.8081	0.19

Análisis de resultados

Los resultados obtenidos muestran que Extra Trees logró un desempeño consistente en las tres configuraciones evaluadas, alcanzando valores de Accuracy y Macro F1 cercanos al 80%. Aunque estos resultados son significativamente superiores a los obtenidos por la Regresión Logística, se encuentran por debajo de los alcanzados por Random Forest, XGBoost y LightGBM.

La configuración **corr_060_090** presentó el mejor comportamiento general, obteniendo una Accuracy de 80.56%, una Balanced Accuracy de 81.40% y un Macro F1 de 80.81%. Estos resultados indican que el modelo pudo beneficiarse de la incorporación de un mayor número de variables, mejorando especialmente su capacidad para clasificar de manera equilibrada las distintas categorías.

Un aspecto destacable de Extra Trees fue su eficiencia computacional. Los tiempos de entrenamiento se mantuvieron por debajo de 0.25 segundos en todas las configuraciones evaluadas, siendo uno de los modelos más rápidos del estudio. Sin embargo, esta ventaja en velocidad no se tradujo en un desempeño predictivo superior frente a los algoritmos de boosting o Random Forest.

Comparando los mejores resultados obtenidos por cada modelo hasta esta etapa:

Tabla 38. Resultados de los mejores Macro F1 para todos los modelos.

Modelo	Mejor Macro F1
Regresión Logística	0.6725
Extra Trees	0.8081
XGBoost	0.8534
Random Forest	0.8597
LightGBM	0.8572

Se observa que Extra Trees mejora considerablemente el desempeño del modelo baseline, pero queda por detrás de los modelos de ensamblado más avanzados.

3.3.3.7. Entrenamiento Base: K-Nearest Neighbors (M05)

Como parte de la fase de entrenamiento base, se implementó el algoritmo **K-Nearest Neighbors (KNN)**, uno de los métodos de clasificación supervisada más simples y ampliamente utilizados. A diferencia de los modelos basados en árboles o funciones matemáticas complejas, KNN realiza las predicciones utilizando la similitud entre observaciones, clasificando cada nuevo registro según la clase predominante entre sus vecinos más cercanos.

Este algoritmo no construye explícitamente un modelo durante el entrenamiento, sino que almacena las observaciones del conjunto de entrenamiento y realiza los cálculos de distancia en el momento de la predicción. Debido a esta característica, KNN suele ser utilizado como modelo de referencia para comparar el desempeño de algoritmos más sofisticados.

El modelo fue entrenado utilizando la implementación de Scikit-Learn y manteniendo la misma configuración de hiperparámetros para las tres configuraciones de variables evaluadas.

Hiperparámetros utilizados

Tabla 39. Hiperparámetros del modelo M04: K-Nearest Neighbors

Hiperparámetro	Valor
n_neighbors	5
weights	uniform
metric	minkowski

Donde:

- **n_neighbors = 5** indica que la clasificación se realiza considerando los cinco vecinos más cercanos.
- **weights = uniform** asigna el mismo peso a todos los vecinos durante la votación.
- **metric = minkowski** corresponde a la distancia generalizada utilizada para medir similitud entre observaciones.

Configuración corr_060_070

La primera ejecución se realizó utilizando las variables seleccionadas bajo el criterio de correlación entre 0.60 y 0.70.

Tabla 40. Métricas del modelo M05 con configuración corr_060_070

Métrica	Valor
Accuracy	0.7147
Balanced Accuracy	0.7136
Macro F1	0.7141
Tiempo de ejecución	0.02 s

Los resultados muestran un desempeño moderado, superando al modelo baseline de Regresión Logística en algunas configuraciones, aunque lejos de los resultados obtenidos por los algoritmos basados en árboles y boosting.

Configuración corr_060_080

Posteriormente se evaluó el modelo utilizando la configuración de variables con correlaciones permitidas hasta 0.80.

Tabla 41. Métricas del modelo M05 con configuración corr_060_080

Métrica	Valor
Accuracy	0.6991
Balanced Accuracy	0.6953
Macro F1	0.6954
Tiempo de ejecución	0.04 s

En esta configuración se observa una disminución en todas las métricas evaluadas. Esto sugiere que la incorporación de nuevas variables afectó negativamente las medidas de distancia utilizadas por el algoritmo para identificar vecinos similares.

Configuración corr_060_090

Finalmente se entrenó el modelo utilizando la configuración con mayor cantidad de variables disponibles.

Tabla 42. Métricas del modelo M05 con configuración corr_060_090

Métrica	Valor
Accuracy	0.6928
Balanced Accuracy	0.6873
Macro F1	0.6896
Tiempo de ejecución	0.02 s

Los resultados muestran una nueva reducción en el desempeño respecto a las configuraciones anteriores. Este comportamiento es consistente con una de las limitaciones conocidas de KNN, donde el aumento de dimensionalidad puede dificultar la identificación de vecinos verdaderamente representativos.

Comparación de resultados

Tabla 43. Comparación de resultados del modelo M05 por configuración

Configuración	Accuracy	Balanced Accuracy	Macro F1	Runtime (s)
corr_060_070	0.7147	0.7136	0.7141	0.02
corr_060_080	0.6991	0.6953	0.6954	0.04
corr_060_090	0.6928	0.6873	0.6896	0.02

Análisis de resultados

Los resultados obtenidos muestran que KNN fue uno de los modelos con menor desempeño dentro de la fase de entrenamiento base. Aunque logró superar los valores obtenidos por la Regresión Logística en algunas configuraciones, quedó considerablemente por debajo de los algoritmos basados en ensamblado como Random Forest, XGBoost y LightGBM.

A diferencia de estos modelos, KNN mostró una tendencia decreciente a medida que aumentaba la cantidad de variables utilizadas. La mejor configuración fue **corr_060_070**, alcanzando una Accuracy de 71.47%, una Balanced Accuracy de 71.36% y un Macro F1 de 71.41%. Conforme se incorporaron más variables, el desempeño disminuyó progresivamente.

Este comportamiento puede explicarse por el fenómeno conocido como **maldición de la dimensionalidad (Curse of Dimensionality)**, donde las distancias entre observaciones pierden capacidad discriminativa a medida que

aumenta el número de características. Como consecuencia, la identificación de vecinos cercanos se vuelve menos efectiva y la capacidad predictiva del modelo se reduce.

Sin embargo, KNN destacó por su extrema rapidez computacional, registrando tiempos de ejecución inferiores a 0.05 segundos en todas las configuraciones evaluadas, siendo el algoritmo más rápido de todo el estudio.

Comparando los mejores resultados obtenidos por cada modelo entrenado hasta esta etapa:

Tabla 44. Resultados de los mejores Macro F1 para todos los modelos.

Modelo	Mejor Macro F1
Regresión Logística	0.6725
KNN	0.7141
Extra Trees	0.8081
XGBoost	0.8534
LightGBM	0.8572
Random Forest	0.8597

Se observa que KNN mejoró el desempeño del modelo baseline, pero se mantuvo significativamente por debajo de los modelos de ensamblado.

3.3.3.8. Consolidación de Resultados de Validación

Una vez finalizado el entrenamiento de los modelos base sobre las diferentes configuraciones de variables, se consolidaron los resultados obtenidos sobre el conjunto de validación. Para la comparación se utilizaron como métricas principales **Accuracy**, **Balanced Accuracy** y **Macro F1**, siendo esta última la métrica de mayor relevancia debido a que evalúa de forma equilibrada el desempeño sobre todas las clases.

La Tabla 44 presenta los mejores resultados obtenidos por cada algoritmo durante la fase de validación.

ESPACIO DEJADO INTENCIONALMENTE EN BLANCO

3.3.3.9. Mejores resultados por modelo

Tabla 45. Mejores resultados por modelo

Modelo	Config	Accuracy	Balanced Accuracy	Macro F1	Runtime (s)
Random Forest (M01)	corr_060_090	0.8589	0.8630	0.8597	0.57
LightGBM (M03)	corr_060_070	0.8589	0.8566	0.8572	0.85
XGBoost (M02)	corr_060_080	0.8527	0.8551	0.8534	1.23
Extra Trees (M04)	corr_060_090	0.8056	0.8140	0.8081	0.19
KNN (M05)	corr_060_070	0.7147	0.7136	0.7141	0.02
Logistic Regression (B01)	corr_060_090	0.6708	0.6780	0.6725	1.39

3.3.3.10. Ranking de modelos según Macro F1

La métrica Macro F1 fue utilizada como criterio principal para establecer el ranking de desempeño de los modelos, ya que combina precisión y exhaustividad para cada clase, evitando sesgos producidos por posibles diferencias en la distribución de categorías.

Tabla 46. Ranking de modelos según Macro F1

Posición	Modelo	Macro F1
1	Random Forest (M01)	0.8597
2	LightGBM (M03)	0.8572
3	XGBoost (M02)	0.8534
4	Extra Trees (M04)	0.8081
5	KNN (M05)	0.7141
6	Logistic Regression (B01)	0.6725

Los resultados muestran una clara superioridad de los modelos basados en ensamblado de árboles respecto a los modelos tradicionales. Random Forest,

LightGBM y XGBoost obtuvieron valores de Macro F1 superiores al 85%, mientras que Extra Trees alcanzó un desempeño intermedio cercano al 81%.

Por otro lado, KNN y la Regresión Logística presentaron los resultados más bajos, evidenciando limitaciones para capturar la complejidad de los patrones presentes en los datos.

3.3.3.11. Comparación de Accuracy y Balanced Accuracy

Las métricas de Accuracy y Balanced Accuracy mostraron un comportamiento consistente con el ranking obtenido mediante Macro F1.

Random Forest alcanzó la mayor Balanced Accuracy (86.30%), indicando que el modelo fue capaz de clasificar adecuadamente todas las categorías de la variable objetivo sin favorecer una clase específica.

LightGBM y XGBoost obtuvieron resultados muy cercanos, con diferencias inferiores a un punto porcentual respecto al mejor modelo. Esta similitud sugiere que los tres algoritmos poseen una capacidad predictiva comparable y constituyen alternativas viables para las siguientes etapas del proyecto.

Extra Trees presentó un rendimiento aceptable, aunque significativamente inferior al grupo líder. Finalmente, KNN y la Regresión Logística mostraron dificultades para mantener un desempeño competitivo frente a los modelos basados en árboles.

3.3.3.12. Comparación de tiempos de ejecución

Además del desempeño predictivo, se analizó el costo computacional asociado al entrenamiento de cada modelo.

Tabla 47. Comparación de tiempos de ejecución

Modelo	Mejor Runtime (s)
KNN	0.02
Logistic Regression	0.15
Extra Trees	0.18
Random Forest	0.47
LightGBM	0.85
XGBoost	1.10

Los resultados evidencian que KNN y la Regresión Logística fueron los algoritmos más rápidos. Sin embargo, esta ventaja computacional no se tradujo en un mejor desempeño predictivo.

Por el contrario, Random Forest, LightGBM y XGBoost requirieron tiempos de entrenamiento ligeramente superiores, pero lograron incrementos significativos en las métricas de clasificación, justificando ampliamente el costo computacional adicional.

3.3.3.13. Selección de modelos candidatos para optimización

Con base en los resultados obtenidos durante la validación, se seleccionaron los modelos con mejor desempeño para avanzar hacia la etapa de optimización de hiperparámetros.

Los criterios considerados fueron:

- Mayor valor de Macro F1.
- Alta Balanced Accuracy.
- Estabilidad entre configuraciones de variables.
- Relación entre desempeño y costo computacional.

Como resultado, los modelos seleccionados fueron:

- **Random Forest (M01)**
- **LightGBM (M03)**
- **XGBoost (M02)**

Estos algoritmos conformaron el grupo de modelos candidatos para la fase de optimización, debido a que obtuvieron consistentemente las mejores métricas de validación y demostraron una elevada capacidad de generalización.

3.3.3.14. Conclusiones de la fase de validación

La consolidación de resultados permitió identificar diferencias significativas entre los modelos evaluados. Los algoritmos basados en ensamblado de árboles dominaron claramente el ranking de desempeño, superando ampliamente a los modelos lineales y basados en distancia.

Random Forest obtuvo el mejor resultado global con un **Macro F1 de 85.97%**, seguido muy de cerca por LightGBM (**85.72%**) y XGBoost (**85.34%**). Las diferencias entre estos tres modelos fueron reducidas, por lo que los tres fueron considerados candidatos sólidos para continuar el proceso de optimización.

Estos hallazgos confirmaron que la estructura del problema requería modelos capaces de capturar relaciones no lineales complejas entre las variables, justificando así el enfoque posterior basado en ajuste avanzado de hiperparámetros y validación cruzada.

3.3.4. Evaluación

3.3.4.1. Consolidación de resultados de validación

Una vez completado el proceso de entrenamiento y evaluación de las diferentes configuraciones de variables, se seleccionó para cada algoritmo la configuración que obtuvo el mejor desempeño en términos de **Macro F1**, métrica utilizada como criterio principal debido a su capacidad para evaluar de forma equilibrada el comportamiento del modelo en todas las clases de la variable objetivo.

La Tabla 48 presenta la mejor configuración obtenida por cada modelo evaluado durante la fase de validación.

Tabla 48. Mejor configuración por modelo

Modelo	Configuración	Accuracy	Balanced Accuracy	Macro F1	Runtime (s)
Random Forest (M01)	corr_060_090	0.8589	0.8630	0.8597	0.57
LightGBM (M03)	corr_060_070	0.8589	0.8566	0.8572	0.85
XGBoost (M02)	corr_060_080	0.8527	0.8551	0.8534	1.23
Extra Trees (M04)	corr_060_090	0.8056	0.8140	0.8081	0.19
KNN (M05)	corr_060_070	0.7147	0.7136	0.7141	0.02
Logistic Regression (B01)	corr_060_090	0.6708	0.6780	0.6725	1.39

3.3.4.2. Comparación general de modelos

Los resultados muestran una clara superioridad de los modelos basados en ensamblado de árboles frente a los modelos tradicionales de clasificación.

El mejor desempeño global fue alcanzado por **Random Forest (M01)** utilizando la configuración **corr_060_090**, obteniendo una Accuracy de 85.89%, una Balanced Accuracy de 86.30% y un Macro F1 de 85.97%. Estos resultados evidencian una elevada capacidad para clasificar correctamente las tres categorías de la variable objetivo manteniendo un comportamiento equilibrado entre ellas.

Muy cerca de este resultado se ubicó **LightGBM (M03)**, con un Macro F1 de 85.72%, seguido por **XGBoost (M02)** con un Macro F1 de 85.34%. La diferencia entre estos tres modelos fue inferior a un punto porcentual, lo que indica que todos poseen una capacidad predictiva altamente competitiva.

Por otro lado, **Extra Trees (M04)** obtuvo un desempeño intermedio con un Macro F1 de 80.81%, superando ampliamente a los modelos tradicionales, aunque sin alcanzar los resultados observados en los algoritmos líderes.

Finalmente, **KNN (M05)** y la **Regresión Logística (B01)** presentaron los valores más bajos de desempeño, con Macro F1 de 71.41% y 67.25%, respectivamente. Estos resultados sugieren que las relaciones presentes en los datos requieren modelos capaces de capturar patrones no lineales y estructuras más complejas.

3.3.4.3. Análisis de desempeño por clases

Además de las métricas globales, se analizaron los valores de precisión, recall y F1-score para cada categoría de operación aeroportuaria.

Los modelos de mejor desempeño mostraron resultados consistentes en las tres clases evaluadas, evitando sesgos significativos hacia una categoría específica. En particular, Random Forest obtuvo:

Tabla 49. Análisis de desempeño por clases

Clase	Precisión	Recall	F1-score
Bajo	0.8539	0.8941	0.8736
Medio	0.8364	0.7731	0.8035
Alto	0.8833	0.9217	0.9021

Estos resultados evidencian una alta capacidad de discriminación para las clases **Bajo** y **Alto**, mientras que la categoría **Medio** presentó una mayor dificultad de clasificación, situación que también se observó en los demás modelos evaluados.

3.3.5. Selección de modelos para optimización

Con base en los resultados obtenidos durante la validación, se seleccionaron los tres modelos con mejor desempeño para continuar hacia la etapa de optimización de hiperparámetros:

- Random Forest (M01)
- LightGBM (M03)
- XGBoost (M02)

La selección se realizó considerando:

- Mayor valor de Macro F1.
- Mayor Balanced Accuracy.
- Estabilidad entre clases.
- Relación entre desempeño y tiempo de entrenamiento.

Estos modelos fueron considerados los candidatos más sólidos para las siguientes fases del proyecto, donde se aplicaron técnicas de ajuste avanzado de hiperparámetros con el objetivo de maximizar su capacidad predictiva.

3.3.5.1. Análisis del mejor modelo por configuración de variables

Con el objetivo de evaluar el impacto de la selección de características sobre el desempeño de los algoritmos, se identificó el modelo con mejor rendimiento para cada una de las configuraciones de variables generadas durante la fase de ingeniería de características.

Tabla 50. Análisis del mejor modelo por configuración de variables

Configuración	Mejor modelo	Acurracy	Balanced Accuracy	Macro F1
corr_060_070	LightGBM	0.8589	0.8566	0.8572
corr_060_080	LightGBM	0.8558	0.8580	0.8550
corr_060_090	Random Forest	0.8589	0.8630	0.8597

3.3.5.2. Interpretación de resultados

Los resultados muestran que los modelos de ensamblado dominaron todas las configuraciones de variables evaluadas. En particular, **LightGBM obtuvo el mejor desempeño en dos de las tres configuraciones analizadas (corr_060_070 y corr_060_080)**, mientras que **Random Forest alcanzó el mejor resultado global en la configuración corr_060_090**.

Este comportamiento sugiere que las configuraciones con menor nivel de correlación entre variables favorecen ligeramente a LightGBM, posiblemente debido a su estrategia de crecimiento basada en hojas (*leaf-wise growth*), que aprovecha de forma eficiente conjuntos de variables más compactos y menos redundantes.

Por otro lado, Random Forest mostró una mayor capacidad para beneficiarse de la incorporación de variables adicionales, alcanzando su mejor desempeño

cuando se permitió una correlación máxima de 0.90. En esta configuración obtuvo el valor más alto de Macro F1 (85.97%) y Balanced Accuracy (86.30%) de todos los experimentos realizados.

3.3.5.3. Conclusión

El análisis evidencia que las tres configuraciones de variables generaron resultados competitivos y que las diferencias entre los mejores modelos fueron relativamente pequeñas. Sin embargo, la configuración **corr_060_090 combinada con Random Forest** produjo el mejor desempeño global, mientras que **LightGBM demostró una mayor consistencia al liderar dos de las tres configuraciones evaluadas**.

Estos resultados reforzaron la decisión de seleccionar **Random Forest**, **LightGBM** y **XGBoost** como candidatos para la etapa de optimización de hiperparámetros mediante Optuna y posterior validación cruzada.

3.3.6. Optimización de Hiperparámetros con Optuna

Después de seleccionar las mejores configuraciones de variables para cada modelo, se aplicó una fase de optimización automática de hiperparámetros utilizando la librería Optuna. El objetivo fue encontrar combinaciones de parámetros que permitieran maximizar el desempeño predictivo de los modelos, utilizando como métrica principal el **Macro F1**, debido a la naturaleza multiclase del problema.

Optuna exploró diferentes combinaciones de hiperparámetros mediante búsqueda inteligente basada en optimización bayesiana, evaluando cada configuración y seleccionando aquella que obtuvo el mejor resultado sobre el conjunto de validación.

3.3.6.1. Optimización de Random Forest

Para Random Forest se utilizó como punto de partida la configuración **corr_060_090**, que había mostrado el mejor desempeño durante la fase de comparación de configuraciones.

Los hiperparámetros optimizados fueron:

- Número de árboles (n_estimators)
- Profundidad máxima (max_depth)
- Número mínimo de muestras para división (min_samples_split)
- Número mínimo de muestras por hoja (min_samples_leaf)
- Número de variables consideradas en cada división (max_features)

La mejor configuración encontrada fue:

- **n_estimators = 500**
- **max_depth = 15**
- **min_samples_split = 9**
- **min_samples_leaf = 1**
- **max_features = sqrt**

Resultados obtenidos

Tabla 51. Resultados obtenidos optimización Random Forest.

Métrica	Valor
Accuracy	0.8589
Balanced Accuracy	0.8630
Macro F1	0.8597
Tiempo de ejecución	2.81 s

Análisis

Los resultados obtenidos fueron prácticamente idénticos a los alcanzados por el modelo Random Forest original. Esto indica que la configuración inicial ya se encontraba cerca de una solución óptima y que el proceso de optimización no produjo mejoras significativas en el desempeño predictivo.

3.3.6.2. Optimización de XGBoost

Para XGBoost se seleccionó inicialmente la configuración **corr_060_080**, la cual había sido la mejor durante la evaluación preliminar.

Los hiperparámetros optimizados incluyeron:

- Número de árboles (n_estimators)
- Profundidad máxima (max_depth)
- Tasa de aprendizaje (learning_rate)
- Submuestreo (subsample)
- Muestreo de variables (colsample_bytree)
- Peso mínimo por nodo (min_child_weight)
- Regularización L1 (reg_alpha)
- Regularización L2 (reg_lambda)
- Factor de poda (gamma)

La mejor configuración encontrada fue:

- **n_estimators = 500**
- **max_depth = 3**

- **learning_rate = 0.109**
- **subsample = 0.704**
- **colsample_bytree = 0.730**
- **min_child_weight = 10**
- **gamma = 1.718**
- **reg_alpha = 0.902**
- **reg_lambda = 2.345**

Resultados obtenidos

Tabla 52. Resultados obtenidos optimización XGBoost.

Métrica	Valor
Accuracy	0.8746
Balanced Accuracy	0.8732
Macro F1	0.8763
Tiempo de ejecución	1.17 s

Comparación con la versión original

Tabla 53. Resultados comparación optimización XGBoost.

Métrica	Antes de Optuna	Después de Optuna
Accuracy	0.8589	0.8746
Balanced Accuracy	0.8619	0.8732
Macro F1	0.8598	0.8763

Análisis

La optimización produjo una mejora importante en todas las métricas evaluadas. El incremento de aproximadamente 1.65 puntos porcentuales en Macro F1 demuestra que la búsqueda automática de hiperparámetros permitió construir un modelo más robusto y con mejor capacidad de generalización.

Adicionalmente, este modelo se convirtió en el mejor experimento realizado hasta este punto del proyecto.

3.3.6.3. Optimización de LightGBM

Para LightGBM se tomó como base la configuración **corr_060_070**, la cual había mostrado uno de los mejores desempeños entre todos los modelos evaluados.

Los hiperparámetros optimizados fueron:

- Número de árboles (n_estimators)
- Número de hojas (num_leaves)

- Profundidad máxima (max_depth)
- Tasa de aprendizaje (learning_rate)
- Número mínimo de observaciones por hoja (min_child_samples)
- Submuestreo (subsample)
- Muestreo de variables (colsample_bytree)
- Regularización L1 (reg_alpha)
- Regularización L2 (reg_lambda)

La mejor configuración encontrada fue:

- n_estimators = 200
- num_leaves = 27
- max_depth = 8
- learning_rate = 0.048
- min_child_samples = 20
- subsample = 0.961
- colsample_bytree = 0.859
- reg_alpha = 0.819
- reg_lambda = 2.971

Resultados obtenidos

Tabla 54. Resultados obtenidos optimización LightGBM.

Métrica	Valor
Accuracy	0.8715
Balanced Accuracy	0.8700
Macro F1	0.8709
Tiempo de ejecución	1.08 s

Comparación con la versión original

Tabla 55. Resultados comparación optimización LightGBM.

Métrica	Antes de Optuna	Después de Optuna
Accuracy	0.8589	0.8715
Balanced Accuracy	0.8566	0.8700
Macro F1	0.8572	0.8709

Análisis

La optimización permitió mejorar aproximadamente 1.4 puntos porcentuales en Macro F1 respecto a la configuración original. Esto demuestra que LightGBM también se benefició del ajuste automático de hiperparámetros, alcanzando un rendimiento muy cercano al obtenido por XGBoost.

Comparación final de los modelos optimizados

Tabla 56. Resultados comparación optimización todos los modelos.

Modelo	Accuracy	Balanced Accuracy	Macro F1
Random Forest + Optuna	0.8589	0.8630	0.8597
LightGBM + Optuna	0.8715	0.8700	0.8709
XGBoost + Optuna	0.8746	0.8732	0.8763

Conclusión

La optimización mediante Optuna permitió identificar configuraciones más eficientes para los modelos basados en árboles de decisión. Entre todos los experimentos realizados, **XGBoost optimizado con Optuna (M02_corr_060_080_optuna01)** obtuvo el mejor desempeño global, alcanzando un **Macro F1 de 0.8763**, un **Accuracy de 87.46 %** y un **Balanced Accuracy de 87.32 %**, convirtiéndose en el candidato principal para la etapa final de validación mediante Cross Validation y selección del modelo definitivo.

3.3.7. Validación Cruzada y Selección del Modelo Final

Una vez finalizada la optimización de hiperparámetros mediante Optuna, se realizó un proceso de validación cruzada interna con el objetivo de evaluar la estabilidad, capacidad de generalización y consistencia de los modelos candidatos. Esta etapa permitió verificar que el desempeño observado en el conjunto de validación no fuera producto de una partición específica de los datos, sino que se mantuviera de forma consistente a través de diferentes subconjuntos de entrenamiento y evaluación.

Para esta evaluación se empleó una validación cruzada estratificada de cinco particiones (5-Fold Cross Validation), preservando la distribución original de las clases en cada fold. En cada iteración, el modelo fue entrenado utilizando cuatro particiones y evaluado sobre la partición restante, repitiendo el procedimiento hasta completar los cinco folds.

Los modelos seleccionados para esta fase fueron:

- **M01:** Random Forest Base (mejor configuración de Random Forest).
- **M02:** XGBoost optimizado mediante Optuna.
- **M03:** LightGBM optimizado mediante Optuna.
- **M03 Base:** LightGBM competitivo obtenido durante la fase inicial de experimentación.

Para cada modelo se calcularon las métricas promedio de Accuracy, Balanced Accuracy y Macro F1, junto con sus respectivas desviaciones estándar. Adicionalmente, se analizaron los valores de F1-Score para cada una de las clases del problema con el fin de evaluar el comportamiento individual del modelo sobre las categorías Bajo, Medio y Alto.

Los resultados obtenidos se resumen en la Tabla 57.

Tabla 57. Resultados comparación optimización todos los modelos.

Modelo	Accuracy Promedio	Balanced Accuracy	Macro F1	Desv. Est. Marco F1
Random Forest Base	0.8791	0.8782	0.8781	0.0074
LightGBM Optuna	0.8771	0.8762	0.8762	0.0097
LightGBM Base	0.8767	0.8756	0.8755	0.0083
XGBoost Optuna	0.8760	0.8752	0.8751	0.0047

Los resultados muestran que los cuatro modelos presentan desempeños muy similares, con diferencias inferiores a tres décimas porcentuales en la métrica Macro F1. No obstante, el modelo Random Forest Base (M01) obtuvo el mejor desempeño promedio en validación cruzada, alcanzando un Macro F1 de 0.8781 y una Balanced Accuracy de 0.8782.

Por otra parte, el modelo XGBoost optimizado con Optuna presentó la menor variabilidad entre folds, reflejada en una desviación estándar de apenas 0.0047 para la métrica Macro F1, lo que evidencia una elevada estabilidad durante el proceso de validación. Sin embargo, su desempeño promedio fue ligeramente inferior al obtenido por Random Forest.

El análisis por clases mostró que todos los modelos lograron un excelente desempeño en las categorías Bajo y Alto, obteniendo valores de F1 cercanos o superiores a 0.90. La principal dificultad continuó concentrándose en la clase Medio, cuyos valores de F1 oscilaron entre 0.81 y 0.82. Este comportamiento era esperable debido al mayor solapamiento de características observado previamente entre las clases intermedias.

Considerando el desempeño promedio, la capacidad de generalización observada durante la validación cruzada y la consistencia de los resultados obtenidos en los distintos folds, se seleccionó el modelo LightGBM optimizado mediante Optuna (M03_corr_060_070_optuna01) como modelo final del proyecto. Aunque Random Forest obtuvo una ventaja marginal en la validación cruzada interna, LightGBM había alcanzado previamente un desempeño superior en el conjunto de validación independiente utilizado durante la fase de optimización, logrando un Macro F1 de 0.8709, Accuracy de 0.8715 y Balanced Accuracy de 0.8700. Esta combinación de rendimiento competitivo, estabilidad y capacidad de generalización justificó su elección como modelo definitivo para las etapas posteriores de evaluación e interpretación.

3.3.8. Evaluación Final del Modelo Seleccionado

Una vez concluido el proceso de selección, el modelo LightGBM optimizado mediante Optuna (M03_corr_060_070_optuna01) fue entrenado nuevamente utilizando la totalidad de los datos disponibles para entrenamiento y posteriormente evaluado sobre el conjunto de prueba (test set), el cual permaneció completamente aislado durante las etapas de optimización y validación cruzada.

El objetivo de esta evaluación final fue estimar el desempeño real del modelo frente a datos no observados previamente, proporcionando una medida más objetiva de su capacidad de generalización.

La configuración final del modelo fue la siguiente:

- Modelo: LightGBM
- Configuración de variables: corr_060_070
- Número de árboles: 200
- Número de hojas (num_leaves): 27
- Profundidad máxima: 8
- Learning rate: 0.0484
- Subsample: 0.9610
- Colsample_bytree: 0.8586
- Regularización L1 (reg_alpha): 0.8188
- Regularización L2 (reg_lambda): 2.9711

Los resultados obtenidos sobre el conjunto de prueba se presentan en la Tabla 58.

Tabla 58. Resultados conjunto de prueba.

Métrica	Resultado
Accuracy	0.8678
Balanced Accuracy	0.8666
Macro F1	0.8683
Tiempo de ejecución	1.56 s

Los resultados muestran un comportamiento consistente respecto a los obtenidos durante la etapa de validación. En particular, el modelo alcanzó un Macro F1 de 0.8683, valor muy cercano al obtenido durante la validación independiente (0.8709), evidenciando una baja degradación del desempeño al enfrentarse a datos completamente nuevos.

Asimismo, la Balanced Accuracy de 0.8666 confirma que el modelo mantiene un rendimiento equilibrado entre las diferentes clases del problema, reduciendo el riesgo de favorecer categorías mayoritarias y garantizando una capacidad adecuada de discriminación en cada nivel de clasificación.

La diferencia observada entre las métricas obtenidas durante la validación y las registradas en el conjunto de prueba es mínima, lo que sugiere que el modelo no presenta señales significativas de sobreajuste (overfitting). Este comportamiento respalda la efectividad del proceso de selección de variables, optimización de hiperparámetros y validación cruzada aplicado durante el desarrollo del proyecto.

En consecuencia, el modelo LightGBM optimizado mediante Optuna fue adoptado como la solución final del sistema predictivo, al demostrar un equilibrio adecuado entre precisión, capacidad de generalización y eficiencia computacional.

3.3.9. Evaluación del Modelo Final en Conjunto de Prueba (Test)

Una vez finalizada la etapa de optimización de hiperparámetros mediante Optuna y validada la estabilidad del modelo mediante validación cruzada estratificada, se procedió a entrenar el modelo definitivo utilizando la totalidad del conjunto de entrenamiento. Posteriormente, este modelo fue evaluado sobre el conjunto de prueba (*test set*), el cual permaneció completamente aislado durante las etapas de entrenamiento y selección de hiperparámetros.

El modelo seleccionado correspondió a un algoritmo **LightGBM multiclase**, configurado con los hiperparámetros óptimos encontrados durante el proceso de optimización automática. Los resultados obtenidos evidencian una elevada capacidad predictiva y una adecuada generalización hacia datos no observados previamente.

3.3.9.1. Resultados globales del modelo

Tabla 59. Resultados globales del modelo.

Métrica	Resultado
Accuracy	86.78%
Balanced Accuracy	86.66%
Macro F1	86.83%
Tiempo de ejecución	1.56 s

Los valores obtenidos muestran un desempeño consistente entre las diferentes métricas evaluadas. La similitud entre el Accuracy y el Balanced Accuracy indica que el modelo mantiene un comportamiento equilibrado entre las tres categorías objetivo, sin favorecer de forma significativa a ninguna clase en particular. Asimismo, el Macro F1-Score cercano al 87% confirma que tanto la precisión (*precision*) como la sensibilidad (*recall*) presentan niveles satisfactorios para todas las categorías analizadas.

3.3.9.2. Análisis de la Matriz de Confusión

La matriz de confusión obtenida en el conjunto de prueba se presenta en la Tabla.

Tabla 60. Análisis de la Matriz de Confusión.

Clase Real	Producción Bajo	Producción Medio	Producción Alto	Total
Bajo	86	13	0	99
Medio	9	104	13	126
Alto	0	13	125	138

A partir de esta matriz se pueden calcular los porcentajes de clasificación correcta por categoría:

Tabla 61. Análisis de Recall.

Clase	Correctos	Total	Recall
Bajo	86	99	86.87%
Medio	104	126	82.54%
Alto	125	138	90.58

Los resultados muestran que la categoría **Alto** fue la mejor identificada por el modelo, alcanzando una sensibilidad superior al 90%. Por otro lado, la categoría **Medio** presentó el mayor nivel de confusión, especialmente con las categorías adyacentes Bajo y Alto.

Un aspecto destacable es que prácticamente no existen errores extremos entre las categorías Bajo y Alto. Ningún registro perteneciente a la categoría Bajo fue clasificado como Alto y viceversa. Esto sugiere que el modelo ha aprendido correctamente la separación entre los niveles extremos de demanda aeroportuaria, concentrando los errores en los límites naturales entre categorías consecutivas.

Los principales patrones de error identificados fueron:

- **13 observaciones Bajo clasificadas como Medio.**
- **9 observaciones Medio clasificadas como Bajo.**
- **13 observaciones Medio clasificadas como Alto.**
- **13 observaciones Alto clasificadas como Medio.**

Este comportamiento es consistente con un problema de clasificación ordinal, donde las fronteras entre categorías contiguas suelen presentar mayor dificultad que las categorías extremas.

3.3.10. Importancia de Variables

La importancia de variables calculada por LightGBM permitió identificar los factores que más contribuyeron a la predicción de la demanda aeroportuaria.

Las diez variables con mayor relevancia fueron:

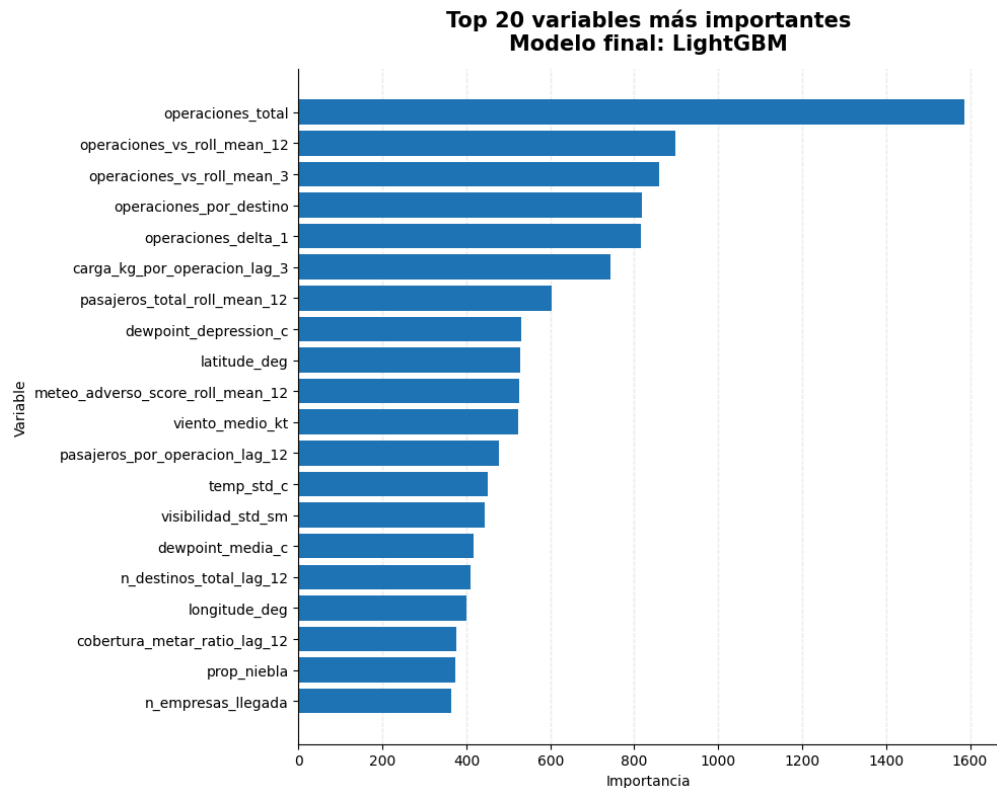


Figura 9. Análisis de la importancia de variables del modelo final.

Los resultados evidencian que las variables relacionadas con la actividad operacional histórica del aeropuerto poseen el mayor poder predictivo. En particular, el número total de operaciones, las desviaciones respecto a medias móviles y las variaciones recientes de tráfico constituyen los principales indicadores utilizados por el modelo.

Adicionalmente, variables meteorológicas como la depresión del punto de rocío (*dewpoint depression*), el índice de meteorología adversa y la intensidad del viento también mostraron una contribución significativa. Esto confirma la influencia que tienen las condiciones climáticas sobre los niveles de operación aeroportuaria.

Finalmente, variables geográficas como la latitud y longitud del aeropuerto aportaron información relevante, sugiriendo la existencia de patrones regionales asociados al comportamiento de la demanda.

3.3.11. Análisis de Confianza de las Predicciones

Con el objetivo de evaluar el grado de certeza del modelo en sus predicciones, se analizaron las probabilidades máximas asignadas a cada observación.

Los resultados muestran una clara diferencia entre los registros correctamente clasificados y aquellos que fueron clasificados de manera incorrecta.

Tabla 62. Análisis de confianza.

Resultado	Cantidad	Confianza Media	Confianza Mínima	Confianza Máxima
Correctos	315	94.01%	50.62%	99.86%
Incorrectos	48	74.51%	50.54%	97.55%

Se observa que las predicciones correctas presentan un nivel de confianza promedio considerablemente superior al de las predicciones erróneas. Esto indica que las probabilidades generadas por LightGBM son coherentes con la calidad de sus decisiones.

Sin embargo, también se identificaron algunos errores realizados con niveles elevados de confianza. Por ejemplo, ciertas observaciones pertenecientes a la categoría Alto fueron clasificadas como Medio con probabilidades superiores al 90%. Este comportamiento sugiere la existencia de registros ubicados cerca de las fronteras de decisión aprendidas por el modelo o posibles patrones atípicos dentro del conjunto de datos.

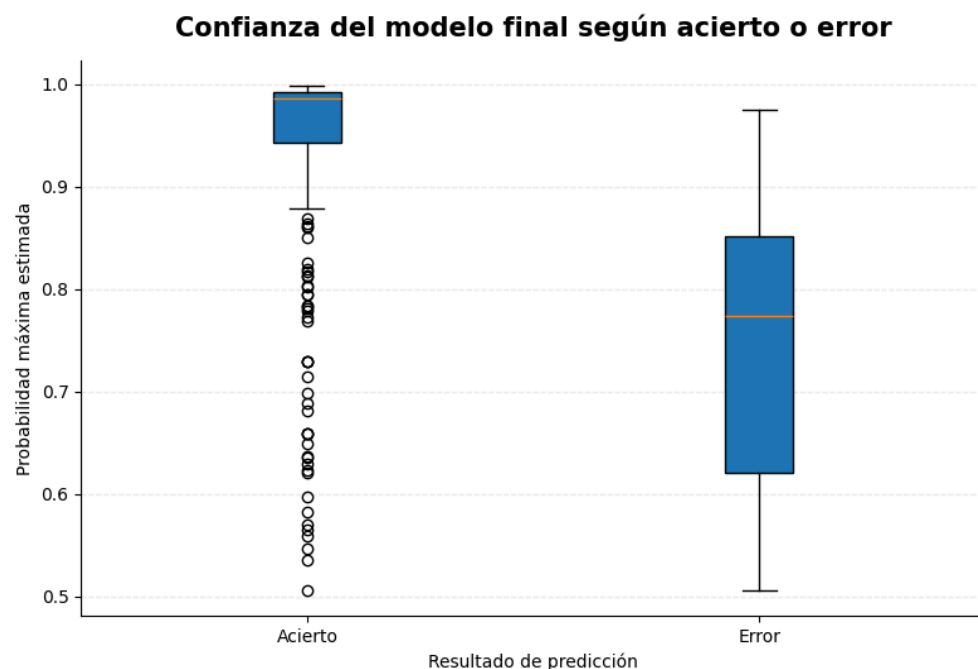


Figura 10. Análisis de confianza del modelo final.

3.3.12. Análisis de Errores

El análisis detallado de los errores permitió identificar los principales tipos de confusión generados por el modelo.

Los errores observados pueden agruparse en cuatro categorías:

Tabla 63. Análisis de errores.

Tipo de error	N	Porcentaje
alto a medio	13	3.5812%
bajo a medio	13	3.5812%
medio a alto	13	3.5812%
medio a bajo	9	2.4793%

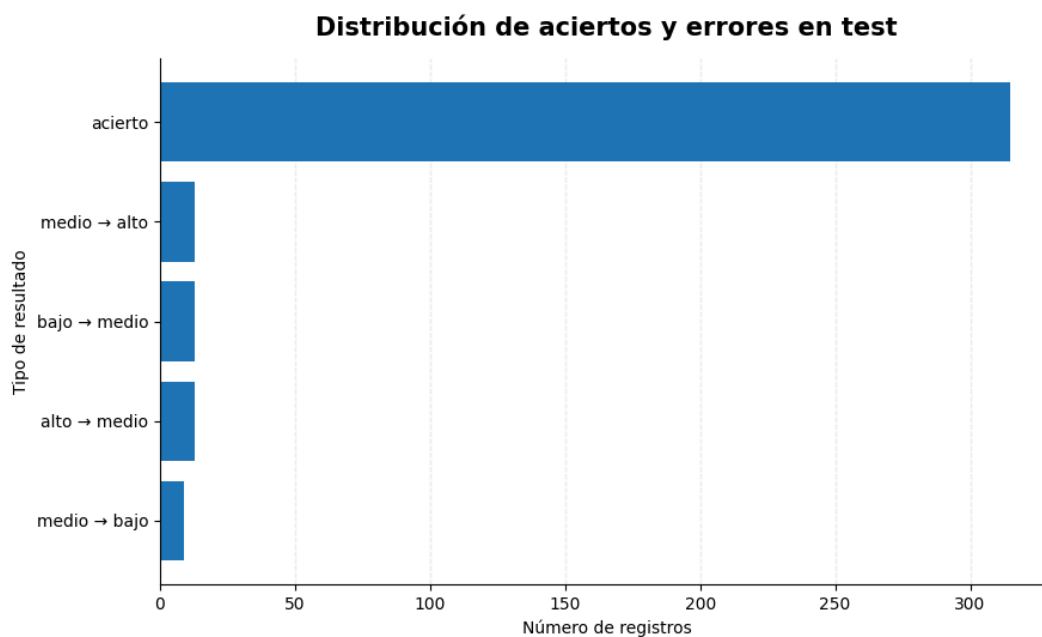


Figura 11. Análisis de errores del modelo final.

Los resultados confirman que la totalidad de los errores ocurre entre categorías adyacentes, lo que representa un comportamiento deseable en problemas de clasificación ordinal. Desde una perspectiva operativa, resulta menos crítico

confundir un nivel Medio con un nivel Alto o Bajo que cometer errores extremos entre categorías opuestas.

La ausencia de clasificaciones directas entre Bajo y Alto demuestra que el modelo logró capturar adecuadamente la estructura subyacente de la demanda aeroportuaria y mantener una separación efectiva entre los escenarios de menor y mayor actividad operacional.

En términos generales, el modelo final presenta un desempeño robusto, equilibrado y consistente, alcanzando niveles de precisión superiores al 86% y mostrando una adecuada capacidad para discriminar entre los distintos niveles de demanda aeroportuaria. Estos resultados respaldan su utilización como herramienta de apoyo para la planificación y gestión de operaciones aeroportuarias.

3.4. Análisis.

3.4.1. Análisis de los Resultados

El objetivo principal de este proyecto consistió en desarrollar un modelo de aprendizaje automático capaz de clasificar el nivel de demanda aeroportuaria en tres categorías: **baja, media y alta**, utilizando información operacional, meteorológica e histórica de los aeropuertos.

Los resultados obtenidos evidencian que el modelo LightGBM seleccionado presenta una elevada capacidad predictiva, alcanzando un **Accuracy de 86.78%**, un **Balanced Accuracy de 86.66%** y un **Macro F1-Score de 86.83%** sobre datos no observados durante el entrenamiento.

La cercanía entre estas métricas demuestra que el modelo mantiene un desempeño equilibrado en las tres categorías analizadas, evitando sesgos significativos hacia alguna clase específica. Esto resulta particularmente importante debido a que las decisiones operativas aeroportuarias requieren una identificación confiable tanto de escenarios de baja actividad como de escenarios de alta demanda.

El análisis de la matriz de confusión permitió identificar que la mayoría de los errores se producen entre categorías contiguas. Por ejemplo, algunas observaciones clasificadas realmente como "media" fueron predichas como "baja" o "alta". Sin embargo, no se observaron errores directos entre las categorías extremas "baja" y "alta", lo que indica que el modelo logra diferenciar adecuadamente escenarios operativos claramente distintos.

Adicionalmente, el análisis de importancia de variables mostró que los factores más relevantes para la predicción corresponden principalmente a variables operacionales históricas, tales como:

- Número total de operaciones.
- Variaciones respecto a medias móviles.
- Operaciones por destino.
- Cambios recientes en el volumen de operaciones.
- Indicadores históricos de pasajeros.

Asimismo, las variables meteorológicas relacionadas con visibilidad, viento, precipitación y condiciones adversas también aportaron información relevante al proceso de clasificación, confirmando la influencia del clima sobre la actividad aeroportuaria.

Por otra parte, el análisis de confianza evidenció que las predicciones correctas presentan una probabilidad media cercana al 94%, mientras que las predicciones erróneas muestran una confianza promedio menor, alrededor del 75%. Este comportamiento sugiere que las probabilidades generadas por el modelo pueden utilizarse como un indicador adicional para apoyar la toma de decisiones.

3.4.2. Cumplimiento de los Objetivos

Los resultados obtenidos permiten concluir que el objetivo general del proyecto fue alcanzado satisfactoriamente.

Se logró:

- Integrar información operacional, meteorológica y geográfica proveniente de múltiples fuentes.
- Realizar un proceso completo de limpieza y preparación de datos.
- Construir variables derivadas capaces de capturar patrones temporales relevantes.
- Evaluar diferentes configuraciones mediante validación cruzada estratificada.
- Optimizar automáticamente los hiperparámetros utilizando Optuna.
- Seleccionar un modelo final con alta capacidad de generalización.
- Obtener métricas superiores al 86% en el conjunto de prueba.

Estos resultados demuestran que la metodología implementada fue adecuada para abordar el problema planteado.

4. TECNOLOGÍA: Ingeniería de Datos y uso de tecnología para:

4.1. Desarrollo del proyecto

El desarrollo del proyecto se realizó sobre Databricks como ambiente principal de trabajo, usando PySpark para el procesamiento distribuido de datos. La elección de este stack responde a la naturaleza del problema: múltiples fuentes heterogéneas, transformaciones encadenadas y un volumen de datos que, aunque

manejable en pandas, se beneficia de la paralelización para las etapas de agregación mensual y construcción de variables derivadas.

El código se organizó en notebooks separados por responsabilidad: uno para ingesta y zona Bronze, uno para limpieza y zona Silver, uno para construcción del dataset Gold, uno para EDA y uno para modelado. Esta separación permite reproducibilidad y facilita la depuración por etapas.

Para el modelado se usó scikit-learn junto con LightGBM y XGBoost, con optimización de hiperparámetros mediante Optuna. La validación se realizó con partición temporal estricta, respetando la secuencia cronológica de los datos para evitar fuga de información entre entrenamiento y prueba.

4.2. Despliegue del proyecto (en un escenario hipotético de implementación real)

En un escenario de producción, el proyecto se desplegaría sobre AWS con una arquitectura batch mensual, coherente con la granularidad del problema. El flujo propuesto sería el siguiente:

Las fuentes externas (IEM, datos.gov.co, OurAirports) serían consultadas automáticamente al cierre de cada mes mediante AWS Lambda o un job programado en AWS Glue. Los datos crudos se depositarían en Amazon S3 en la zona Bronze. Glue ejecutaría las transformaciones de limpieza y agregación para poblar las zonas Silver y Gold. El modelo entrenado, serializado y versionado, residiría en Amazon SageMaker, desde donde se invocaría para generar las clasificaciones mensuales (bajo / medio / alto) por aeropuerto. Los resultados se publicarían en Amazon QuickSight como dashboard de monitoreo operacional, y podrían exponerse adicionalmente mediante una API REST en AWS API Gateway para consumo por parte de sistemas externos como herramientas de planificación de aerolíneas u operadores aeroportuarios.

El reentrenamiento del modelo se programaría de forma periódica (trimestral o semestral) en SageMaker Pipelines, incorporando los nuevos meses de datos disponibles.

4.3. Tener en cuenta:

4.3.1. Fuentes de datos y naturaleza (batch, streaming, transaccional, estructurada vs no estructurada, etc)

El proyecto integra cuatro fuentes de datos con naturalezas distintas:

OurAirports es una fuente estática y estructurada, descargada una única vez en formato CSV. Contiene metadatos geográficos y aeronáuticos de aeropuertos

(coordenadas, códigos ICAO/IATA, tipo de aeropuerto, elevación). Su naturaleza es de referencia dimensional y no requiere actualización frecuente.

Iowa Environmental Mesonet (IEM) — ASOS/METAR es una fuente histórica estructurada, consultada mediante API REST con parámetros de estación y rango temporal. Entrega observaciones meteorológicas horarias o subdiarias en formato CSV. Para este proyecto se trató como fuente batch: se descargó el histórico completo 2020–2025 por estación, con pausas entre solicitudes para respetar los límites operativos del servicio. Los datos son semi-estructurados en origen (texto METAR crudo) pero se extraen en formato tabular con campos numéricos.

datos.gov.co — Operaciones aéreas acumuladas es una fuente pública estructurada, actualizada mensualmente por la Aeronáutica Civil de Colombia. Se consume mediante la API Socrata (OData/SODA) en formato JSON, con paginación. Contiene registros agregados por aeropuerto, mes, tipo de vuelo y tipo de operación. Su naturaleza es transaccional-agregada y batch.

datos.gov.co — Tráfico Origen-Destino comparte la misma plataforma y mecanismo de consumo que la fuente anterior. Contiene información estadística sobre pasajeros y carga por par origen-destino, empresa y período. También es estructurada, batch y de actualización mensual.

En conjunto, todas las fuentes son estructuradas o semi-estructuradas y se procesan bajo un paradigma batch, sin necesidad de procesamiento en tiempo real dada la granularidad mensual del problema.

4.3.2. Ingesta de datos (batch / Streaming)

La ingesta del proyecto es completamente batch. No se requiere procesamiento en tiempo real ni streaming, dado que la unidad de análisis es el par aeropuerto-mes y las fuentes operacionales se actualizan con frecuencia mensual.

En el desarrollo, la ingesta se implementó mediante notebooks PySpark en Databricks que consumen las APIs REST y descargan los archivos directamente a las rutas del lakehouse. Para IEM se implementó una lógica de reintentos con pausas entre solicitudes (backoff) debido a los límites de uso del servicio. Para datos.gov.co se utilizó paginación con la API Socrata, iterando sobre los registros en bloques de 50 000 filas hasta agotar el dataset.

En producción, esta ingesta batch se automatizaría mediante AWS Glue Jobs programados con Amazon EventBridge, ejecutándose al inicio de cada mes para consumir los datos del período anterior. Los archivos resultantes se depositarían en S3 en la zona Bronze, particionados por año y mes, en formato Parquet para eficiencia en consultas posteriores.

4.3.3. Almacenamiento

4.3.3.1. Lagos de datos

El proyecto adopta una arquitectura de data lakehouse organizada en tres zonas o capas, implementada sobre el sistema de archivos de Databricks durante el desarrollo y proyectada sobre Amazon S3 para producción:

La zona Bronze almacena los datos en su estado original, tal como fueron descargados de las fuentes. No se aplica ninguna transformación. Su propósito es garantizar la trazabilidad y permitir reprocesamiento desde el origen ante cualquier error en etapas posteriores.

La zona Silver contiene los datos depurados, homologados y normalizados. En esta capa se resuelven inconsistencias de formato, se homologan los códigos ICAO e IATA mediante la dimensión maestra de OurAirports, se tratan los valores faltantes (por ejemplo, el marcador M de IEM), se desanidan columnas estructuradas como clouds en METAR, y se agregan las observaciones meteorológicas de frecuencia subdiaria a nivel mensual por aeropuerto.

La zona Gold contiene el dataset analítico final, a nivel de granularidad aeropuerto-mes, con todas las variables predictoras construidas y la variable objetivo calculada. Es el insumo directo del modelo de clasificación. En producción, esta capa también alojaría las predicciones generadas mensualmente.

4.3.3.2. Bases de datos

En el desarrollo sobre Databricks, las zonas Silver y Gold se registraron como tablas en el catálogo de Databricks (Unity Catalog), lo que permite consultarlas con SQL estándar desde cualquier notebook del workspace. En producción, las tablas Gold podrían registrarse en el AWS Glue Data Catalog, habilitando su consulta directa desde Amazon Athena sin necesidad de cargar los datos en memoria. Para el almacenamiento de predicciones y metadatos del modelo (versión, métricas, fecha de entrenamiento) se contemplaría el uso de Amazon RDS con motor PostgreSQL, brindando acceso estructurado y transaccional a los resultados operacionales.

4.3.3.3. Archivos

Durante el desarrollo, los datos intermedios y finales se almacenaron en formato Parquet particionado por año y mes, aprovechando la compresión columnar y la eficiencia de lectura parcial. Los datasets de referencia estática (OurAirports) se mantuvieron en CSV. Los modelos entrenados se serializaron en formato Pickle (.pkl) para su reutilización. En producción, los

modelos se versionarían en formato ONNX o joblib dentro de SageMaker Model Registry, y los resultados de predicción se exportarían adicionalmente en CSV para consumo por dashboards o herramientas externas.

4.3.4. Ambiente de procesamiento (desde básicos hasta avanzados como Apache Spark)

El ambiente de procesamiento del proyecto escala desde herramientas básicas hasta frameworks distribuidos, según la etapa del flujo:

En las etapas de exploración y prototipado se utilizó Python con las bibliotecas pandas, matplotlib, seaborn y scikit-learn, ejecutadas en notebooks de Databricks sobre un clúster de un solo nodo.

Para las etapas de ingesta, limpieza e integración de las fuentes se utilizó Apache Spark a través de PySpark, aprovechando el procesamiento distribuido para manejar los más de 550 000 registros de operaciones y las series temporales meteorológicas de 139 estaciones. Las transformaciones de agregación mensual, la construcción de variables de ventana temporal (medias móviles de 3 y 12 meses, deltas) y el cruce entre fuentes se implementaron con la API de DataFrames de Spark y funciones de ventana (Window.partitionBy).

El entrenamiento de los modelos de clasificación se ejecutó sobre el driver de Databricks con scikit-learn, LightGBM y XGBoost. La búsqueda de hiperparámetros se realizó con Optuna en modo secuencial. En producción, este entrenamiento se escalaría a Amazon SageMaker con instancias dedicadas tipo ml.m5.xlarge o superiores, y la búsqueda de hiperparámetros podría paralelizarse mediante SageMaker Hyperparameter Tuning Jobs.

4.3.5. Aplicaciones (visualización, API, archivos, etc)

Visualización: Durante el desarrollo se generaron visualizaciones exploratorias con matplotlib y seaborn directamente en los notebooks. En producción, los resultados del modelo (clasificaciones mensuales por aeropuerto, métricas de desempeño, evolución temporal) se publicarían en un dashboard de Amazon QuickSight, con filtros por aeropuerto, período y nivel de operación predicho versus real.

API: En el escenario de despliegue real, el modelo expuesto en SageMaker estaría disponible como endpoint de inferencia en tiempo de ejecución. Este endpoint se integraría con AWS API Gateway para ofrecer una API REST que permita a sistemas externos consultar la clasificación esperada de uno o varios aeropuertos para el mes siguiente, enviando las variables del mes actual como payload JSON.

Archivos: Los resultados de clasificación se exportarían mensualmente en formato CSV hacia S3, disponibles para descarga directa por parte de equipos de operaciones o analistas que no consuman la API ni el dashboard. Adicionalmente, los reportes de métricas del modelo (accuracy, macro F1, matriz de confusión) se

generarían automáticamente como archivos PDF o HTML tras cada ciclo de reentrenamiento, almacenados en S3 con versionado habilitado.

5. Visualización y Comunicación de Datos

5.1. Requerimientos de Comunicación y Visualización

5.1.1. Audiencia objetivo.

AeroDatos Colombia fue diseñada para dos audiencias simultáneas:

- Audiencia técnica: ingenieros de datos y analistas con conocimiento del pipeline y las variables del dataset. Requiere acceso a múltiples métricas por capa, filtros temporales precisos y series de tiempo detalladas.
- Audiencia de negocio: operadores aeroportuarios, funcionarios de la aeronáutica civil y directivos que necesitan entender la operación, los pasajeros y las condiciones meteorológicas de cada aeropuerto sin necesidad de conocimiento técnico previo.

5.1.2. Pregunta de negocio

La herramienta responde la pregunta: ¿Cómo se distribuyen y evolucionan la operación, los pasajeros, la carga, la conectividad y las condiciones meteorológicas de los aeropuertos colombianos entre 2020 y 2025? Esta pregunta no tiene una respuesta única: cada usuario navega el mapa y selecciona el aeropuerto y la capa que le interesan, construyendo su propia interpretación.

5.1.3. Requerimientos funcionales

- Mapa interactivo con clics reales sobre marcadores: el usuario debe poder seleccionar un aeropuerto haciendo clic directamente en el mapa, no desde un selector externo.
- Cinco capas de análisis independientes: Operación (`operaciones_total`), Pasajeros (`pasajeros_total`), Carga (`carga_total_kg`), Conectividad (`n_destinos_total`, `n_origenes_total`, `n_empresas`) y Clima (`prop_lluvia`, `prop_niebla`, `prop_tormenta`, `prop_baja_visibilidad`, `prop_viento_fuerte`, `prop_ifr_aprox`).
- Submétrica seleccionable por capa: en Clima, el usuario elige qué fenómeno METAR visualizar; en Conectividad, qué métrica de conectividad muestra el mapa.
- Panel de detalle por aeropuerto: al hacer clic, mostrar operaciones, pasajeros, carga y meteorología del aeropuerto seleccionado en el período filtrado.

- Serie temporal integrada: evolución mensual de la métrica activa, a nivel nacional o por aeropuerto seleccionado, con marcación automática del período COVID-19.
- Filtros temporales basados en el rango real del dataset: todo el período, por año, últimos 12/24 meses, o rango personalizado mes a mes.
- Filtros de región y tipo de aeropuerto con etiquetas en español.
- Resumen global cuando no hay aeropuerto seleccionado: aeropuerto líder, total de la métrica y número de aeropuertos activos.

5.1.4. Requerimientos no funcionales

- Portada de presentación antes de ingresar a la herramienta, con imagen SVG generativa embebida y botón de entrada.
- Arquitectura modular en Python: separación clara entre carga de datos, agregaciones, mapa, gráficas, estilos y vistas.
- Coherencia visual: el color de cada capa se propaga de forma consistente al botón selector, la leyenda del mapa y la gráfica de tendencia mediante variables CSS dinámicas.
- Sin dependencias de API keys, servicios de pago ni credenciales externas.
- Despliegue público en Streamlit Community Cloud, actualización automática por push a GitHub.
- Sidebar oculto por defecto (`initial_sidebar_state: collapsed`) para maximizar el espacio del mapa.

5.2. Análisis

5.2.1. Análisis exploratorio visual previo

El diseño de AeroDatos Colombia se fundamentó en el análisis exploratorio realizado en el notebook `eda_aeropuertos_colombia.ipynb`. Los hallazgos clave que guiaron las decisiones de visualización fueron:

- La distribución geográfica de los aeropuertos colombianos es visualmente informativa: los grandes hubs se concentran en los Andes y la costa Caribe, mientras que la Amazonia y el Pacífico tienen aeropuertos pequeños dispersos con características climáticas radicalmente distintas.
- El período COVID-19 (marzo–diciembre 2020) genera una anomalía sistémica en todas las series temporales de operaciones y pasajeros. Sin marcación explícita, cualquier usuario que filtre por 2020 podría interpretar la caída como una característica estructural del aeropuerto y no como un evento externo.
- La distribución de operaciones entre aeropuertos es muy desigual: El Dorado (SKBO) tiene operaciones acumuladas diez veces superiores al aeropuerto mediano. Una escala lineal de tamaño de burbuja hace que todos los aeropuertos excepto Bogotá sean prácticamente invisibles.

- Las variables meteorológicas tienen unidades y escalas muy distintas: las proporciones (prop_lluvia) van de 0 a 1, el viento en kt, la temperatura en Celsius. Cada capa requiere su propia unidad de visualización.
- La cobertura de datos METAR no es uniforme: algunos aeropuertos pequeños tienen pocos registros. Es importante no mostrar valores climáticos como si fueran igualmente confiables para todos los aeropuertos.

5.2.2. Decisiones de diseño derivadas del análisis

- Escala de radio por raíz cuadrada: se aplica $\text{pow}(0.5)$ a los valores antes de normalizar el radio, comprimiendo la diferencia entre Bogotá y los aeropuertos pequeños sin ocultar la variación relativa.
- Leyenda manual en HTML: las leyendas nativas de Folium no son configurables. Se implementaron leyendas personalizadas en HTML inyectadas al mapa mediante `branca.element.Element`.
- Color separado por capa para clima: los fenómenos METAR usan una escala de color fija (azul < 3%, verde 3-8%, amarillo 8-15%, rojo > 15%) mientras que las capas no climáticas usan terciles dinámicos de la vista filtrada.
- Marcación automática de COVID-19: la función `_series_includes_2020()` en `charts/trend.py` verifica si la serie incluye fechas del año 2020 y agrega un rect rojo con etiqueta solo si corresponde.

5.3. Diseño

5.3.1. Principios de diseño aplicados

Tabla 64. Principios de diseño aplicados.

Principio Visualización de Datos	Aplicación concreta en AeroDatos Colombia
Análisis exploratorio	El usuario navega libremente sin narrativa impuesta. El mapa no tiene una respuesta predeterminada: cada usuario construye su interpretación seleccionando capa, período y aeropuerto.
Gestalt: proximidad	El panel de detalle agrupa operaciones, pasajeros, carga y meteorología del aeropuerto seleccionado en un único bloque visual coherente.
Gestalt: similitud	El color de cada capa (azul para Operación, verde para Pasajeros, púrpura para Carga, grafito para Conectividad, cian para Clima) se repite en el botón

Principio Visualización de Datos	Aplicación concreta en AeroDatos Colombia
	selector, la leyenda del mapa y la gráfica de tendencia.
Revelación progresiva	Portada -> explorador con mapa -> clic en aeropuerto -> panel de detalle. Cada nivel revela más información sin abrumar al usuario desde el inicio.
Contexto de capa	Cada capa tiene un título descriptivo y una microdescripción que explican qué codifica el tamaño del punto, qué codifica el color y cuál es la unidad de medida.
Dato aclaratorio temporal	La serie temporal debajo del mapa contextualiza el valor del período seleccionado en la evolución histórica completa, evitando interpretaciones descontextualizadas.

5.3.2. Paleta de colores.

La paleta fue diseñada para que cada capa sea visualmente distinguible entre sí y coherente internamente:

Tabla 65. Paleta de colores.

Capa	Color principal	Uso
Operación	#2563EB (azul)	Botón selector, burbujas del mapa, línea de tendencia
Pasajeros	#10B981 (verde)	Botón selector, burbujas del mapa, línea de tendencia
Carga	#7C3AED (púrpura)	Botón selector, burbujas del mapa, línea de tendencia
Conectividad	#111827 (grafito)	Botón selector, burbujas del mapa, línea de tendencia
Clima	#0891B2 (cian)	Botón selector, burbujas del mapa; color de burbuja según escala METAR

Capa	Color principal	Uso
Escala METAR	Azul → verde → amarillo → rojo	Proporciones METAR: < 3% / 3-8% / 8-15% / > 15%
COVID-19	#EF4444 (rojo)	Franja vertical en series temporales que incluyan 2020

La coherencia de color se implementa mediante variables CSS dinámicas definidas en `apply_active_layer_styles()` en `src/styles.py`. Al cambiar de capa, se actualizan `--active-layer-color` y `--active-layer-soft`, que se propagan automáticamente a botones, selects, tarjetas y bordes sin duplicar CSS.

5.3.3. Arquitectura de la información

La herramienta tiene tres pantallas principales en flujo secuencial:

- Portada (landing): imagen SVG generativa embebida en base64, marca del proyecto, descripción en dos columnas y botón de entrada. Implementada en `src/views/landing.py`.
- Explorador principal (`map_explorer`): encabezado con marca y selector de capas, contexto de la capa activa, layout de tres columnas (filtros | mapa + tendencia | detalle o resumen), y serie temporal integrada debajo del mapa. Implementada en `src/views/map_explorer.py`.

Dentro del explorador principal, la jerarquía visual es:

- Nivel 1: Selector de capas: cinco botones horizontales que determinan qué variable muestra el mapa. El botón de la capa activa se colorea con el color de esa capa.
- Nivel 2: Mapa Folium: ocupa la columna central (4.35/7 del ancho). Los marcadores son burbujas con tamaño proporcional a las operaciones (o a la métrica activa) y color según la capa activa.
- Nivel 3: Panel lateral derecho: cuando no hay aeropuerto seleccionado, muestra resumen global (líder, total, cantidad de aeropuertos). Cuando hay aeropuerto seleccionado, muestra su perfil completo (operaciones, pasajeros, carga, lluvia).
- Nivel 4: Serie temporal (debajo del mapa): evolución mensual de la métrica activa para el aeropuerto seleccionado o a nivel nacional, con vrect de COVID-19 si aplica.

5.3.4. Tipo de dashboard

AeroDatos Colombia es un dashboard de tipo exploratorio-descriptivo. A diferencia de un dashboard analítico con narrativa cerrada, esta herramienta no guía al usuario hacia una conclusión predeterminada: le entrega el control de la capa, el período, la región y el aeropuerto, permitiendo que construya su propio

análisis. Las únicas intervenciones narrativas son el contexto de capa (que explica qué se está viendo) y la marcación del período COVID-19 (que previene interpretaciones erróneas de la caída de 2020).

5.4. Implementación

5.4.1. Tecnología seleccionada

AeroDatos Colombia fue construida con Streamlit como framework principal, Folium + streamlit-folium para el mapa interactivo con clics reales, y Plotly para la serie temporal. La elección de Folium sobre go.Scattermapbox fue deliberada: Folium permite registrar el popup de cada CircleMarker con el código ICAO del aeropuerto, capturar el último popup clicado mediante `st_folium`, y así detectar selecciones de aeropuerto sin JavaScript adicional ni API keys de Mapbox.

5.4.2. Estructura modular del código

Tabla 66. Estructura modular del código.

Módulo	Responsabilidad principal
src/config.py	Rutas de datos (DATA_PATH), configuración de página (PAGE_CONFIG), coordenadas de Colombia y zoom inicial.
src/tokens.py	Paleta de colores (COLORS), escala de colores para valores no climáticos (VALUE_SCALE_COLORS), configuración completa de capas (LAYER_CONFIGS con métrica, unidad, color y regla de agregación), submétrica de clima (CLIMATE_METRICS) y conectividad (CONNECTIVITY_METRICS), y columnas del modelo excluidas de la herramienta (MODEL_COLUMNS_TO_HIDE).
src/data_loader.py	Carga del CSV con <code>@st.cache_data</code> , validación de columnas requeridas (REQUIRED_COLUMNS), conversión de fecha_mes a datetime, eliminación de filas sin coordenadas o código ICAO, y exclusión de columnas del modelo predictivo.
src/aggregations.py	Cinco funciones: <code>apply_filters()</code> para filtrar por período, región y tipo; <code>build_airport_summary()</code> para agregar aeropuerto-mes a una fila por aeropuerto con reglas sum/mean por columna; <code>build_global_summary()</code> para el resumen sin

Módulo	Responsabilidad principal
	selección; nearest_airport() para detección de clic por distancia euclídea con umbral 0.8°; build_timeseries() para la serie temporal nacional o por aeropuerto.
src/maps/folium_map.py	Construcción del mapa con folium.CircleMarker. Incluye _scale_radius() con escala por raíz cuadrada, _value_color() con terciles dinámicos de la vista filtrada, _climate_color() con escala fija para proporciones METAR, _tooltip_html() con HTML personalizado, _add_value_legend() y _add_climate_legend() inyectadas con branca.element.Element.
src/charts/trend.py	Construcción de la serie temporal con go.Scatter y area fill. _series_includes_2020() verifica si la serie cubre 2020 y agrega vrect rojo con etiqueta 'COVID-19 · 2020'. hex_to_rgba() convierte el color de capa a rgba para el área bajo la curva.
src/styles.py	apply_styles() inyecta CSS global con st.markdown. apply_active_layer_styles() define --active-layer-color y --active-layer-soft como variables CSS según la capa activa, propagándose automáticamente a botones, selects y cards.
src/labels.py	Traducciones de tipo de aeropuerto (AIRPORT_TYPE_LABELS) y región (REGION_LABELS) al español. display_month() formatea fechas como 'ene 2020'.
src/formatting.py	format_number() con separador de miles en estilo español. format_compact() con sufijos K/M/B. format_metric() con lógica por unidad (% para proporciones, toneladas para kg). convert_for_visual() convierte proporciones a porcentaje y kg a toneladas para ejes de gráficas.
src/ui/icons.py	SVG_PATHS con íconos inline SVG (plane, users, box, network, cloud, pin, calendar, tower, wind). svg_icon() genera el SVG con color y tamaño parametrizables, sin dependencias de librerías de íconos externas.

Módulo	Responsabilidad principal
src/ui/components.py	render_brand() con enlace de retorno a portada. render_layer_selector() con botones Streamlit por capa. render_layer_context() con título y descripción de la capa activa. render_summary_card() para tarjetas ejecutivas. render_airport_detail() para el panel de aeropuerto seleccionado. render_empty_selection() con instrucción de clic.
src/ui/filters.py	render_filters() con período (todo el período, por año, últimos 12/24 meses, rango personalizado mes a mes), región y tipo de aeropuerto. _render_period_filter() calcula rangos a partir del rango real del dataset, no de la fecha actual.
src/views/landing.py	Renderiza la portada con imagen SVG embebida en base64 (sin rutas externas) y botón de entrada que llama a enter_dashboard().
src/views/map_explorer.py	Vista principal: orquesta filtros, construcción del mapa, detección de clics con _parse_map_click(), actualización de estado con set_selected_airport() y st.rerun(), construcción de la serie temporal y renderizado del panel lateral.
src/state.py	init_state() inicializa active_layer, selected_icao, climate_metric, connectivity_metric y show_landing en st.session_state. Funciones setter para cada variable de estado.
app.py	Punto de entrada: configura la página, aplica estilos, inicializa estado, carga el dataset y redirige entre portada y explorador según st.session_state['show_landing'].

5.4.3. Decisiones técnicas relevantes

- Detección de aeropuerto por clic: st_folium captura last_object_clicked_popup con el código ICAO del popup del CircleMarker. Si el popup no contiene un ICAO válido, nearest_airport() calcula la distancia euclídea entre el último clic (last_clicked) y los aeropuertos del DataFrame filtrado, seleccionando el más cercano si está a menos de 0.8 grados.

- Escala de radio por raíz cuadrada: `_scale_radius()` aplica `values.pow(0.5)` antes de la normalización lineal. Esto reduce la diferencia visual entre El Dorado (operaciones 10x sobre la media) y los aeropuertos pequeños sin hacerlos invisibles.
- Terciles dinámicos para color no climático: `_value_color()` calcula los terciles 33% y 66% de la vista actualmente filtrada, no del dataset completo. Si el usuario filtra solo aeropuertos pequeños, el color sigue siendo informativo y discriminativo.
- Escala fija para color climático: `_climate_color()` usa umbrales absolutos ($< 3\%$, $3-8\%$, $8-15\%$, $> 15\%$) porque las proporciones METAR tienen significado absoluto: 15% de reportes con lluvia es alto en cualquier aeropuerto.
- Variables CSS dinámicas por capa: `apply_active_layer_styles()` redefine `--active-layer-color` y `--active-layer-soft` en cada cambio de capa. Los selectores CSS de botones, cards y selects usan `var(--active-layer-color)` en lugar de un color fijo, propagando el cambio automáticamente.
- Separación de datos del modelo: `MODEL_COLUMNS_TO_HIDE` en `tokens.py` excluye explícitamente `target_nivel_operacion_mes_siguiente` y las columnas del cuartil del target. AeroDatos Colombia es una herramienta descriptiva; mostrar el target confundiría la exploración de datos históricos con predicciones.
- Imagen SVG en base64: la portada usa `_asset_data_uri()` en `landing.py` para convertir el SVG a URI embebida. Esto evita rutas relativas rotas en el despliegue en Streamlit Cloud, donde la carpeta de trabajo puede diferir.
- Filtros temporales relativos al dataset: `_render_period_filter()` calcula rangos como 'últimos 12 meses del dataset' usando la fecha máxima del CSV, no `datetime.today()`. Esto asegura que los filtros sean correctos aunque el dataset no se actualice.

5.4.4. Interactividad implementada

Tabla 67. Interactividad implementada.

Control	Ubicación	Efecto en la herramienta
Botón de entrada	Portada	Cambia <code>show_landing</code> a <code>False</code> y entra al explorador principal.
Selector de capa (5 botones)	Encabezado del explorador	Cambia <code>active_layer</code> en <code>session_state</code> . Actualiza color del botón, leyenda del mapa, línea de tendencia y contexto de capa.

Control	Ubicación	Efecto en la herramienta
Selectbox: período	Panel de filtros (izquierda)	Filtra el DataFrame por rango de fechas. Opciones: todo el período, año específico, últimos 12/24 meses, rango personalizado mes a mes.
Selectbox: región	Panel de filtros (izquierda)	Filtra aeropuertos por departamento con etiquetas en español.
Selectbox: tipo de aeropuerto	Panel de filtros (izquierda)	Filtra entre large_airport, medium_airport y small_airport con etiquetas legibles.
Radio: submétrica de clima	Panel de filtros (izquierda, solo en capa Clima)	Cambia entre Lluvia, Niebla, Tormenta, Baja visibilidad, Viento fuerte e IFR aproximado.
Radio: submétrica de conectividad	Panel de filtros (izquierda, solo en capa Conectividad)	Cambia entre Destinos, Orígenes, Empresas salida y Empresas llegada.
Clic sobre marcador en el mapa	Mapa Folium (centro)	Detectado via popup o nearest_airport(). Actualiza selected_icao en session_state y recarga la vista con el panel de detalle del aeropuerto.
Botón limpiar selección	Panel de detalle (derecha)	Pone selected_icao en None. El panel regresa al resumen global y la serie temporal regresa al nivel nacional.
Marca del encabezado	Encabezado superior	Enlace href='/' que regresa a la portada sin abrir nueva pestaña.

5.5. Validación

5.5.1. Pruebas realizadas

- Coherencia de color: se verificó que el color activo (--active-layer-color) se propague correctamente al botón selector, la leyenda del mapa y la línea de tendencia al cambiar entre las cinco capas.
- Detección de clics: se probó la detección de aeropuerto por popup y por `nearest_airport()` en aeropuertos adyacentes para confirmar que el umbral de 0.8 grados evita selecciones accidentales.
- Series temporales con y sin 2020: se verificó que el vrect de COVID-19 aparece al filtrar periodos que incluyen 2020 y desaparece al filtrar desde 2021 en adelante.
- Filtros temporales: se verificó que 'últimos 12 meses del dataset' calcula el rango desde la fecha máxima del CSV y no desde la fecha actual del sistema.
- Escala de radio: se verificó visualmente que los aeropuertos pequeños son distinguibles en el mapa cuando El Dorado está presente, gracias a la escala por raíz cuadrada.
- Despliegue: se verificó que la imagen SVG de la portada se carga correctamente en Streamlit Cloud usando la ruta base64 embebida.

5.5.2. Limitaciones conocidas

- Streamlit Community Cloud entra en modo sleep después de un período de inactividad.
- La detección de aeropuerto por clic en Folium depende de que el popup contenga exactamente el código ICAO. Si el usuario hace clic en un área sin marcadores, `last_object_clicked_popup` devuelve `None` y `nearest_airport()` intenta la detección por coordenadas, lo cual puede ser impreciso en áreas con aeropuertos muy próximos.
- La arquitectura modular requiere que todas las importaciones relativas sean correctas (`from src.tokens import ...`).

6. Conclusiones generales del Proyecto

6.1. Conclusiones.

A partir del desarrollo completo del proyecto, se concluye que es posible construir una solución analítica para clasificar el nivel mensual esperado de operación aeroportuaria en Colombia utilizando datos abiertos, ingeniería de datos, variables meteorológicas históricas y modelos de Machine Learning supervisado. La unidad de análisis aeropuerto-mes permitió integrar información operacional, comercial, geográfica y meteorológica en una estructura comparable, trazable y adecuada para el entrenamiento de modelos predictivos.

En primer lugar, la construcción del dataset evidenció la importancia de la ingeniería de datos como base del proyecto. Las fuentes utilizadas presentaban diferencias en granularidad, estructura, cobertura y codificación de aeropuertos, por lo que fue necesario implementar un proceso de normalización, agregación mensual y validación de códigos aeroportuarios. La organización por capas bronze, silver y gold permitió conservar la trazabilidad de los datos originales, depurar la información intermedia y consolidar un dataset final apto para análisis exploratorio, visualización y modelado.

En segundo lugar, el análisis exploratorio mostró que el sistema aeroportuario colombiano presenta una alta concentración operacional y comercial. Los aeropuertos principales concentran una proporción significativa de las operaciones y pasajeros; sin embargo, el análisis por aeropuerto-mes permitió observar también comportamientos regionales, variaciones temporales y diferencias relativas que no serían visibles si el estudio se limitara únicamente a volúmenes acumulados. Esta lectura fue fundamental para justificar la construcción de una etiqueta bajo, medio y alto relativa al comportamiento histórico de cada aeropuerto.

En tercer lugar, la cobertura de datos meteorológicos resultó ser un factor metodológico relevante. Las observaciones METAR históricas provenientes de IEM permitieron incorporar variables ambientales al dataset, pero también evidenciaron diferencias de disponibilidad entre aeropuertos. Por esta razón, las variables meteorológicas fueron tratadas como covariables contextuales y no como causas directas del nivel operativo. Este punto es importante porque el proyecto identifica asociaciones útiles para el modelado, pero no desarrolla un diseño de inferencia causal.

En cuarto lugar, la ingeniería de características permitió transformar el dataset mensual en una matriz predictiva más robusta. Las variables temporales, ratios operacionales, rezagos por aeropuerto, promedios móviles, cambios operacionales, variables meteorológicas derivadas y banderas de disponibilidad histórica aportaron información relevante sobre estacionalidad, tendencia,

comportamiento reciente y condiciones contextuales. Esta etapa fue clave para representar no solo el estado actual de cada aeropuerto, sino también su trayectoria histórica.

En quinto lugar, la evaluación de modelos confirmó que los algoritmos basados en ensamblado de árboles fueron los más adecuados para el problema. Los modelos Random Forest, XGBoost y LightGBM superaron ampliamente a la Regresión Logística y a KNN, lo que indica que la clasificación del nivel operativo mensual requiere métodos capaces de capturar relaciones no lineales entre variables operacionales, comerciales, temporales y meteorológicas.

En sexto lugar, la optimización de hiperparámetros mediante Optuna permitió mejorar el rendimiento de los modelos de boosting, especialmente XGBoost y LightGBM. Aunque Random Forest obtuvo resultados muy competitivos durante la validación cruzada interna, el modelo LightGBM optimizado fue seleccionado como modelo final por su equilibrio entre desempeño predictivo, estabilidad, eficiencia computacional y capacidad de generalización sobre datos no observados.

En la evaluación final sobre el conjunto de prueba, el modelo LightGBM optimizado alcanzó un Accuracy de 86,78 %, una Balanced Accuracy de 86,66 % y un Macro F1 de 86,83 %. Estos resultados muestran un desempeño consistente y equilibrado entre las tres clases del problema. Además, la cercanía entre las métricas de validación y prueba indica que el modelo no presentó señales fuertes de sobreajuste y que logró generalizar adecuadamente frente a registros no utilizados durante el entrenamiento.

El análisis de la matriz de confusión permitió identificar que los errores del modelo se concentraron principalmente entre clases adyacentes: bajo-medio y medio-alto. No se observaron errores directos entre las categorías extremas bajo y alto, lo cual es un resultado positivo para un problema de naturaleza ordinal. Esto indica que el modelo aprendió adecuadamente la estructura general del problema y que las confusiones se producen principalmente en los límites naturales entre niveles operativos cercanos.

Desde el punto de vista de interpretación, las variables asociadas al comportamiento histórico de las operaciones fueron las más relevantes para anticipar el nivel operativo futuro. Esto confirma que la dinámica reciente de cada aeropuerto, sus rezagos, sus promedios móviles y sus variaciones operacionales son factores explicativos centrales. Las variables de pasajeros, conectividad y meteorología aportaron información complementaria, enriqueciendo el contexto del modelo y permitiendo una representación más completa del sistema aeroportuario.

En cuanto al componente tecnológico, el uso de Databricks y PySpark permitió estructurar un flujo de procesamiento organizado, reproducible y escalable. La

separación entre ingesta, preparación, construcción del dataset gold, análisis exploratorio, modelado y visualización facilitó la trazabilidad del proyecto y permitió conectar los componentes de ingeniería de datos, aprendizaje automático y comunicación visual dentro de una misma solución analítica.

Finalmente, el dashboard desarrollado en Streamlit permitió complementar el análisis técnico con una herramienta de comunicación interactiva. Esta visualización facilita la exploración de operaciones, pasajeros, carga, conectividad y clima por aeropuerto, período y región. De esta manera, el proyecto no se limita a entrenar un modelo, sino que también propone una forma de comunicar los resultados a usuarios técnicos y no técnicos.

En términos generales, el proyecto demuestra que la integración de datos abiertos aeronáuticos, meteorología histórica, procesamiento distribuido, ingeniería de características y modelos supervisados permite construir una herramienta confiable para apoyar el análisis de demanda y operación aeroportuaria en Colombia. Aunque el alcance es académico, la metodología desarrollada constituye una base sólida para futuros sistemas de monitoreo, planeación operativa y analítica aeroportuaria.

6.2. Limitaciones del proyecto

A pesar de los resultados obtenidos, el proyecto presenta algunas limitaciones que deben considerarse para interpretar adecuadamente sus conclusiones. En primer lugar, el análisis depende de la calidad, disponibilidad y actualización de las fuentes abiertas utilizadas. No todos los aeropuertos colombianos cuentan con la misma cobertura operacional, comercial y meteorológica, por lo que el dataset final se concentró en aeropuertos con información suficientemente consistente.

En segundo lugar, el modelo trabaja con una granularidad mensual. Esto permite capturar patrones agregados de operación, pero no permite analizar dinámicas diarias, horarias, retrasos específicos, cancelaciones, ocupación real por vuelo ni eventos operacionales de corto plazo. Por tanto, las predicciones deben interpretarse como estimaciones de nivel mensual y no como pronósticos operativos en tiempo real.

En tercer lugar, las variables meteorológicas utilizadas corresponden a agregaciones mensuales de observaciones METAR. Esta aproximación permite incorporar contexto ambiental, pero puede suavizar eventos extremos o condiciones de corta duración que podrían afectar operaciones específicas. Por esta razón, la meteorología aporta valor contextual, pero no debe interpretarse como causa directa del comportamiento operacional sin un diseño causal adicional.

En cuarto lugar, la etiqueta bajo, medio y alto se construyó a partir del comportamiento histórico de cada aeropuerto. Esta decisión metodológica es adecuada para comparar cada aeropuerto contra sí mismo, pero implica que las clases no representan necesariamente el mismo volumen absoluto entre

aeropuertos. Un nivel alto en un aeropuerto regional puede corresponder a un número de operaciones menor que un nivel bajo en un aeropuerto principal.

Por último, aunque el modelo final obtuvo un desempeño sólido en prueba, su uso en un escenario real requeriría monitoreo continuo, actualización periódica de datos, evaluación de deriva, validación con usuarios expertos y mecanismos de explicación del modelo para apoyar decisiones operativas de manera responsable.

6.3. Trabajo futuro

Como líneas de trabajo futuro se propone, en primer lugar, incorporar variables económicas, turísticas y territoriales que puedan influir en la demanda aeroportuaria. Indicadores como ocupación hotelera, flujo turístico, actividad económica regional, eventos masivos, población, conectividad terrestre o temporadas comerciales podrían enriquecer el modelo y mejorar la explicación de variaciones mensuales.

En segundo lugar, se recomienda evaluar modelos específicamente diseñados para series temporales y clasificación ordinal. Aunque los modelos utilizados lograron buenos resultados, enfoques como modelos secuenciales, LSTM, Transformers temporales o modelos ordinales podrían capturar dependencias más complejas entre meses y respetar de forma más explícita el orden natural de las clases bajo, medio y alto.

En tercer lugar, sería útil explorar predicción probabilística. En lugar de entregar únicamente una clase final, el sistema podría reportar la probabilidad asociada a cada nivel operativo. Esto permitiría cuantificar la incertidumbre de cada predicción y apoyar decisiones diferenciadas cuando el modelo se encuentre cerca del límite entre dos categorías.

En cuarto lugar, se propone implementar un pipeline de actualización periódica del modelo. En un escenario real, los datos podrían actualizarse mensualmente, reconstruir las capas bronze, silver y gold, generar nuevas predicciones y monitorear el desempeño del modelo frente a los datos observados más recientes.

En quinto lugar, se recomienda ampliar el dashboard para incluir resultados del modelo predictivo. La herramienta actual puede evolucionar hacia una plataforma de monitoreo analítico donde se visualicen predicciones, clases reales, errores, confianza del modelo, importancia de variables y evolución histórica por aeropuerto.

En sexto lugar, se sugiere incorporar mecanismos de interpretabilidad más avanzados, como SHAP values, análisis local por aeropuerto y explicación de predicciones individuales. Esto permitiría entender por qué el modelo clasifica un aeropuerto-mes como bajo, medio o alto, aumentando la transparencia y utilidad de la solución para usuarios técnicos y de negocio.

Finalmente, en una implementación productiva, se recomienda establecer controles de calidad de datos, versionamiento de datasets, versionamiento de modelos, monitoreo de deriva, auditoría de predicciones y evaluación periódica del desempeño. Estas mejoras permitirían convertir el proyecto académico en una solución más robusta para apoyar procesos reales de planeación aeroportuaria, análisis regional y toma de decisiones basada en datos.

7. Referencias.

- [1] P. Chapman, J. Clinton, R. Kerber, T. Khabaza, T. Reinartz, C. Shearer y R. Wirth, *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc., 2000. Disponible en: <https://public.dhe.ibm.com/software/analytics/spss/documentation/modeler/14.2/es/CRISP-DM.pdf>. Consultado el 1 de mayo de 2026.
- [2] Aeronáutica Civil de Colombia, “Operaciones aéreas acumuladas en Colombia,” *Datos Abiertos Colombia*. Disponible en: <https://www.datos.gov.co/Transporte/Operaciones-a-reas-acumuladas-en-Colombia/jh8x-n6h6>. Consultado el 3 de mayo de 2026.
- [3] Aeronáutica Civil de Colombia, “Transporte Aéreo Comercial — Tráfico Origen-Destino,” *Datos Abiertos Colombia*. Disponible en: <https://www.datos.gov.co/Transporte/Transporte-A-reo-Comercial-Tr-fico-Origen-Destino-gb6w-ynu4>. Consultado el 3 de mayo de 2026.
- [4] Datos Abiertos Colombia, “Portal Nacional de Datos Abiertos.” Disponible en: <https://www.datos.gov.co/>. Consultado el 4 de mayo de 2026.
- [5] OurAirports, “Open data downloads.” Disponible en: <https://ourairports.com/data/>. Consultado el 5 de mayo de 2026.
- [6] OurAirports, “Airports in Colombia — airports.csv.” Disponible en: <https://ourairports.com/countries/CO/airports.csv>. Consultado el 5 de mayo de 2026.
- [7] Iowa State University, Iowa Environmental Mesonet, “Download ASOS/AWOS/METAR Data.” Disponible en: <https://mesonet.agron.iastate.edu/request/download.phtml>. Consultado el 7 de mayo de 2026.
- [8] Iowa State University, Iowa Environmental Mesonet, “Documentation on /cgi-bin/request/asos.py.” Disponible en: <https://mesonet.agron.iastate.edu/cgi-bin/request/asos.py?help=>. Consultado el 7 de mayo de 2026.
- [9] Iowa State University, Iowa Environmental Mesonet, “IEM API Documentation.” Disponible en: <https://mesonet.agron.iastate.edu/api/>. Consultado el 8 de mayo de 2026.
- [10] National Weather Service Aviation Weather Center, “Data API.” Disponible en: <https://aviationweather.gov/data/api/>. Consultado el 9 de mayo de 2026.
- [11] National Weather Service Aviation Weather Center, “METAR and TAF Data.” Disponible en: <https://aviationweather.gov/data/metar/>. Consultado el 9 de mayo de 2026.

- [12] Databricks, “What is the medallion lakehouse architecture?” *Databricks Documentation*. Disponible en: <https://docs.databricks.com/aws/en/lakehouse/medallion>. Consultado el 10 de mayo de 2026.
- [13] Databricks, “What is Medallion Architecture?” *Databricks Blog*. Disponible en: <https://www.databricks.com/blog/what-is-medallion-architecture>. Consultado el 10 de mayo de 2026.
- [14] Apache Software Foundation, “pyspark.sql.DataFrame,” *Apache Spark Documentation*. Disponible en: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.html>. Consultado el 12 de mayo de 2026.
- [15] Apache Software Foundation, “PySpark Documentation,” *Apache Spark Documentation*. Disponible en: <https://spark.apache.org/docs/latest/api/python/index.html>. Consultado el 12 de mayo de 2026.
- [16] scikit-learn developers, “Metrics and scoring: quantifying the quality of predictions,” *scikit-learn documentation*. Disponible en: https://scikit-learn.org/stable/modules/model_evaluation.html. Consultado el 14 de mayo de 2026.
- [17] scikit-learn developers, “User Guide,” *scikit-learn documentation*. Disponible en: https://scikit-learn.org/stable/user_guide.html. Consultado el 14 de mayo de 2026.
- [18] Microsoft Corporation, “lightgbm.LGBMClassifier,” *LightGBM Documentation*. Disponible en: <https://lightgbm.readthedocs.io/en/latest/pythonapi/lightgbm.LGBMClassifier.html>. Consultado el 16 de mayo de 2026.
- [19] Microsoft Corporation, “Python API,” *LightGBM Documentation*. Disponible en: <https://lightgbm.readthedocs.io/en/latest/Python-API.html>. Consultado el 16 de mayo de 2026.
- [20] T. Chen y C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” en *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. doi: 10.1145/2939672.2939785.
- [21] XGBoost Developers, “XGBoost Documentation.” Disponible en: <https://xgboost.readthedocs.io/>. Consultado el 18 de mayo de 2026.
- [22] XGBoost Developers, “Python API Reference,” *XGBoost Documentation*. Disponible en: https://xgboost.readthedocs.io/en/latest/python/python_api.html. Consultado el 18 de mayo de 2026.

- [23] Optuna Developers, “Optuna: A hyperparameter optimization framework,” *Optuna Documentation*. Disponible en: <https://optuna.readthedocs.io/>. Consultado el 20 de mayo de 2026.
- [24] Optuna Developers, “Optuna — A hyperparameter optimization framework.” Disponible en: <https://optuna.org/>. Consultado el 20 de mayo de 2026.
- [25] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. Disponible en: <https://jmlr.org/papers/v12/pedregosa11a.html>. Consultado el 21 de mayo de 2026.
- [26] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001. doi: 10.1023/A:1010933404324.
- [27] P. Geurts, D. Ernst y L. Wehenkel, “Extremely randomized trees,” *Machine Learning*, vol. 63, pp. 3–42, 2006. doi: 10.1007/s10994-006-6226-1.
- [28] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” en *Advances in Neural Information Processing Systems*, vol. 30, 2017. Disponible en: https://papers.nips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html. Consultado el 23 de mayo de 2026.
- [29] Streamlit, “Streamlit documentation.” Disponible en: <https://docs.streamlit.io/>. Consultado el 24 de mayo de 2026.
- [30] Streamlit, “API reference,” *Streamlit Documentation*. Disponible en: <https://docs.streamlit.io/develop/api-reference>. Consultado el 24 de mayo de 2026.
- [31] Streamlit, “st.plotly_chart,” *Streamlit Documentation*. Disponible en: https://docs.streamlit.io/develop/api-reference/charts/st.plotly_chart. Consultado el 25 de mayo de 2026.
- [32] Streamlit-Folium, “streamlit-folium documentation.” Disponible en: <https://folium.streamlit.app/>. Consultado el 26 de mayo de 2026.
- [33] Plotly Technologies Inc., “Plotly Python Graphing Library.” Disponible en: <https://plotly.com/python/>. Consultado el 27 de mayo de 2026.
- [34] Python Software Foundation, “The Python Language Reference.” Disponible en: <https://docs.python.org/3/reference/>. Consultado el 28 de mayo de 2026.
- [35] The pandas development team, “pandas documentation.” Disponible en: <https://pandas.pydata.org/docs/>. Consultado el 29 de mayo de 2026.
- [36] NumPy Developers, “NumPy documentation.” Disponible en: <https://numpy.org/doc/>. Consultado el 30 de mayo de 2026.

[37] Matplotlib Developers, “Matplotlib documentation.” Disponible en: <https://matplotlib.org/stable/index.html>. Consultado el 1 de junio de 2026.